1 Appendix U: Agent Semantics

U.1. Scope and platform status

This appendix defines the semantics of agents used in the search, platform, proof-management, and audit layers of the AK framework. It belongs to the Search / Platform Appendices. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

Agent output is not proof unless verified by the declared verifier-side protocol.

In particular, no human proposal, AI-generated answer, search score, confidence value, ranking, heuristic, proof sketch, platform artifact, or stored object may replace:

$$B\text{-Gate}^{+}, \qquad \mathrm{Eval}_{1,W,\tau}(F) = 0, \qquad C_{\tau,W}F, \qquad \phi_{i,W,\tau},$$
$$(\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}) = (0,0), \qquad \mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_{\tau}(\mathfrak{B}).$$

Remark 1.1 (Search / Platform status)**.** This appendix describes who or what may generate, transform, inspect, verify, store, or audit objects in the AK workflow. It does not create new mathematical theorems.

Remark 1.2 (Relation to Core sovereignty)**.** Chapter 7 established Core sovereignty. The same rule governs this appendix: agents may propose, navigate, annotate, package, or criticize data, but only verifier-side Core conditions determine Core pass/reject.

Remark 1.3 (Why agent semantics are needed)**.** The AK framework is search-heavy and platform-heavy. It may involve human judgment, AI generation, automated computation, proof assistants, artifact stores, and certificate graphs. Without explicit agent semantics, candidate outputs can be accidentally inflated into claims. This appendix prevents that inflation.

U.1bis. v19.0 agent sovereignty and verifier-side boundary

Definition 1.4 (Agent-sovereignty boundary)**.** The *agent-sovereignty boundary* is the rule that no agent-side object, action, score, artifact, approval, storage event, or generated proof text changes a Core verdict unless it is converted into declared verifier-side data and passes the applicable verifier-side protocol. The boundary applies uniformly to human agents, AI agents, tool agents, proof assistants, platform services, and hybrid workflows.

Definition 1.5 (Verifier-side conversion record)**.** A *verifier-side conversion record* is a record converting an agent-side output into a verifier-side object. It must declare:

(i) the source agent output and its status label;

(ii) the target verifier-side schema;

(iii) the source and target mathematical scopes;

(iv) the authority allowed to check the conversion;

(v) all required hypotheses, artifacts, environments, versions, and dependency identifiers;

(vi) the resulting verifier-side status: pass, reject, or undefined;

(vii) the failure route when the conversion is missing, ill-typed, stale, or unsupported.

A conversion record is not optional when an agent output is consumed as evidence.

Definition 1.6 (Agent-side default status)**.** Every agent output is proposer-side and non-verdict by default until a status label, authority scope, artifact identity, and verifier-side conversion record justify a stronger role. This default applies even if the output is written in theorem style, generated by a strong model, approved by a human, stored in the platform, or produced by a deterministic tool.

Proposition 1.7 (No agent sovereignty override)**.** *No agent-side object or action overrides the Core verifier-side requirements.*

Proof. The v19 Core verdict consumes repaired evaluation records, representatives, eligibility records, tower artifacts, diagnostics, comparison records, ledgers, gate verdicts, and manifests at their declared types. Agent-side objects have provenance and workflow semantics, not verifier-side status by default. Without a verifier-side conversion record, they are outside the verdict function. Therefore they cannot override the Core requirements. □

Proposition 1.8 (Agent success is non-compensating)**.** *A successful agent action, including generation, replay, retrieval, ranking, formal checking, platform storage, or human approval, cannot compensate for a failed or missing non-scalar obligation.*

Proof. Non-scalar obligations such as actual morphisms, representative existence, eligibility, terminal tameness, manifest identity, and artifact availability have their own typed requirements. Agent success in another role does not supply those requirements. Since the v19 obligation calculus is non-compensating, no favorable agent-side result can repair a missing non-scalar obligation. □

Remark 1.9 (Agent-semantics scope)**.** This appendix governs workflow semantics only. It does not rank agents by epistemic authority and does not assert that any class of agent is inherently reliable or unreliable. Reliability is established only relative to a declared task, authority scope, environment, and verifier-side evidence.

U.2. Agent types

Definition 1.10 (Agent)**.** An *agent* is any human, AI system, automated tool, proof assistant component, search module, platform service, or hybrid workflow component that performs an action in the AK search, platform, proof-management, or audit pipeline.

Definition 1.11 (Human agent)**.** A *human agent* is a person who proposes, reviews, edits, verifies, audits, approves, rejects, or interprets AK-related objects or claims.

Definition 1.12 (AI agent)**.** An *AI agent* is a machine-learning, language-model, symbolic, neuro-symbolic, or automated reasoning component that proposes, transforms, summarizes, searches, ranks, critiques, or assists with AK-related objects or claims.

Definition 1.13 (Tool agent)**.** A *tool agent* is a deterministic or semi-deterministic software component, such as a persistence-computation engine, proof assistant, type checker, parser, build system, replay engine, artifact validator, dependency resolver, or certificate checker.

Definition 1.14 (Platform agent)**.** A *platform agent* is a service that stores, versions, indexes, retrieves, displays, routes, or synchronizes AK artifacts, proof objects, logs, manifests, or certificate records.

Definition 1.15 (Hybrid agent)**.** A *hybrid agent* is a workflow in which human, AI, tool, and platform agents jointly produce, transform, inspect, verify, or audit an object.

Remark 1.16 (Agent type does not determine validity)**.** The type of agent that produced an object does not determine the object's proof status. A human-written claim may be false. An AI-generated lemma may be correct. A tool-generated artifact may be misconfigured. A stored artifact may be obsolete. Validity depends on verifier-side checks, not on authorship.

U.3. Agent roles

Definition 1.17 (Proposer)**.** A *proposer* is an agent that suggests a candidate object, lemma, bridge, window, threshold, realization, proof route, diagnostic interpretation, certificate record, or proof object field.

Definition 1.18 (Searcher)**.** A *searcher* is an agent that explores a search space and returns candidate regions, structures, windows, bridges, proof fragments, counterexamples, or artifact references.

Definition 1.19 (Mapper)**.** A *mapper* is an agent that organizes candidate objects into maps, diagrams, dependency graphs, validity maps, bridge maps, or certificate DAGs.

Definition 1.20 (Lifter)**.** A *lifter* is an agent that attempts to turn a candidate, advisory object, search artifact, or platform artifact into a Core-readable object, interface object, formal fragment, or certificate candidate.

Definition 1.21 (Verifier)**.** A *verifier* is an agent or procedure that checks whether a declared object, formal fragment, bridge, certificate record, gate, diagnostic, ledger, or proof object satisfies the relevant verifier-side criteria.

Definition 1.22 (Auditor)**.** An *auditor* is an agent that checks whether the verification process, artifact references, manifest entries, proof object schema, provenance records, and replay data are sufficient for independent reconstruction.

Definition 1.23 (Curator)**.** A *curator* is an agent that stores, labels, versions, indexes, maintains, or retires proof objects, artifacts, status labels, and platform records.

Definition 1.24 (Adjudicator)**.** An *adjudicator* is an agent that resolves status conflicts among outputs, for example when one agent labels an object as a candidate while another reports a failure. Adjudication does not establish mathematical validity unless backed by verifier-side data.

Remark 1.25 (Roles may overlap)**.** A single agent may act in multiple roles. For example, an AI system may propose a bridge, a human may edit it, and a proof assistant may verify a formal fragment. However, the semantic role of each action must remain distinguishable.

U.4. Proposer–verifier separation

Definition 1.26 (Proposer-side data)**.** *Proposer-side data* are outputs generated before verifier-side validation. They include:

(i) candidate lemmas;

(ii) candidate proof routes;

(iii) advisory indicators;

(iv) search rankings;

(v) AI confidence values;

(vi) validity maps;

(vii) bridge program sketches;

(viii) informal explanations;

(ix) draft certificate records;

(x) candidate normalization or realization routes;

(xi) candidate [Spec] bridges;

(xii) proposed proof object fields.

Definition 1.27 (Verifier-side data)**.** *Verifier-side data* are the data used to determine verified status, Core pass/reject, or audit-readable acceptance. They include:

(i) declared objects;

(ii) declared windows W;

(iii) threshold and collapse policies;

(iv) repaired evaluation records

$$\mathrm{Eval}_{i,W,\tau}(F) = T_\tau^{\mathrm{bar}}\big(\mathsf{W}_W \mathbf{P}_i(F)\big);$$

(v) admissible representative records

$$C_{\tau,W} F$$

when invoked;

(vi) eligibility records when the Ext^1-clause is invoked;

(vii) declared tower comparison artifacts

$$\phi_{i,W,\tau} : \mathrm{ColimProfile}_{i,W,\tau}(F_\bullet) \to \mathrm{ApexProfile}_{i,W,\tau}(F_\infty)$$

when Type IV diagnostics are invoked;

(viii) monitored diagnostics;

(ix) gate verdicts;

(x) ledger policy and budget comparison

$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B});$$

(xi) artifact references;

(xii) proof objects satisfying the Minimal Manifest Axiom;

(xiii) formal or reconstructible checks of required claims.

Theorem 1.28 (Proposer–verifier separation)**.** *Core pass/reject is determined only by verifier-side data. Proposer-side data may guide, motivate, or prioritize verification, but they do not determine the verdict.*

Proof. By Chapter 7 and Appendix G, Core verdicts are determined by repaired evaluation records, admissible representative and eligibility records when invoked, declared tower comparison artifacts when invoked, gate verdicts, monitored diagnostics, ledger comparison, and audit-readable manifest data. Proposer-side data are not part of this verifier-side rule. Therefore proposer-side data cannot determine Core pass/reject. □

Remark 1.29 (The central safety boundary)**.** This theorem is the central safety boundary of Part V. Search may be aggressive. Agents may be creative. The verifier remains conservative.

U.4bis. Promotion, adjudication, and claim-use preflight

Definition 1.30 (Agent promotion certificate)**.** An *agent promotion certificate* is the evidence package required to promote an agent output to a stronger status. It contains the source output, prior status, target status, verifier-side conversion record, claim-use preflight result, dependency closure, artifact identity, and failure-route mapping.

Definition 1.31 (Agent claim-use preflight)**.** An *agent claim-use preflight* checks whether an agent output may be consumed at a proposed role. It verifies identity, status label, document role, evidence status, invocation role, authority scope, runtime comparison mode if any, dependency closure, artifact freshness, and prohibited stronger readings. Missing preflight data yield undefined, not pass.

Definition 1.32 (Adjudication record)**.** An *adjudication record* resolves a conflict among agent outputs, status labels, or review results. It records the conflicting claims, the adjudication rule, the selected status, the losing statuses, the evidence actually consumed, and whether the resolution is operational, verifier-side, or non-verdict.

Proposition 1.33 (Adjudication is not verification)**.** *An adjudication record does not establish mathematical validity unless it includes the verifier-side evidence required for the adjudicated claim.*

Proof. Adjudication resolves a workflow conflict. Verification checks a typed mathematical or certificate condition. Resolving which status label is currently assigned does not by itself prove the underlying mathematical condition. Thus adjudication is not verification unless it carries the required verifier-side evidence. □

Proposition 1.34 (Promotion without preflight is invalid)**.** *A status promotion from candidate, draft, advisory, search artifact, proof sketch, bridge draft, or platform artifact to theorem evidence, bridge theorem, Core verified, or audit-ready status is invalid without a successful claim-use preflight and the required promotion certificate.*

Proof. Higher statuses impose additional evidence, scope, artifact, and role requirements. If preflight is missing, those requirements are not known to be satisfied. By the weaker-safe rule, the promotion cannot be consumed. □

Remark 1.35 (Human approval and adjudication)**.** Human approval may be part of adjudication or governance. It remains scoped to the approved purpose and does not itself become the proof of a mathematical claim.

U.5. AI confidence, ranking, and plausibility

Definition 1.36 (AI confidence value)**.** An *AI confidence value* is any scalar, categorical, probabilistic, linguistic, or model-generated estimate of plausibility, correctness, priority, or relevance associated with an AI-generated output.

Definition 1.37 (Agent ranking)**.** An *agent ranking* is an ordering of candidate objects, regions, lemmas, proof routes, bridge statements, or artifacts produced by an agent.

Definition 1.38 (Plausibility score)**.** A *plausibility score* is any non-verifier-side score estimating whether a candidate is worth checking. It may be human-assigned, AI-assigned, tool-assisted, or platform-derived.

Proposition 1.39 (AI confidence is not proof)**.** *An AI confidence value is not a proof, not a Core gate verdict, not a monitored diagnostic, and not a budget comparison.*

Proof. A proof or Core verdict must be reconstructible through declared verifier-side conditions. An AI confidence value is a model-side estimate. It does not establish B-Gate$^+$, repaired evaluation vanishing, eligibility, Type IV cleanliness, tower artifact validity, or ledger admissibility. □

Proposition 1.40 (Ranking does not imply validity)**.** *The rank assigned to a candidate by an agent does not imply that the candidate is valid.*

Proof. Ranking is a search or prioritization operation. Validity requires verification. A highly ranked candidate may fail a gate, diagnostic, ledger, eligibility, artifact, or bridge condition. □

Proposition 1.41 (Plausibility does not imply verifiability)**.** *A plausible candidate is not necessarily verifiable.*

Proof. Plausibility concerns apparent promise. Verifiability requires typed objects, declared criteria, artifact support, and reconstructible checks. A candidate may appear plausible while lacking the data needed for verification. □

Remark 1.42 (Correct use of confidence)**.** AI confidence, rankings, and plausibility scores may be useful for triage. They may determine what is checked first. They may not determine what is accepted.

U.5bis. Retrieval, consensus, and model-context boundaries

Definition 1.43 (Retrieval context record)**.** A *retrieval context record* stores the corpus, query, retrieved artifact identifiers, locator ranges, freshness policy, ranking policy, and extraction method used to ground an agent output. It is a provenance record, not a proof record.

Definition 1.44 (Model-context boundary)**.** The *model-context boundary* is the distinction between a model's generated statement and the artifact-backed statements available in the declared context. A generated statement is not source evidence unless it is tied to an immutable source locator or a verifier-side proof object.

Definition 1.45 (Agent consensus record)**.** An *agent consensus record* records agreement among multiple agents, models, tools, reviewers, or search procedures. It must identify the agents, prompts or tasks, independence assumptions, compared outputs, agreement predicate, and unresolved disagreements.

Proposition 1.46 (Retrieval is not verification)**.** *Retrieving a document, passage, theorem statement, citation, or artifact does not verify the mathematical claim for which it is retrieved.*

Proof. Retrieval supplies access or evidence candidates. Verification checks whether the retrieved item has the correct statement, hypotheses, scope, proof, status, and permitted use for the target obligation. The retrieval action alone performs none of those checks. □

Proposition 1.47 (Agent consensus is not proof)**.** *Agreement among agents, including agreement among multiple AI systems or human reviewers, is not proof and does not determine a Core verdict.*

Proof. Consensus is a proposer-side or governance-side correlation among outputs. All agreeing outputs may share the same unsupported premise, stale source, prompt error, or scope mistake. The Core verdict depends on verifier-side records, not on the number of agreeing agents. □

Proposition 1.48 (Context inclusion is not claim adoption)**.** *A statement included in an agent's prompt, retrieved context, memory, note, or platform page is not adopted as a theorem claim unless it is independently registered and consumed at an admissible status.*

Proof. Context inclusion affects what an agent may read or generate. Claim adoption requires role, evidence status, invocation role, source binding, and permitted use. Therefore context inclusion cannot itself promote a statement. □

Remark 1.49 (Stale or partial context)**.** A retrieved or remembered context may be stale, partial, or superseded. The current source artifact and claim-governance record control over the agent's context window.

U.6. Agent outputs and status labels

Definition 1.50 (Agent output)**.** An *agent output* is any object produced by an agent, including text, code, theorem statements, diagrams, proof sketches, formal fragments, data files, certificate candidates, maps, rankings, annotations, logs, or status updates.

Definition 1.51 (Agent-output status label)**.** Every agent output used in the AK workflow must be assigned one of the following status labels:

(i) *search artifact*;

(ii) *candidate*;

(iii) *draft*;

(iv) *advisory*;

(v) *operational policy*;

(vi) *interface specification*;

(vii) [Spec];

(viii) *formal fragment*;

(ix) *verified formal fragment*;

(x) *proof sketch*;

(xi) *bridge draft*;

(xii) *bridge theorem*;

(xiii) *Core certificate candidate*;

(xiv) *Core verified*;

(xv) *audit-ready*;

(xvi) *non-claim*;

(xvii) *rejected*;

(xviii) *incomplete*;

(xix) *deprecated*;

(xx) *superseded*.

Definition 1.52 (Unlabeled agent output)**.** An *unlabeled agent output* is an agent output lacking an explicit status label.

Definition 1.53 (Status promotion)**.** *Status promotion* is an operation that moves an output from a lower-status class to a higher-status class, for example:

$$\text{candidate} \longrightarrow \text{Core certificate candidate} \longrightarrow \text{Core verified}.$$

Definition 1.54 (Status demotion)**.** *Status demotion* is an operation that moves an output to a lower or safer status class, for example:

$$\text{Core certificate candidate} \longrightarrow \text{incomplete} \quad \text{or} \quad \text{verified formal fragment} \longrightarrow \text{deprecated}.$$

Proposition 1.55 (Unlabeled outputs are non-verdict by default)**.** *An unlabeled agent output is non-verdict data by default.*

Proof. Without a status label, an auditor cannot determine whether the output is intended as a search artifact, theorem, proof fragment, policy, bridge, certificate, or non-claim. Therefore it cannot affect Core verdicts. □

Proposition 1.56 (Promotion requires evidence)**.** *An agent output may be promoted to a higher status only if the required evidence for that higher status is supplied.*

Proof. Higher status labels impose stronger semantic requirements. For example, Core verified status requires verifier-side data, and bridge theorem status requires a proved bridge. Without such evidence, the promotion is unsupported. □

Remark 1.57 (Labeling prevents claim inflation)**.** Status labels prevent a search artifact from being misread as a theorem, or a draft from being misread as a verified proof.

U.7. Agent action semantics

Definition 1.58 (Generate)**.** An agent action *generate* produces a new candidate object or artifact. Generation does not imply verification.

Definition 1.59 (Transform)**.** An agent action *transform* modifies an existing object into another form, such as translating, normalizing, refactoring, formalizing, compressing, expanding, or summarizing it. Transformation does not preserve validity unless preservation is proved or checked.

Definition 1.60 (Rank)**.** An agent action *rank* orders candidates by a declared priority function, heuristic, score, or policy. Ranking does not imply proof.

Definition 1.61 (Annotate)**.** An agent action *annotate* adds metadata, comments, explanations, warnings, or status labels to an object. Annotation does not change the mathematical content unless explicitly declared.

Definition 1.62 (Verify)**.** An agent action *verify* checks a claim, object, formal fragment, bridge, or certificate against declared verifier-side criteria. Verification must specify the criteria being checked.

Definition 1.63 (Audit)**.** An agent action *audit* checks whether the verification record, artifacts, manifest, provenance, replay data, and proof object schema allow independent reconstruction.

Definition 1.64 (Replay)**.** An agent action *replay* re-executes, recomputes, or reconstructs a declared workflow from stored artifacts and manifests. Replay checks reproducibility, not necessarily mathematical truth beyond the replayed criteria.

Proposition 1.65 (Action semantics are non-interchangeable)**.** *Generation, transformation, ranking, annotation, verification, audit, and replay are distinct actions and cannot be substituted for one another without explicit justification.*

Proof. Each action has a different semantic effect. Generation creates candidates. Ranking prioritizes. Verification checks validity. Audit checks reconstructibility. Replay checks reproducibility. Since their outputs answer different questions, they cannot be interchanged. □

Remark 1.66 (Transformation caution)**.** A transformed proof, transformed definition, or transformed certificate may no longer be equivalent to the source object. Equivalence requires a preservation check or proof.

U.7bis. Transformation preservation and semantic diff

Definition 1.67 (Transformation preservation certificate)**.** A *transformation preservation certificate* proves or verifies that a transformation preserves the declared mathematical content, status label, hypotheses, dependency links, artifact identities, and prohibited stronger readings. It is required for transformations such as rewriting, summarizing, translating, refactoring, formalizing, normalizing, compressing, regenerating, or migrating a proof object when the transformed object is consumed as equivalent to the source.

Definition 1.68 (Semantic diff record)**.** A *semantic diff record* compares a source object and a transformed object at the level of definitions, theorem statements, hypotheses, quantifiers, scopes, symbols, references, claims, and status labels. It distinguishes text-only changes from mathematical, interface, governance, or artifact changes.

Proposition 1.69 (Transformation without preservation is non-equivalent)**.** *A transformed object cannot be consumed as equivalent to its source without a transformation preservation certificate or a successful semantic diff showing that no consumed semantic field changed.*

Proof. Transformations may alter hypotheses, scope, quantifiers, notation, references, or artifact identity. Equivalence is therefore an additional claim. Without a preservation certificate or semantic diff, that claim is unsupported. □

Proposition 1.70 (Formalization is a transformation, not theorem promotion)**.** *Translating an informal statement into proof-assistant syntax is a transformation and does not promote the informal statement to theorem status unless the formal statement is verified and an interpretation theorem connects it back to the intended claim.*

Proof. Formalization may change the statement or omit intended hypotheses. A proof assistant verifies only the formal statement in its environment. The connection to the informal claim requires a separate interpretation theorem. □

Remark 1.71 (Summary and translation caution)**.** Summaries, translations, and refactorings are useful audit tools, but a shorter or clearer text may be weaker, stronger, or differently scoped than the original. When theorem evidence is at stake, the original source binding or preservation certificate controls.

U.8. Verification authority

Definition 1.72 (Verification authority)**.** A *verification authority* is a declared procedure or agent whose output is allowed to establish a specific verifier-side status, such as verified formal fragment, passed gate, passed diagnostic, passed budget check, passed manifest check, or replay success.

Definition 1.73 (Authority scope)**.** The *authority scope* of a verification authority is the class of statements, objects, or checks it is authorized to verify.

Definition 1.74 (Authority environment)**.** The *authority environment* is the declared versioned environment in which a verification authority operates, including tool versions, libraries, assumptions, configuration, input artifacts, and accepted axioms.

Proposition 1.75 (Verification authority is scoped)**.** *A verification authority verifies only claims within its declared authority scope and declared environment.*

Proof. Different tools and agents check different properties. A type checker may verify type correctness but not mathematical soundness of an informal bridge. A persistence engine may compute barcodes but not prove a PDE regularity theorem. A proof assistant verifies only relative to its formal environment. Thus authority must be scoped and environment-relative. □

Remark 1.76 (Tool output requires interpretation)**.** Even deterministic tool output requires interpretation. A successful run proves only what the tool is specified, configured, and authorized to check.

U.8bis. Verification modes and tool-scope discipline

Definition 1.77 (Verification mode)**.** A *verification mode* declares what kind of check has been performed. Typical modes include:

(i) syntax_checked;

(ii) compiled;

(iii) type_checked;

(iv) computed;

(v) replayed;

(vi) artifact_validated;

(vii) formally_verified;

(viii) interpretation_proved;

(ix) human_reviewed;

(x) not_checked;

(xi) failed.

The mode determines only the checked property, not every stronger property one might desire.

Definition 1.78 (Authority-to-mode compatibility)**.** An *authority-to-mode compatibility record* states which verification modes a declared authority may establish in a declared environment. A build system may establish compiled; a type checker may establish type_checked; a proof assistant may establish formally_verified for encoded statements; none of these modes automatically establishes an external-domain theorem.

Proposition 1.79 (Tool success is mode-scoped)**.** *A successful tool run establishes only the verification mode and scope declared for that tool run.*

Proof. Tools operate under specifications and environments. The output of a parser, compiler, persistence engine, proof assistant, or renderer has the meaning assigned by that tool and its configuration. No stronger theorem follows unless a further theorem or authority record supplies it. □

Proposition 1.80 (Build success is not mathematical verification)**.** *A successful build, render, serialization, replay, or platform validation does not establish the mathematical truth of a theorem statement unless the verified mode explicitly includes the relevant mathematical proof obligation.*

Proof. Build and render systems check syntactic, packaging, or reproducibility properties. Mathematical truth requires proof of the statement under the stated hypotheses. Those are different modes. □

Remark 1.81 (Mode labels prevent tool inflation)**.** Verification mode labels prevent a successful local check from being read as a stronger global theorem, bridge theorem, or Core certificate.

U.9. Human review

Definition 1.82 (Human review)**.** *Human review* is the process by which a human agent reads, checks, critiques, edits, approves, rejects, or interprets an AK-related object.

Definition 1.83 (Human approval)**.** *Human approval* is an explicit statement that a human agent accepts an object for a declared purpose.

Definition 1.84 (Human mathematical judgment)**.** *Human mathematical judgment* is a human assessment of correctness, plausibility, rigor, usefulness, or risk. It may guide verification, but it is not identical to verifier-side proof unless supported by proof data.

Proposition 1.85 (Human approval is scoped)**.** *Human approval establishes only the approved status within the declared scope. It does not automatically establish Core validity.*

Proof. A human may approve style, structure, plausibility, publication readiness, or readiness for verification. Core validity requires Core verifier-side checks. Therefore human approval is scoped to its declared purpose. □

Remark 1.86 (Human review remains important)**.** Human review is crucial for detecting conceptual gaps, claim inflation, poor definitions, missing hypotheses, and misaligned interfaces. However, review and verification should remain semantically distinct.

U.10. Agent provenance

Definition 1.87 (Agent provenance)**.** *Agent provenance* is metadata recording which agents produced, transformed, verified, audited, approved, rejected, deprecated, or superseded an object.

Definition 1.88 (Provenance record)**.** A *provenance record* contains:

(i) the agent identifier or class;

(ii) the agent role;

(iii) the action performed;

(iv) the input object;

(v) the output object;

(vi) timestamp or version reference, if used;

(vii) authority scope, if verification is claimed;

(viii) status label before and after the action;

(ix) artifact references;

(x) audit notes.

Definition 1.89 (Versioned provenance)**.** *Versioned provenance* is provenance tied to immutable or version-controlled artifact identifiers.

Proposition 1.90 (Provenance supports audit but does not prove validity)**.** *Agent provenance supports auditability, but it does not by itself prove mathematical validity.*

Proof. Provenance records who did what. Validity requires that the resulting object satisfies the relevant mathematical or verifier-side conditions. The former supports reconstruction, but does not replace verification. □

Remark 1.91 (Why provenance matters)**.** Provenance is important for reproducibility, accountability, debugging, and version control. It also helps detect when an output has moved from search artifact to certificate candidate to verified object.

U.10bis. Artifact identity, drift, and immutable replay

Definition 1.92 (Immutable artifact identity)**.** An *immutable artifact identity* is a content-addressed or version-pinned identifier sufficient to distinguish an artifact from every changed, regenerated, repaired, superseded, or environment-dependent variant.

Definition 1.93 (Agent artifact-drift record)**.** An *agent artifact-drift record* is opened when an input, output, dependency, tool environment, retrieval corpus, prompt, model, proof assistant library, or platform object differs from the artifact identity referenced by a certificate or proof object. It records whether the drift is irrelevant, requires replay, requires semantic diff, requires demotion, or creates a new claim.

Definition 1.94 (Replay boundary)**.** The *replay boundary* is the exact collection of inputs, environments, code, models, artifacts, settings, random seeds, and external dependencies that a replay action reconstructs. A replay success is meaningful only inside this boundary.

Proposition 1.95 (Artifact drift blocks silent reuse)**.** *If a consumed artifact has drifted outside its declared identity or replay boundary, it cannot be silently reused as current verifier-side evidence.*

Proof. The consumed evidence was tied to a specific artifact identity and environment. A drifted artifact may have different mathematical content, code, data, or dependencies. Silent reuse would erase the distinction between the old and new object. Therefore replay, semantic diff, or demotion is required before reuse. □

Proposition 1.96 (Replay success is not semantic identity)**.** *Replay success proves reconstruction of the declared workflow output under the replay boundary; it does not by itself prove semantic identity with a different artifact, statement, or environment.*

Proof. Replay checks reproducibility of the recorded process. Semantic identity with another artifact is a separate comparison claim between contents and scopes. That comparison requires a semantic diff or preservation theorem. □

Remark 1.97 (New artifact, new claim when semantics change)**.** If an artifact change alters a theorem statement, hypothesis, comparison mode, diagnostic artifact, ledger, manifest field, or external-domain implication, the repaired object should be treated as a new claim or new revision rather than a silent update.

U.11. Agent failure modes

Definition 1.98 (Agent failure mode)**.** An *agent failure mode* is a way in which an agent output may be misleading, invalid, incomplete, unsupported, stale, or misclassified.

Definition 1.99 (Hallucinated claim)**.** A *hallucinated claim* is a claim generated by an agent without adequate support, proof, citation, artifact, or verification.

Definition 1.100 (Claim inflation)**.** *Claim inflation* occurs when a lower-status output, such as a search artifact, advisory indicator, draft, confidence score, or [Spec] bridge, is described or used as if it were a theorem or verified certificate.

Definition 1.101 (Scope overrun)**.** *Scope overrun* occurs when an agent or tool output is used beyond its declared authority scope.

Definition 1.102 (Artifact drift)**.** *Artifact drift* occurs when an artifact changes, is replaced, becomes stale, or becomes inconsistent with the certificate or proof object that references it.

Definition 1.103 (Bridge inflation)**.** *Bridge inflation* occurs when a bridge draft, bridge program, or [Spec] bridge is treated as a proved bridge theorem.

Definition 1.104 (Formalization mismatch)**.** *Formalization mismatch* occurs when a verified formal statement is interpreted as proving a stronger or different informal mathematical claim.

Proposition 1.105 (Agent failure modes require status downgrade)**.** *If an agent failure mode is detected in an object, the object must be downgraded to an appropriate non-verified status until repaired and rechecked.*

Proof. A detected failure mode undermines the current status of the object. Until the defect is repaired and verification or audit is repeated, the previous verified or candidate status is not justified. □

Remark 1.106 (Failure modes are expected)**.** Agent failure modes are not exceptional in a search-heavy workflow. The platform must assume they will occur and must provide status labels, provenance, and audit mechanisms to contain them.

U.12. Agent outputs and Core claims

Definition 1.107 (Core claim by agent)**.** A *Core claim by agent* is any assertion by an agent that a Core condition, theorem, gate, diagnostic, budget comparison, representative condition, eligibility condition, artifact condition, or certificate status holds.

Definition 1.108 (Admissible agent Core claim)**.** An agent Core claim is *admissible* only if it is supported by verifier-side data sufficient to reconstruct the claimed Core status.

Proposition 1.109 (Agent assertion does not establish Core claim)**.** *An agent assertion that a Core claim holds does not establish that claim unless the supporting verifier-side data are present.*

Proof. Core claims require verification. An assertion is a statement about the claim, not the verification itself. Without verifier-side data, the assertion is unsupported. □

Remark 1.110 (This includes AI and human assertions)**.** This rule applies equally to AI agents and human agents. Neither confidence nor authority substitutes for the certificate record.

U.13. Agent interaction with proof assistants

Definition 1.111 (Proof assistant interaction)**.** A *proof assistant interaction* is an action in which an agent submits, modifies, checks, or reads formal code or proof objects in a proof assistant.

Definition 1.112 (Formal fragment)**.** A *formal fragment* is a piece of formal code, theorem statement, definition, lemma, tactic script, or proof script intended for proof assistant checking.

Definition 1.113 (Verified formal fragment)**.** A *verified formal fragment* is a formal fragment that has been successfully checked by the declared proof assistant under the declared environment.

Definition 1.114 (Formal interpretation theorem)**.** A *formal interpretation theorem* is a theorem connecting a verified formal statement to an intended informal mathematical statement or interface claim.

Proposition 1.115 (Verified formal fragment is scoped to its formal statement)**.** *A verified formal fragment establishes only the formal statement that was checked in the declared environment.*

Proof. A proof assistant checks formal syntax, typing, and proof obligations relative to its environment. It does not automatically verify informal interpretations, external mathematical claims, or problem-specific meanings not encoded in the formal statement. □

Proposition 1.116 (Formal interpretation requires a bridge)**.** *A verified formal fragment may support an informal or problem-specific claim only if an interpretation theorem connects the formal statement to that claim.*

Proof. The formal statement and the intended informal claim may differ in hypotheses, semantics, or scope. A connecting theorem is required to justify the interpretation. □

Remark 1.117 (Formalization boundary)**.** A formal proof of a formal statement may still require an interpretation theorem connecting that statement to the intended mathematical claim. This is especially important for problem interfaces.

U.14. Agent manifest entries

Definition 1.118 (Agent manifest entry)**.** An *agent manifest entry* records:

(i) the agent identity or class;

(ii) the agent role;

(iii) the action performed;

(iv) the input artifacts;

(v) the output artifacts;

(vi) the output status label;

(vii) any confidence, score, plausibility, or ranking, if present;

(viii) the authority scope, if verification is claimed;

(ix) the authority environment, if a tool or proof assistant is used;

(x) artifact references;

(xi) audit notes.

Definition 1.119 (Agent action log)**.** An *agent action log* is an ordered list of agent manifest entries recording the lifecycle of a search artifact, proof object, certificate candidate, formal fragment, bridge statement, or verified object.

Definition 1.120 (Agent lifecycle)**.** The *agent lifecycle* of an object is the sequence:

$$\text{generated} \longrightarrow \text{transformed} \longrightarrow \text{labeled} \longrightarrow \text{checked} \longrightarrow \text{audited} \longrightarrow \text{curated},$$

with stages omitted or repeated as appropriate.

Proposition 1.121 (Agent manifest entries support reproducibility)**.** *Agent manifest entries support reproducibility by recording how objects were generated, transformed, verified, audited, and curated.*

Proof. Reproducibility requires reconstruction of the workflow. Agent manifest entries record the relevant actions, inputs, outputs, roles, status labels, environments, and artifact references. Thus they support reconstruction. □

Remark 1.122 (Manifest is not verdict)**.** An agent manifest entry records actions and status. It does not itself establish mathematical truth unless it contains or references the required verifier-side evidence.

U.15. Agent interaction with bridges and proof objects

Definition 1.123 (Agent-generated bridge draft)**.** An *agent-generated bridge draft* is a proposed implication connecting two layers of the AK framework, such as:

$$\text{Core-visible condition} \Longrightarrow \text{problem-specific conclusion},$$

generated by an agent before proof.

Definition 1.124 (Agent-generated proof object)**.** An *agent-generated proof object* is a structured proof object or proof-object candidate generated, filled, transformed, or assembled by an agent.

Definition 1.125 (Bridge verification requirement)**.** A bridge draft may be promoted to a bridge theorem only after:

(i) the statement is precisely formulated;

(ii) all hypotheses are declared;

(iii) all source and target semantics are fixed;

(iv) the proof is supplied;

(v) artifacts or formal fragments are attached where required;

(vi) the bridge is checked by a declared verification authority.

Proposition 1.126 (Agent-generated bridge draft is not bridge theorem)**.** *An agent-generated bridge draft is not a bridge theorem.*

Proof. A bridge theorem requires a precise statement, declared hypotheses, semantic compatibility, and proof. A draft supplies only a candidate bridge. Therefore it is not theorem-level until verified. □

Proposition 1.127 (Agent-generated proof object is not proof by default)**.** *An agent-generated proof object is not a proof by default.*

Proof. A proof object becomes proof-level only after its required fields, claims, formal fragments, bridges, artifacts, and verifier-side checks are supplied and verified. Generation alone does not establish those conditions. □

Remark 1.128 (Bridge drafts are useful)**.** Bridge drafts are useful because they expose what must be proved. Their value lies in structuring the proof search, not in establishing theorem-level validity.

U.16. Agent interaction with Core repaired objects

Definition 1.129 (Agent-produced repaired evaluation record)**.** An *agent-produced repaired evaluation record* is an output in which an agent claims to have produced or computed:

$$\mathrm{Eval}_{i,W,\tau}(F) = T_{\tau}^{\mathrm{bar}}\big(\mathsf{W}_W \mathbf{P}_i(F)\big).$$

Definition 1.130 (Agent-produced representative record)**.** An *agent-produced representative record* is an output in which an agent claims to have produced or checked an admissible representative:

$$C_{\tau,W}F.$$

Definition 1.131 (Agent-produced tower artifact)**.** An *agent-produced tower artifact* is an output in which an agent claims to have produced or checked:

$$\phi_{i,W,\tau} : \mathrm{ColimProfile}_{i,W,\tau}(F_\bullet) \to \mathrm{ApexProfile}_{i,W,\tau}(F_\infty).$$

Proposition 1.132 (Agent-produced Core records require verification)**.** *Agent-produced repaired evaluation records, representative records, and tower artifacts require verification before they may support Core certification.*

Proof. These records are verifier-side data only when they are well-typed, artifact-supported, and checked under the declared Core protocol. Agent production alone does not establish those conditions. □

Remark 1.133 (Core object sensitivity)**.** Small changes in windows, thresholds, barcode policies, representative choices, or tower artifacts may change the verdict. Therefore agent-produced Core records must be treated as candidate records until verified.

U.16bis. Agent-produced metric, budget, and Type IV records

Definition 1.134 (Agent-produced comparison record)**.** An *agent-produced comparison record* is an agent output claiming an exact comparison, a metric-gap-certified comparison, or a non-comparison status between declared profiles or artifacts. It must declare exactly one runtime mode:

$$\texttt{exact}, \qquad \texttt{metric\_gap\_certified}, \qquad \texttt{not\_compared}.$$

The mode not_compared cannot discharge an invoked comparison obligation.

Definition 1.135 (Agent-produced threshold-gap record)**.** An *agent-produced threshold-gap record* is an agent output claiming that a metric comparison is protected through hard threshold deletion. It must identify

the actual windowed pre-threshold profiles, bottleneck discrepancy evidence, protected profile family $\mathfrak{B}$, scalarization val_B, and strict inequality

$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}).$$

It must also record the specialized threshold-gap status

$$\text{gap\_certified}, \qquad \text{gap\_failed}, \qquad \text{gap\_pending},$$

with the canonical mapping to pass, reject, and undefined in the audit status calculus.

Definition 1.136 (Agent-produced Type IV diagnostic record)**.** An *agent-produced Type IV diagnostic record* is an agent output claiming one of

$$\text{not\_invoked}, \quad \text{undefined}, \quad \text{certified\_zero}, \quad \text{obstructed}$$

for a declared monitored Type IV scope. If invoked and well-typed, numerical diagnostics are computed from the window-relative terminal-generic dimensions

$$\mathrm{tgdim}_W(\ker \phi_{i,W,\tau}), \qquad \mathrm{tgdim}_W(\mathrm{coker}\, \phi_{i,W,\tau})$$

of the actual declared tower comparison artifact. No numerical pair is assigned when the status is not_invoked or undefined.

Proposition 1.137 (Agent comparison mode does not self-certify)**.** *An agent's declaration of comparison mode does not establish that the declared mode is valid.*

Proof. Mode validity requires the corresponding evidence: an exact equality or profile isomorphism for exact mode, a bottleneck bound plus two-sided threshold-gap record for metric mode, or proof that no comparison obligation is consumed for not-compared mode. The declaration names the intended mode but does not supply the evidence. □

Proposition 1.138 (Exact output is not robustness)**.** *An agent-produced exact equality or profile isomorphism is not a positive-radius robustness certificate unless a separate metric-gap-certified record is supplied.*

Proof. Exact comparison and metric robustness are different v19 comparison modes. A specific equality may hold even when the threshold clearance is zero. Robustness through hard deletion requires a bottleneck discrepancy bound and two-sided threshold-gap protection. □

Proposition 1.139 (Agent Type IV zero requires actual artifact)**.** *An agent-produced claim that Type IV diagnostics are zero is unsupported unless the declared tower comparison artifact, terminal-tameness evidence, monitored scope, and terminal-generic defect computation are present.*

Proof. Type IV diagnostics are defined from the kernel and cokernel defect profiles of an actual comparison artifact. Without that artifact and terminal-tameness evidence, the diagnostic status is undefined or not_invoked, not certified zero. □

Remark 1.140 (Agent-produced zero values)**.** A displayed zero, empty list, passed test, green dashboard, or natural-language phrase such as "no defect found" is not by itself a certified zero diagnostic, exact comparison, or zero metric defect. The typed record determines the meaning of zero.

U.17. Summary of agent semantics

Theorem 1.141 (Appendix U agent-semantics package)**.** *Within the Search / Platform layer, the following package is available.*

(i) *Agents may be human, AI, tool-based, platform-based, or hybrid.*

(ii) *Agents may act as proposers, searchers, mappers, lifters, verifiers, auditors, curators, or adjudicators.*

(iii) *Agent type does not determine validity.*

(iv) *Proposer-side data do not determine Core verdicts.*

(v) *Core pass/reject is determined only by verifier-side data.*

(vi) *Verifier-side data include repaired evaluation records, admissible representatives when invoked, eligibility records when invoked, tower comparison artifacts when invoked, diagnostics, gates, ledgers, and artifact references.*

(vii) *AI confidence is not proof.*

(viii) *Agent rankings and plausibility scores do not imply validity.*

(ix) *Agent outputs require explicit status labels.*

(x) *Unlabeled outputs are non-verdict by default.*

(xi) *Status promotion requires evidence.*

(xii) *Agent actions such as generation, ranking, verification, audit, and replay are semantically distinct.*

(xiii) *Verification authority is scoped and environment-relative.*

(xiv) *Human approval is scoped.*

(xv) *Provenance supports audit but does not prove validity.*

(xvi) *Agent failure modes require status downgrade until repaired.*

(xvii) *Agent assertions do not establish Core claims without verifier-side support.*

(xviii) *Verified formal fragments are scoped to their checked formal statements.*

(xix) *Formal interpretation requires a bridge.*

(xx) *Agent manifest entries support reproducibility but do not themselves determine mathematical truth.*

(xxi) *Agent-generated bridge drafts are not bridge theorem-level by default.*

(xxii) *Agent-generated proof objects are not proof by default.*

(xxiii) *Agent-produced Core records require verification before they may support Core certification.*

Proof. Items (*i*)–(*iii*) follow from the definitions of agent types and roles and Remark 1.16. Items (*iv*)–(*vi*) are Theorem 1.28 and the definition of verifier-side data. Items (*vii*)–(*viii*) are Propositions 1.39, 1.40, and 1.41. Items (*ix*)–(*xi*) follow from the status-label definitions and Propositions 1.55 and 1.56. Item (*xii*) is Proposition 1.65. Item (*xiii*) is Proposition 1.75. Item (*xiv*) is Proposition 1.85. Item (*xv*) is Proposition 1.90. Item (*xvi*) is Proposition 1.105. Item (*xvii*) is Proposition 1.109. Items (*xviii*)–(*xix*) are Propositions 1.115 and 1.116. Item (*xx*) follows from Proposition 1.121 and Remark 1.122. Items (*xxi*)–(*xxii*) are Propositions 1.126 and 1.127. Item (*xxiii*) is Proposition 1.132. □

Theorem 1.142 (Appendix U v19.0 hardening package)**.** *The additional v19.0 Search / Platform hardening supplies the following constraints.*

(i) Agent-side outputs are non-verdict unless converted through a verifier-side conversion record.

(ii) Agent success cannot compensate for missing non-scalar obligations.

(iii) Promotion requires a promotion certificate and claim-use preflight.

(iv) Adjudication resolves workflow conflicts but is not mathematical verification.

(v) Retrieval context, prompt context, memory, and platform context do not adopt claims as theorems.

(vi) Consensus among agents is not proof.

(vii) Transformations require preservation certificates or semantic diff records before equivalence is consumed.

(viii) Formalization is transformation and does not promote the informal statement without verification and interpretation.

(ix) Verification modes are scoped; tool success establishes only the declared mode.

(x) Build, render, replay, and serialization success are not mathematical verification by themselves.

(xi) Artifact identity and replay boundaries control reuse; artifact drift blocks silent consumption.

(xii) Replay success is not semantic identity with another artifact or environment.

(xiii) Agent-produced comparison records require valid exact, metric-gap-certified, or not-compared evidence at the declared mode.

(xiv) Agent-produced exact outputs do not supply positive-radius robustness.

(xv) Agent-produced Type IV zero claims require the actual tower artifact, terminal-tameness evidence, monitored scope, and terminal-generic computation.

Proof. Items (*i*)–(*ii*) follow from Definitions 1.4, 1.5, and 1.6, and from Propositions 1.7 and 1.8. Items (*iii*)–(*iv*) follow from Definitions 1.30, 1.31, and 1.32, and from Propositions 1.33 and 1.34. Items (*v*)–(*vi*) follow from Definitions 1.43, 1.44, and 1.45, and from Propositions 1.46, 1.47, and 1.48. Items (*vii*)–(*viii*) follow from Definitions 1.67 and 1.68, and from Propositions 1.69 and 1.70. Items (*ix*)–(*x*) follow from Definitions 1.77 and 1.78, and from Propositions 1.79 and 1.80. Items (*xi*)–(*xii*) follow from Definitions 1.92, 1.93, and 1.94, and from Propositions 1.95 and 1.96. Items (*xiii*)–(*xv*) follow from Definitions 1.134, 1.135, and 1.136, and from Propositions 1.137, 1.138, and 1.139. □

U.18. Explicit exclusions

The following are not asserted in Appendix U.

- No AI output is treated as proof by itself.
- No human assertion is treated as proof by itself.
- No agent confidence value is treated as a Core verdict.
- No agent ranking is treated as validity.
- No plausibility score is treated as verifiability.
- No search artifact is promoted to theorem status by authorship.
- No bridge draft is treated as a bridge theorem.
- No proof object generated by an agent is treated as proof by generation alone.
- No tool output is used beyond its declared authority scope.
- No proof assistant result is interpreted beyond the checked formal statement without an interpretation theorem.
- No unlabeled output affects Core pass/reject.
- No provenance record proves mathematical validity by itself.
- No platform storage event certifies an object.
- No agent approval replaces B-Gate^{+}.
- No agent approval replaces the repaired topological condition

$$\mathrm{Eval}_{1,W,\tau}(F) = 0.$$

- No agent approval replaces admissible representative records

$$C_{\tau,W}F.$$

- No agent approval replaces tower comparison artifacts

$$\phi_{i,W,\tau}.$$

- No agent approval replaces monitored Type IV diagnostics.
- No agent approval replaces the budget condition

$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}).$$

- No agent-generated validity map is treated as a theorem.
- No agent-produced repaired evaluation record is accepted without verification.

- No agent-produced tower artifact is accepted without verification.
- No status promotion is accepted without evidence.
- No artifact drift is ignored when detected.
- No retrieval result, context window, memory item, or platform page is treated as theorem evidence by inclusion alone.
- No consensus among agents is treated as proof.
- No transformation, summary, translation, refactoring, or formalization is treated as semantics-preserving without a preservation certificate or semantic diff.
- No successful build, render, serialization, or replay is treated as mathematical verification by itself.
- No exact comparison output is treated as a positive-radius robustness certificate without a metric-gap-certified record.
- No agent-produced zero value is treated as certified Type IV zero without the actual tower artifact and terminal-generic diagnostic computation.
- No stale artifact, changed environment, or drifted dependency is silently reused as verifier-side evidence.

U.19. Provenance and status

The material in this appendix depends on:

(i) the claim taxonomy and Minimal Manifest discipline of Chapter 1;

(ii) the claim-governance and claim-use preflight discipline of Appendix CR-v19;

(iii) the repaired evaluation discipline of Appendix A;

(iv) the admissible representative discipline of Appendix B;

(v) the eligible realization regime of Appendix C;

(vi) the monitored Type IV diagnostic and tower artifact discipline of Appendix D;

(vii) the Core sovereignty doctrine of Chapter 7;

(viii) the non-claim discipline of Chapter 8;

(ix) the interface protocol of Chapter 8;

(x) the manifest schema of Appendix G;

(xi) the advisory-invariant discipline of Appendix H;

(xii) the ledger catalog of Appendix M;

(xiii) the proof-first program discipline of Appendix NS-C.

Its role is to define how agents may participate in search and proof-management workflows without contaminating Core verdicts.

Status labels for Appendix U. *Search / Platform definitions:* Definitions 1.10, 1.11, 1.12, 1.13, 1.14, 1.15, 1.17, 1.18, 1.19, 1.20, 1.21, 1.22, 1.23, 1.24, 1.26, 1.27, 1.36, 1.37, 1.38, 1.50, 1.51, 1.52, 1.53, 1.54, 1.58, 1.59, 1.60, 1.61, 1.62, 1.63, 1.64, 1.72, 1.73, 1.74, 1.82, 1.83, 1.84, 1.87, 1.88, 1.89, 1.98, 1.99, 1.100, 1.101, 1.102, 1.103, 1.104, 1.107, 1.108, 1.111, 1.112, 1.113, 1.114, 1.118, 1.119, 1.120, 1.123, 1.124, 1.125, 1.129, 1.130, 1.131.

Search / Platform propositions/theorems: Theorems 1.28, 1.141, Propositions 1.39, 1.40, 1.41, 1.55, 1.56, 1.65, 1.75, 1.85, 1.90, 1.105, 1.109, 1.115, 1.116, 1.121, 1.126, 1.127, 1.132.

Operational cautions: Remarks 1.1, 1.2, 1.3, 1.16, 1.25, 1.29, 1.42, 1.57, 1.66, 1.76, 1.86, 1.91, 1.106, 1.110, 1.117, 1.122, 1.128, 1.133.

Additional v19.0 hardening definitions: Definitions 1.4, 1.5, 1.6, 1.30, 1.31, 1.32, 1.43, 1.44, 1.45, 1.67, 1.68, 1.77, 1.78, 1.92, 1.93, 1.94, 1.134, 1.135, 1.136.

Additional v19.0 hardening propositions/theorems: Theorem 1.142, Propositions 1.7, 1.8, 1.33, 1.34, 1.46, 1.47, 1.48, 1.69, 1.70, 1.79, 1.80, 1.95, 1.96, 1.137, 1.138, 1.139.

Additional v19.0 hardening remarks: Remarks 1.9, 1.35, 1.49, 1.71, 1.81, 1.97, 1.140.

Explicit exclusions: Section U.18.

2 Appendix V: Hunter / Mapper / Lifter Search Protocols

V.1. Scope and search status

This appendix defines the Hunter / Mapper / Lifter search protocols used in the AK framework. It belongs to the Search / Platform Appendices. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

Search protocols generate candidates; they do not certify them.

In particular, no search hit, mapped structure, lifted candidate, priority score, agent recommendation, search map edge, or bridge draft may replace:

$$B\text{-Gate}^{+}, \qquad \mathrm{Eval}_{1,W,\tau}(F) = 0, \qquad C_{\tau,W}F, \qquad \phi_{i,W,\tau},$$
$$(\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}) = (0,0), \qquad \mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_{\tau}(\mathfrak{B}).$$

Remark 2.1 (Search / Platform status)**.** The protocols in this appendix organize discovery, mapping, prioritization, and candidate preparation. They do not establish mathematical validity by themselves.

Remark 2.2 (Relation to Appendix U)**.** Appendix U defined agent semantics. The present appendix specializes those semantics to three search roles:

$$\text{Hunter}, \qquad \text{Mapper}, \qquad \text{Lifter}.$$

These roles may be performed by human agents, AI agents, tool agents, platform agents, or hybrid agents.

Remark 2.3 (Relation to Core re-entry)**.** The purpose of the Hunter / Mapper / Lifter workflow is not to prove claims directly. Its purpose is to prepare candidates that may later be submitted to verifier-side Core re-entry. The re-entry boundary is therefore the semantic boundary between search and certification.

V.1bis. Search sovereignty and verifier-side conversion

Definition 2.4 (Search sovereignty)**.** *Search sovereignty* is the rule that Hunter, Mapper, Lifter, ranking, search-map, and platform outputs remain proposer-side or workflow-side data until converted into verifier-side records by a declared protocol. A search-side object may guide the creation of a verifier-side record, but it is not itself a Core verdict, bridge theorem, Type IV diagnostic, metric budget certificate, admissible representative, or proof object.

Definition 2.5 (Search-to-verifier conversion record)**.** A *search-to-verifier conversion record* documents a transition from a search-side object to a verifier-side candidate. It is a Search / Platform specialization of the verifier-side conversion record discipline of Appendix U. It must declare:

(i) the source search object and its status;

(ii) the target verifier-side record type;

(iii) the conversion rule;

(iv) all hypotheses and non-scalar obligations;

(v) all defects introduced by the conversion;

(vi) artifact identifiers before and after conversion;

(vii) the verification authority and authority scope;

(viii) the result status, chosen from *pass*, *reject*, or *undefined*;

(ix) the typed failure route when the result status is *reject* or *undefined*.

Definition 2.6 (Search claim inflation)**.** *Search claim inflation* occurs when a Hunter output, Mapper edge, Lifter output, score, search map, bridge draft, or re-entry package is described or consumed as if it already established a verifier-side Core condition, bridge theorem, or external-domain theorem.

Proposition 2.7 (No search sovereignty override)**.** *No search protocol, search lifecycle status, score, map edge, lift output, or re-entry package overrides verifier-side Core requirements.*

Proof. Search-side records belong to the proposer and workflow layers. Core verdicts are determined by verifier-side repaired evaluations, representatives, eligibility data, tower artifacts, diagnostics, ledgers, gates, and manifests. Therefore a search-side object can at most propose or prepare verifier-side evidence; it cannot override or replace it. □

Proposition 2.8 (Conversion is not achieved by relabeling)**.** *Relabeling a search artifact as* verified*,* Core-ready*,* theorem-like*, or* certificate-like *does not convert it into verifier-side data.*

Proof. Verifier-side status depends on typed construction, artifact support, and verification under the declared protocol. A label change supplies none of these data. Hence conversion requires a search-to-verifier conversion record and the corresponding checks. □

Remark 2.9 (Search remains proposer-side until conversion)**.** The search layer may be aggressive, heuristic, automated, or AI-assisted. That is acceptable precisely because the transition to Core evidence is guarded by conversion, verification, and audit records.

V.2. Search objects

Definition 2.10 (Search object)**.** A *search object* is any object produced, selected, transformed, ranked, mapped, or lifted during search. Examples include:

(i) candidate windows;

(ii) candidate thresholds;

(iii) candidate stable bands;

(iv) candidate bridge statements;

(v) candidate proof routes;

(vi) advisory indicators;

(vii) persistence profiles;

(viii) candidate filtered objects;

(ix) candidate tower diagrams;

(x) candidate normalization routes;

(xi) candidate problem-interface realizations;

(xii) candidate repaired evaluation records;

(xiii) candidate admissible representative records;

(xiv) candidate tower comparison artifacts;

(xv) draft certificate records;

(xvi) proof object fragments.

Definition 2.11 (Search artifact)**.** A *search artifact* is a stored or referenced output of a search process. It may include text, code, diagrams, rankings, maps, datasets, proof sketches, logs, generated candidates, or structured records.

Definition 2.12 (Candidate)**.** A *candidate* is a search object that has been selected for possible further development, mapping, lifting, verification, formalization, or audit.

Definition 2.13 (Candidate status)**.** A candidate has one of the following statuses:

(i) *raw*;

(ii) *ranked*;

(iii) *mapped*;

(iv) *lift-attempted*;

(v) *lifted*;

(vi) *verification-ready*;

(vii) *blocked*;

(viii) *rejected*;

(ix) *incomplete*;

(x) *deprecated*;

(xi) *superseded*.

Proposition 2.14 (Search artifacts are non-verdict by default)**.** *A search artifact does not affect Core pass/reject unless it is converted into verifier-side data and checked under the Core protocol.*

Proof. Search artifacts are proposer-side data. By Appendix U, proposer-side data do not determine Core verdicts. Only verifier-side data determine Core pass/reject. □

Remark 2.15 (Search artifact value)**.** Search artifacts may still be highly valuable. They support discovery, comparison, reproducibility, prioritization, debugging, and proof planning. Their limitation is logical, not operational.

V.3. Hunter protocol

Definition 2.16 (Hunter)**.** A *Hunter* is an agent role whose task is to find candidate objects, regions, lemmas, bridges, failure modes, proof routes, or artifact references in a declared search space.

Definition 2.17 (Hunt space)**.** A *hunt space* is a declared space of possible search targets. Examples include:

(i) window families;

(ii) threshold bands;

(iii) tower configurations;

(iv) arithmetic interface routes;

(v) PDE observable families;

(vi) normalization choices;

(vii) bridge lemma templates;

(viii) candidate proof programs;

(ix) formalization fragments;

(x) certificate DAG fragments;

(xi) known failure-mode neighborhoods.

Definition 2.18 (Hunt objective)**.** A *hunt objective* is the declared goal of a Hunter protocol, such as finding a stable band, locating a bridge candidate, identifying a Type IV proxy, searching for a counterexample, or discovering a Core-readable realization route.

Definition 2.19 (Hunt output)**.** A *hunt output* is a list, set, graph, table, or ranked family of candidates produced by a Hunter, together with any scores, rankings, reasons, constraints, or artifact references.

Definition 2.20 (Hunter protocol)**.** A *Hunter protocol* specifies:

(i) the hunt space;

(ii) the hunt objective;

(iii) the search constraints;

(iv) the candidate selection rule;

(v) the ranking or scoring method, if any;

(vi) stopping or continuation conditions, if any;

(vii) the output status labels;

(viii) artifact references.

Proposition 2.21 (Hunter output is not certificate)**.** *A Hunter output is not a Core certificate.*

Proof. Hunter output is a candidate-generating search artifact. A Core certificate requires verifier-side data satisfying the Minimal Manifest Axiom, repaired evaluation records, gates, diagnostics, ledgers, tower artifacts when invoked, and artifact requirements. Thus Hunter output is not a certificate. □

Remark 2.22 (Hunter success)**.** Hunter success means that promising candidates were found. It does not mean that the candidates are valid.

V.4. Mapper protocol

Definition 2.23 (Mapper)**.** A *Mapper* is an agent role whose task is to organize search objects into maps, dependency structures, diagrams, classifications, validity landscapes, certificate DAG fragments, or bridge-route graphs.

Definition 2.24 (Search map)**.** A *search map* is a structured representation of relations among search objects. It may include:

(i) dependency arrows;

(ii) obstruction labels;

(iii) candidate bridge routes;

(iv) window overlap relations;

(v) tower comparison relations;

(vi) status labels;

(vii) search priorities;

(viii) known gaps;

(ix) failure-mode labels;

(x) artifact references.

Definition 2.25 (Map edge semantics)**.** A *map edge semantics* specifies what an edge in a search map means. Allowed edge types include:

(i) dependency;

(ii) analogy;

(iii) candidate implication;

(iv) verified implication;

(v) obstruction;

(vi) refinement;

(vii) replacement;

(viii) contradiction;

(ix) artifact reference;

(x) status transition.

Definition 2.26 (Mapper output)**.** A *Mapper output* is a search map, validity map fragment, dependency diagram, certificate DAG fragment, bridge graph, obstruction map, or classification table produced by a Mapper.

Definition 2.27 (Mapper protocol)**.** A *Mapper protocol* specifies:

(i) the objects to be mapped;

(ii) the relation types;

(iii) the edge semantics;

(iv) the status labels;

(v) the artifact references;

(vi) the update policy;

(vii) the conflict-resolution policy.

Proposition 2.28 (Mapper output is navigation, not theorem)**.** *A Mapper output is a navigation artifact and does not establish theorems by itself.*

Proof. A map records relations, candidates, priorities, dependencies, or gaps. The truth of a theorem requires proof or verification of the relevant statements. A map may point to such proof objects, but it does not replace them. □

Remark 2.29 (Map edges must be typed)**.** Edges in a search map must be typed. An edge may mean dependency, analogy, conjectural implication, verified implication, blocked route, or artifact reference. Untyped edges invite claim inflation.

V.4bis. Map-edge verification discipline

Definition 2.30 (Map-edge verification status)**.** A map edge has exactly one declared verification status:

(i) *untyped*;

(ii) *proposed*;

(iii) *artifact-referenced*;

(iv) *theorem-backed*;

(v) *verified*;

(vi) *failed*;

(vii) *deprecated*;

(viii) *superseded*.

A *verified* map edge must name the exact theorem, proof object, formal fragment, artifact, or verifier-side record that establishes the edge semantics.

Definition 2.31 (Map-edge promotion certificate)**.** A *map-edge promotion certificate* is the record required to promote a map edge from proposed or artifact-referenced status to theorem-backed or verified status. It contains the edge type, source and target objects, proof or artifact reference, authority scope, dependencies, and prohibited stronger readings.

Proposition 2.32 (Candidate implication edge is not implication)**.** *A search-map edge labeled as a candidate implication does not establish the corresponding implication.*

Proof. A candidate implication edge records a proposed route in the search map. The truth of an implication requires proof or a verified bridge. Since the edge by itself supplies only map structure, it does not establish the implication. □

Proposition 2.33 (Verified edge requires evidence)**.** *A map edge may be assigned verified status only when its declared semantics are supported by verifier-side evidence or by a proved theorem at the stated strength.*

Proof. Map edges can represent dependency, analogy, refinement, replacement, contradiction, artifact reference, or implication. Each edge type has different evidentiary requirements. A verified label is therefore admissible only after the required evidence for that specific edge type is supplied. □

Remark 2.34 (Map semantics are not inherited from drawing conventions)**.** Arrow direction, graph layout, color, proximity, clustering, or visual prominence does not determine logical implication. Only the declared edge semantics and supporting evidence determine how a map edge may be consumed.

V.5. Lifter protocol

Definition 2.35 (Lifter)**.** A *Lifter* is an agent role whose task is to transform a search object into a Core-readable object candidate, bridge candidate, certificate candidate, formal fragment candidate, or proof object candidate.

Definition 2.36 (Lift input)**.** A *lift input* is a candidate produced by a Hunter or organized by a Mapper and selected for possible conversion into verifier-side or verifier-adjacent data.

Definition 2.37 (Lift target)**.** A *lift target* is the declared type into which the candidate is intended to be transformed. Examples include:

$$\mathsf{FiltCh}(()k), \qquad \mathsf{Pers}_k^{\mathrm{cons}}, \qquad \mathrm{Eval}_{i,W,\tau}(F), \qquad C_{\tau,W}F, \qquad \phi_{i,W,\tau},$$

a ledger entry, a bridge lemma candidate, a formal fragment, or a certificate record candidate.

Definition 2.38 (Lift output)**.** A *lift output* is one of:

(i) a Core-readable filtered object candidate;

(ii) a constructible persistence object candidate;

(iii) a repaired evaluation record candidate
$$\mathrm{Eval}_{i,W,\tau}(F);$$

(iv) an admissible representative candidate
$$C_{\tau,W}F;$$

(v) an eligible realized object candidate;

(vi) a tower comparison artifact candidate
$$\phi_{i,W,\tau};$$

(vii) a tower diagnostic candidate;

(viii) a ledger entry candidate;

(ix) a bridge lemma candidate;

(x) a certificate record candidate;

(xi) a formal fragment candidate.

Definition 2.39 (Lifter protocol)**.** A *Lifter protocol* specifies:

(i) the lift input;

(ii) the lift target;

(iii) the transformation rule;

(iv) the required declarations;

(v) constructibility requirements;

(vi) representative and eligibility requirements, if invoked;

(vii) tower artifact requirements, if Type IV diagnostics are invoked;

(viii) error or defect accounting;

(ix) status labels before and after lifting;

(x) artifact references.

Proposition 2.40 (Lifted candidate is not verified by lifting)**.** *A lifted candidate is not verified merely because it has been lifted.*

Proof. Lifting changes the type or representation of a candidate. Verification checks whether the resulting object satisfies declared verifier-side conditions. These are distinct actions by Appendix U. □

Proposition 2.41 (Core-looking lift is not Core verified)**.** *A lift output that syntactically resembles a Core object, such as*

$$\mathrm{Eval}_{i,W,\tau}(F), \qquad C_{\tau,W}F, \qquad \phi_{i,W,\tau},$$

is not Core verified until checked under the declared Core protocol.

Proof. A Core-looking object may be malformed, unsupported, nonconstructible, incorrectly parameterized, or inconsistent with artifacts. Core verification requires typed declarations, artifact support, and verifier-side checks. Therefore syntactic form alone is insufficient. □

Remark 2.42 (Lifting is the re-entry preparation step)**.** Lifting is the natural preparation step before Core re-entry. It attempts to turn search artifacts into objects that the Core verifier can actually inspect.

V.5bis. Lift preservation and defect accounting

Definition 2.43 (Lift preservation record)**.** A *lift preservation record* states which properties of the lift input are preserved by the lift output. It must declare the preserved feature, source semantics, target semantics, preservation direction, hypotheses, artifact evidence, and failure mode.

Definition 2.44 (Search defect namespace)**.** The search-protocol defect namespace contains the following interface-level components:

$$\delta^{\mathrm{hunt}}, \qquad \delta^{\mathrm{map}}, \qquad \delta^{\mathrm{lift}}, \qquad \delta^{\mathrm{score}}, \qquad \delta^{\mathrm{reentry}}, \qquad \Delta_{\phi,\mathrm{search}}.$$

The intended readings are:

(i) δ^{hunt}: candidate-generation discrepancy;

(ii) δ^{map}: map-serialization or edge-semantics discrepancy;

(iii) δ^{lift}: transformation discrepancy;

(iv) δ^{score}: certified score-to-budget conversion discrepancy;

(v) $\delta^{\mathrm{reentry}}$: verifier-side submission discrepancy;

(vi) $\Delta_{\phi,\mathrm{search}}$: transport or reconstruction discrepancy for a search-produced tower artifact.

Definition 2.45 (Lift equivalence claim)**.** A *lift equivalence claim* asserts that the lifted output preserves the source object up to a declared equivalence, exact equality, interleaving, bottleneck bound, bridge implication, or problem-interface semantics. Such a claim is [Spec] by default unless supported by a lift preservation record and the required verifier-side evidence.

Proposition 2.46 (Lift is not preservation by default)**.** *A lift operation does not preserve mathematical content, Core-readability, exact comparison, metric safety, or theorem status by default.*

Proof. Lifting transforms a search-side object into a target representation or candidate record. Transformation may discard information, change parameters, alter artifacts, or introduce defects. Therefore any preservation claim requires a separate preservation record. □

Proposition 2.47 (Lift defects must be typed before budget use)**.** *A defect introduced by hunting, mapping, lifting, scoring, re-entry, or search-produced tower-artifact transport may affect the Core budget only after it is typed, registered, scalarized when metric, and included in the declared ledger.*

Proof. The Core budget consumes typed metric-ledger components through a declared scalarization. Untyped search-side discrepancies are not ledger elements. Thus they cannot affect the budget until registered and scalarized in the declared ledger regime. □

Remark 2.48 (Lift defects and global ledger registration)**.** The symbols in Definition 2.44 are local Search / Platform notation until the global ledger catalog registers them as aliases, refinements, aggregates, or disjoint contributions relative to Appendix M and any interface-layer defect namespace consumed by the same certificate.

V.6. Search lifecycle

Definition 2.49 (Search lifecycle)**.** The *search lifecycle* is the ordered process:

$$\text{hunt} \longrightarrow \text{map} \longrightarrow \text{lift} \longrightarrow \text{verify} \longrightarrow \text{audit}.$$

Definition 2.50 (Lifecycle status)**.** A search object may have one of the following lifecycle statuses:

(i) *found*;

(ii) *mapped*;

(iii) *prioritized*;

(iv) *lift-attempted*;

(v) *lifted*;

(vi) *verification-ready*;

(vii) *verified*;

(viii) *audited*;

(ix) *rejected*;

(x) *blocked*;

(xi) *incomplete*;

(xii) *deprecated*;

(xiii) *superseded*.

Definition 2.51 (Lifecycle transition)**.** A *lifecycle transition* is a status change in a search object. Every transition must record the agent, action, evidence, and artifact references supporting the transition.

Remark 2.52 (Lifecycle status and Appendix U labels)**.** The lifecycle status of Definition 2.50 is a workflow field and does not replace the agent-output status label required by Appendix U. Likewise, the candidate status of Definition 2.13 is a pre-verification specialization of the lifecycle field, not a separate proof-status algebra. Operationally, *raw* corresponds to initial *found* status, *ranked* corresponds to *prioritized*, and *mapped*, *lift-attempted*, *lifted*, *verification-ready*, *blocked*, *rejected*, *incomplete*, *deprecated*, and *superseded* retain the same workflow meanings. Every output consumed by the AK workflow still carries an Appendix U status label, and any conflict is resolved by the weaker safe interpretation.

Proposition 2.53 (Lifecycle advancement is not validity monotonicity)**.** *Advancing through the search lifecycle does not monotonically increase mathematical validity.*

Proof. A candidate may be found, mapped, and lifted, yet fail verification. Lifecycle stages record workflow progress, not mathematical truth. Validity is established only at the appropriate verification stage. □

Remark 2.54 (Why lifecycle status matters)**.** Lifecycle status helps manage large search spaces. It prevents premature conversion of found or mapped objects into verified claims.

V.7. Search scores

Definition 2.55 (Search score)**.** A *search score* is a scalar, vector, categorical, or ordered value assigned to a search object for prioritization. Examples include:

(i) plausibility score;

(ii) structural alignment score;

(iii) advisory stability score;

(iv) bridge-likelihood score;

(v) defect-risk score;

(vi) novelty score;

(vii) proof-readiness score;

(viii) agent confidence value;

(ix) re-entry-readiness score;

(x) failure-risk score.

Definition 2.56 (Score semantics)**.** A *score semantics* specifies what a search score measures, how it is computed, and what it does not measure.

Definition 2.57 (Proof-readiness score)**.** A *proof-readiness score* estimates whether a candidate appears ready for formalization, verifier-side checking, or proof-object assembly. It does not establish proof.

Proposition 2.58 (Search score is not proof)**.** *A search score is not proof and does not determine Core pass/reject.*

Proof. A search score is proposer-side prioritization data. By Appendix U, proposer-side data do not determine Core verdicts. □

Remark 2.59 (Score discipline)**.** Scores may be useful for selecting what to inspect. They should not be used to accept or reject mathematical claims without verification.

V.7bis. Score calibration and consensus discipline

Definition 2.60 (Score calibration record)**.** A *score calibration record* declares the input scope, score generator, normalization, codomain, interpretation, prohibited stronger readings, and any theorem or verified computation connecting the score to a verifier-side quantity.

Definition 2.61 (Score-to-budget conversion)**.** A *score-to-budget conversion* is a certified conversion from a search score to a metric-ledger component. It must supply actual pre-threshold profiles, a bottleneck discrepancy upper bound, all crop and window-edge contributions, a declared scalarization, and the strict threshold-gap inequality on the protected family.

Definition 2.62 (Agent consensus)**.** *Agent consensus* is agreement among multiple human, AI, tool, platform, or hybrid agents about a candidate, ranking, map edge, lift output, bridge draft, or proof route.

Proposition 2.63 (Score is not budget without conversion)**.** *A search score, even a zero risk score or high proof-readiness score, is not a metric-ledger component unless a certified score-to-budget conversion is supplied.*

Proof. Scores and metric-ledger components live in different typed spaces. The Core budget consumes scalarized ledger components that upper-bound actual bottleneck discrepancies. A score has no such authority unless converted by a certified record. □

Proposition 2.64 (Consensus is not verification)**.** *Agent consensus does not establish proof, Core pass, bridge-theorem status, or external-domain validity.*

Proof. Consensus records agreement among agents. Verification checks declared mathematical or verifier-side conditions. Agreement may guide prioritization, but it is not the checked evidence required for validity. □

Remark 2.65 (Consensus may guide search ordering)**.** Consensus, ensemble ranking, or repeated agent agreement may be operationally useful for prioritization. It remains proposer-side unless converted into a verified record of the appropriate type.

V.8. Search-to-Core re-entry

Definition 2.66 (Search-to-Core re-entry)**.** *Search-to-Core re-entry* is the process by which a lifted candidate is submitted to the Core verifier. It requires:

(i) declared objects;

(ii) declared windows W;

(iii) threshold and collapse policies;

(iv) repaired evaluation records
$$\mathrm{Eval}_{i,W,\tau}(F) = T_\tau^{\mathrm{bar}}\big(\mathsf{W}_W \mathbf{P}_i(F)\big);$$

(v) admissible representative records
$$C_{\tau,W} F$$
when invoked;

(vi) eligibility records when the Ext^1-clause is invoked;

(vii) declared tower comparison artifacts

$$\phi_{i,W,\tau}$$

when Type IV diagnostics are invoked;

(viii) monitored diagnostics;

(ix) gate verdicts;

(x) ledger policy and budget comparison;

(xi) artifact references;

(xii) proof object or certificate record schema;

(xiii) status labels for all imported search artifacts.

Definition 2.67 (Re-entry candidate)**.** A *re-entry candidate* is a lifted object prepared for Core verification but not yet verified.

Definition 2.68 (Re-entry package)**.** A *re-entry package* is the collection of artifacts, declarations, labels, records, and candidate verifier-side data submitted together for Core verification.

Definition 2.69 (Re-entry failure)**.** A *re-entry failure* occurs when a lifted candidate cannot satisfy the required manifest, typing, constructibility, repaired evaluation, representative, eligibility, diagnostic, tower artifact, ledger, or gate preconditions needed for Core verification.

Proposition 2.70 (Re-entry candidate is not Core certificate)**.** *A re-entry candidate is not a Core certificate until it passes the required verifier-side checks.*

Proof. A re-entry candidate is prepared for verification. A Core certificate has passed verification according to the declared Core conditions. Preparation is not verification. □

Remark 2.71 (Re-entry is the semantic boundary)**.** Before re-entry, data are search-side. After successful verification, data may become Core-certified. This boundary must remain explicit.

V.8bis. Re-entry modes and scalarized search budget

Definition 2.72 (Re-entry verification mode)**.** A re-entry package has exactly one verification mode:

(i) *not-submitted*;

(ii) *candidate*;

(iii) *manifest-complete*;

(iv) *checked*;

(v) *Core verified*;

(vi) *failed*;

(vii) *undefined*.

The mode *manifest-complete* is not proof; it only states that required fields are present for checking.

Definition 2.73 (Search-adjusted metric budget)**.** When a certificate consumes metric discrepancies introduced by the Hunter / Mapper / Lifter workflow, the search-adjusted metric ledger is

$$\mathfrak{d}_{\text{search}} := \mathfrak{d}_{\text{base}} \oplus \delta^{\text{hunt}} \oplus \delta^{\text{map}} \oplus \delta^{\text{lift}} \oplus \delta^{\text{score}} \oplus \delta^{\text{reentry}} \oplus \Delta_{\phi,\text{search}},$$

with omitted unused components equal to the identity element of the declared metric-ledger value system. Only components supported by actual discrepancy bounds may enter $\mathfrak{d}_{\text{search}}$.

Proposition 2.74 (Search-adjusted evidence must pass scalarized budget)**.** *A Core certificate consuming search-induced metric discrepancies may pass only if the declared scalarization supplies a protected profile family* $\mathfrak{B}_{\text{search}}$ *such that*

$$\text{val}_B(\mathfrak{d}_{\text{search}}) < \text{gap}_\tau(\mathfrak{B}_{\text{search}}),$$

together with evidence that the scalarized value upper-bounds the actual bottleneck discrepancies consumed by the certificate.

Proof. The v19 budget comparison is made only after ledger-valued metric discrepancies are scalarized and shown to bound actual bottleneck discrepancies. Search-induced defects are no exception. If they are consumed, they must fit the same scalarized threshold-gap discipline. □

Proposition 2.75 (Search budget cannot repair non-scalar obligations)**.** *No amount of search-budget slack repairs missing constructibility, representatives, eligibility, tower artifacts, terminal-tameness evidence, manifest identity, claim status, or proof obligations.*

Proof. These requirements are non-scalar obligations. They are not components of the metric-ledger value system and cannot be discharged by a real-valued safety margin. □

Remark 2.76 (Re-entry mode and Core status)**.** The only re-entry mode that may support Core consumption is *Core verified*, and only at the exact strength of the verifier-side checks actually performed.

V.9. Hunter / Mapper / Lifter failure modes

Definition 2.77 (Hunter failure)**.** A *Hunter failure* occurs when search produces candidates that are irrelevant, unsupported, misranked, duplicate, malformed, stale, outside the declared hunt space, or inconsistent with the hunt objective.

Definition 2.78 (Mapper failure)**.** A *Mapper failure* occurs when relations, edges, dependencies, status labels, or map structures are incorrect, ambiguous, untyped, cyclic when acyclicity is required, inconsistent with artifacts, or semantically inflated.

Definition 2.79 (Lifter failure)**.** A *Lifter failure* occurs when a candidate cannot be converted into the declared target type, fails constructibility, lacks representative compatibility, lacks eligibility, lacks required tower artifacts, has unledgered defects, or lacks required artifact references.

Definition 2.80 (Re-entry failure mode)**.** A *re-entry failure mode* is a failure occurring after lifting but before Core verification, including missing manifest fields, mismatched labels, invalid artifacts, missing repaired evaluations, missing tower artifacts, or unsupported ledger entries.

Proposition 2.81 (Search failure requires status downgrade)**.** *If a Hunter, Mapper, Lifter, or re-entry failure is detected, the affected object must be downgraded to an appropriate non-verified status until repaired and rechecked.*

Proof. A search failure undermines the reliability of the affected object or its workflow status. By Appendix U, detected failure modes require status downgrade until repair and rechecking. □

Remark 2.82 (Failure mode localization)**.** The distinction among Hunter, Mapper, Lifter, and re-entry failures helps locate the repair task: search objective, relation map, Core-readability conversion, or verifier-side submission.

V.10. Search records

Definition 2.83 (Search record)**.** A *search record* is a structured record of a search process, including hunt, map, lift, verification, re-entry, and audit status.

Definition 2.84 (Hunter record)**.** A *Hunter record* contains:

(i) hunt space;

(ii) hunt objective;

(iii) search constraints;

(iv) candidates found;

(v) scores or rankings, if used;

(vi) selection policy;

(vii) rejected or blocked candidates, if tracked;

(viii) artifact references.

Definition 2.85 (Mapper record)**.** A *Mapper record* contains:

(i) mapped objects;

(ii) relation types;

(iii) edge semantics;

(iv) status labels;

(v) blocked routes;

(vi) unresolved gaps;

(vii) conflict-resolution notes;

(viii) artifact references.

Definition 2.86 (Lifter record)**.** A *Lifter record* contains:

(i) lift input;

(ii) lift target;

(iii) transformation rule;

(iv) resulting lifted object;

(v) constructibility status;

(vi) representative and eligibility status, if invoked;

(vii) tower artifact status, if invoked;

(viii) defects and ledger entries;

(ix) re-entry readiness status;

(x) artifact references.

Definition 2.87 (Re-entry record)**.** A *re-entry record* contains:

(i) the re-entry candidate;

(ii) the re-entry package;

(iii) required verifier-side data;

(iv) missing verifier-side data, if any;

(v) Core verification status;

(vi) failure modes, if any;

(vii) artifact references.

Proposition 2.88 (Search records support reproducibility)**.** *Search records support reproducibility by recording how candidates were found, mapped, lifted, and prepared for verification.*

Proof. Reproducibility requires reconstructing the workflow. Search records record the relevant inputs, outputs, actions, statuses, and artifact references. Therefore they support reconstruction. □

Remark 2.89 (Search record is not proof record)**.** A search record explains how a candidate was obtained. It does not by itself prove the candidate.

V.11. Search protocol manifest entries

Definition 2.90 (Search protocol manifest entry)**.** A *search protocol manifest entry* records:

(i) the protocol type: Hunter, Mapper, Lifter, re-entry, or hybrid;

(ii) the agent identity or class;

(iii) the search objective;

(iv) input artifacts;

(v) output artifacts;

(vi) output status labels;

(vii) scores, rankings, or confidence values, if present;

(viii) edge semantics, if mapping is used;

(ix) lift target, if lifting is used;

(x) verifier-side target, if re-entry is attempted;

(xi) failure modes, if detected;

(xii) repair history, if any;

(xiii) Core re-entry status.

Definition 2.91 (Hybrid search protocol)**.** A *hybrid search protocol* combines Hunter, Mapper, Lifter, re-entry, verification, or audit actions in a single workflow.

Proposition 2.92 (Hybrid protocol does not collapse role distinctions)**.** *A hybrid search protocol does not erase the semantic distinction among hunting, mapping, lifting, verification, re-entry, and audit.*

Proof. A hybrid workflow may combine actions operationally. However, each action has a distinct semantic role by Appendix U. Thus the distinctions must remain visible in the manifest. □

V.11bis. Artifact identity, drift, and replay limits

Definition 2.93 (Search artifact identity record)**.** A *search artifact identity record* contains the artifact identifier, content hash or immutable version, generation environment, input artifacts, output artifacts, status label, and repair ancestry.

Definition 2.94 (Stale search artifact)**.** A search artifact is *stale* when its source data, generator, normalization, dependency, artifact identity, environment, or declared scope has changed relative to the record that produced it.

Definition 2.95 (Replay-success status)**.** *Replay-success status* means that a declared search workflow was reconstructed under the stated environment and artifacts. It does not assert mathematical truth beyond the criteria replayed.

Proposition 2.96 (Stale artifact cannot be silently consumed)**.** *A stale search artifact cannot be silently reused as current verifier-side evidence or current search guidance without repair, revalidation, or explicit demotion.*

Proof. Staleness means that the artifact identity or dependency context has changed. Consuming it as current would erase repair ancestry and may mismatch the declared certificate scope. Therefore repair, revalidation, or demotion is required. □

Proposition 2.97 (Replay success is not proof)**.** *Replay success for a Hunter / Mapper / Lifter workflow does not prove the mathematical validity of the generated candidate.*

Proof. Replay reconstructs a workflow output under declared conditions. Mathematical validity requires the relevant proof or verifier-side checks. The former supports auditability, not theorem status. □

Remark 2.98 (Artifact identity is part of search safety)**.** Search protocols often produce many similar artifacts. Content identity, dependency identity, and repair ancestry prevent accidental reuse of a nearby but nonidentical candidate.

V.12. Search protocol invariants

Definition 2.99 (Search protocol invariant)**.** A *search protocol invariant* is a rule that must remain true throughout the Hunter / Mapper / Lifter workflow.

Definition 2.100 (Non-verdict invariant)**.** The *non-verdict invariant* states that no search-side object determines Core pass/reject before verified Core re-entry.

Definition 2.101 (Typed-edge invariant)**.** The *typed-edge invariant* states that every edge in a search map must have declared semantics.

Definition 2.102 (Lift-target invariant)**.** The *lift-target invariant* states that every lift must declare its target type before the lifted output is used.

Definition 2.103 (Re-entry invariant)**.** The *re-entry invariant* states that no lifted candidate may be treated as Core-certified until it passes the required verifier-side checks.

Proposition 2.104 (Search protocol invariants prevent claim inflation)**.** *The non-verdict, typed-edge, lift-target, and re-entry invariants prevent search artifacts from being silently promoted into Core claims.*

Proof. The non-verdict invariant blocks search-side acceptance. The typed-edge invariant blocks ambiguous map implications. The lift-target invariant blocks untyped conversion. The re-entry invariant blocks treating prepared candidates as verified certificates. Together, these rules prevent claim inflation. □

V.12bis. Stable-band and Type IV proxy discipline

Definition 2.105 (Search stable-band candidate)**.** A *search stable-band candidate* is a threshold interval, window family, or parameter region that search suggests may support stable verifier-side behavior. It is not a Core stable band unless actual diagnostic or threshold-gap evidence is supplied throughout the declared band, or a theorem reduces the band claim to certified finite data.

Definition 2.106 (Search Type IV proxy)**.** A *search Type IV proxy* is a search-side signal suggesting possible terminal-generic kernel or cokernel behavior in a tower. It does not define μ_{Collapse} or u_{Collapse}.

Definition 2.107 (Search-produced Type IV diagnostic candidate)**.** A *search-produced Type IV diagnostic candidate* is a record proposing a declared monitored scope and a candidate tower artifact of the form

$$\phi^{\mathrm{search}}_{i,W,\tau} : \mathrm{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \mathrm{ApexProfile}_{i,W,\tau}(F_\infty).$$

It is not a Type IV diagnostic until the actual artifact, terminal-tameness evidence, and tgdim_W kernel/cokernel computations are verified.

Proposition 2.108 (Stable-band candidate is not Core stable band)**.** *A search stable-band candidate is not a Core stable band by sampling, ranking, visual continuity, or agent agreement alone.*

Proof. A Core stable band is a verifier-side statement over the declared band. Search sampling or ranking supplies only candidate evidence. Universal diagnostic or threshold-gap evidence, or a theorem reducing the universal claim to finite data, is required. □

Proposition 2.109 (Search Type IV proxy is not diagnosis)**.** *A search Type IV proxy does not determine*

$$(\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}) = (0, 0)$$

or its negation.

Proof. The Type IV diagnostic is computed from the kernel and cokernel of a declared tower comparison artifact on a terminal-tame monitored scope. A proxy supplies neither the artifact nor the terminal-generic computation by default. □

Remark 2.110 (Do not encode missing Type IV as zero)**.** If a search-produced tower artifact is missing, ill-typed, stale, or not terminal-tame, the corresponding Type IV status is undefined or not invoked, not $(0, 0)$.

V.13. Summary of Hunter / Mapper / Lifter protocols

Theorem 2.111 (Appendix V search-protocol package)**.** *Within the Search / Platform layer, the following package is available.*

(i) *Search objects and search artifacts are non-verdict by default.*

(ii) *Hunters generate candidates from declared hunt spaces.*

(iii) *Hunter outputs are not Core certificates.*

(iv) *Mappers organize candidates into maps, diagrams, dependencies, validity landscapes, or classifications.*

(v) *Mapper outputs are navigation artifacts, not theorems.*

(vi) *Map edges must have declared semantics.*

(vii) *Lifters attempt to transform candidates into Core-readable objects, bridge candidates, certificate candidates, formal fragments, or proof-object candidates.*

(viii) *Lift outputs may include candidates for*

$$\mathrm{Eval}_{i,W,\tau}, \qquad C_{\tau,W}F, \qquad \phi_{i,W,\tau},$$

but these are not verified merely by lifting.

(ix) *Core-looking lift outputs are not Core verified until checked.*

(x) *Search lifecycle status records workflow progress, not mathematical validity.*

(xi) *Search scores are not proof.*

(xii) *Search-to-Core re-entry requires verifier-side manifest data.*

(xiii) *Re-entry candidates are not Core certificates until verified.*

(xiv) *Hunter, Mapper, Lifter, and re-entry failures require status downgrade.*

(xv) *Search records support reproducibility but do not prove validity.*

(xvi) *Hybrid search protocols must preserve semantic role distinctions.*

(xvii) *Search protocol invariants prevent claim inflation.*

Proof. Item (i) is Proposition 2.14. Items (ii)–(iii) follow from the Hunter definitions and Proposition 2.21. Items (iv)–(vi) follow from the Mapper definitions, Proposition 2.28, and the typed-edge discipline. Items (vii)–(ix) follow from the Lifter definitions and Propositions 2.40 and 2.41. Item (x) is Proposition 2.53. Item (xi) is Proposition 2.58. Items (xii)–$(xiii)$ follow from the Search-to-Core re-entry definitions and Proposition 2.70. Item (xiv) is Proposition 2.81. Item (xv) follows from Proposition 2.88 and Remark 2.89. Item (xvi) is Proposition 2.92. Item $(xvii)$ is Proposition 2.104. □

V.13bis. v19 search-protocol hardening package

Theorem 2.112 (Appendix V v19 search hardening package)**.** *Within the Search / Platform layer, the following additional v19.0 hardening package is available.*

(i) *Search sovereignty prevents search-side records from overriding verifier-side Core requirements.*

(ii) *Search-to-verifier conversion requires a declared conversion record.*

(iii) *Map-edge verification status must be typed and evidence-backed.*

(iv) *Candidate implication edges do not establish implications.*

(v) *Lifting does not preserve mathematical content by default.*

(vi) *Search-induced defects must be typed before budget use.*

(vii) *Scores are not metric-ledger components without certified conversion.*

(viii) *Agent consensus is not verification.*

(ix) *Search-adjusted metric evidence must satisfy a scalarized threshold-gap budget.*

(x) *Search budget slack does not repair non-scalar obligations.*

(xi) *Stale artifacts cannot be silently consumed.*

(xii) *Replay success is not proof.*

(xiii) *Stable-band candidates are not Core stable bands.*

(xiv) *Search Type IV proxies are not Type IV diagnostics.*

(xv) *Missing or undefined Type IV data are not encoded as a zero diagnostic.*

Proof. Items (i)–(ii) follow from Definitions 2.4 and 2.5, together with Propositions 2.7 and 2.8. Items (iii)–(iv) follow from Definitions 2.30 and 2.31, and Propositions 2.32 and 2.33. Items (v)–(vi) follow from Definitions 2.43, 2.44, and 2.45, and Propositions 2.46 and 2.47. Items (vii)–$(viii)$ follow from Definitions 2.60, 2.61, and 2.62, and Propositions 2.63 and 2.64. Items (ix)–(x) are Propositions 2.74 and 2.75. Items (xi)–(xii) are Propositions 2.96 and 2.97. Items $(xiii)$–(xv) follow from Definitions 2.105, 2.106, and 2.107, Propositions 2.108 and 2.109, and Remark 2.110. □

V.14. Explicit exclusions

The following are not asserted in Appendix V.

- No Hunter output is treated as proof.
- No Mapper output is treated as theorem.
- No Lifter output is treated as verified merely by lifting.
- No search score is treated as validity.
- No agent confidence value is treated as a Core verdict.

- No search artifact updates Core pass/reject.
- No search map edge is treated as implication unless typed and verified.
- No candidate stable band is treated as Core stable band without diagnostics.
- No candidate bridge is treated as bridge theorem.
- No candidate repaired evaluation record
$$\mathrm{Eval}_{i,W,\tau}(F)$$
is treated as verified merely by being produced.
- No candidate admissible representative
$$C_{\tau,W}F$$
is treated as verified merely by being produced.
- No candidate tower artifact
$$\phi_{i,W,\tau}$$
is treated as verified merely by being produced.
- No re-entry candidate is treated as Core certificate before verification.
- No search record is treated as proof record.
- No hybrid search workflow collapses proposer and verifier roles.
- No search protocol replaces B-Gate^{+}.
- No search protocol replaces the repaired topological condition
$$\mathrm{Eval}_{1,W,\tau}(F) = 0.$$
- No search protocol replaces admissible representative records.
- No search protocol replaces monitored Type IV diagnostics.
- No search protocol replaces the tower comparison artifact
$$\phi_{i,W,\tau}.$$
- No search protocol replaces the budget condition
$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}).$$
- No search-to-verifier conversion is achieved merely by relabeling.
- No candidate implication edge is treated as a proved implication.
- No map visualization, color, layout, or clustering is treated as logical evidence.
- No lift is treated as preservation without a lift preservation record.
- No search score is treated as a metric-ledger component without certified score-to-budget conversion.

- No agent consensus is treated as verification.
- No search-adjusted metric comparison bypasses

$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{search}}) < \mathrm{gap}_\tau(\mathfrak{B}_{\mathrm{search}}).$$

- No search-budget slack repairs missing non-scalar obligations.
- No stale search artifact is silently reused as current evidence.
- No replay success is treated as proof.
- No search stable-band candidate is treated as a Core stable band.
- No search Type IV proxy is treated as a Type IV diagnostic.
- No missing, not-invoked, or undefined Type IV diagnostic is encoded as $(0, 0)$.

V.15. Provenance and status

The material in this appendix depends on:

(i) the claim taxonomy and Minimal Manifest discipline of Chapter 1;

(ii) the Core sovereignty doctrine of Chapter 7;

(iii) the non-claim discipline, Core/interface boundary, and interface protocol of Chapter 8;

(iv) the Agent Semantics of Appendix U;

(v) the claim-use preflight and repair-governance discipline of Appendix CR-v19;

(vi) the repaired evaluation discipline of Appendix A;

(vii) the admissible representative discipline of Appendix B;

(viii) the eligible realization regime of Appendix C;

(ix) the monitored Type IV diagnostic and tower artifact discipline of Appendix D;

(x) the manifest schema of Appendix G;

(xi) the advisory-invariant discipline of Appendix H;

(xii) the ledger catalog of Appendix M.

Its role is to define how search processes may discover, organize, and prepare candidates without converting search into proof.

Status labels for Appendix V. *Search / Platform definitions:* Definitions 2.10, 2.11, 2.12, 2.13, 2.16, 2.17, 2.18, 2.19, 2.20, 2.23, 2.24, 2.25, 2.26, 2.27, 2.35, 2.36, 2.37, 2.38, 2.39, 2.49, 2.50, 2.51, 2.55, 2.56, 2.57, 2.66, 2.67, 2.68, 2.69, 2.77, 2.78, 2.79, 2.80, 2.83, 2.84, 2.85, 2.86, 2.87, 2.90, 2.91, 2.99, 2.100, 2.101, 2.102, 2.103.

Search / Platform propositions/theorems: Propositions 2.14, 2.21, 2.28, 2.40, 2.41, 2.53, 2.58, 2.70, 2.81, 2.88, 2.92, 2.104, Theorem 2.111.

Operational cautions: Remarks 2.1, 2.2, 2.3, 2.15, 2.22, 2.29, 2.42, 2.54, 2.59, 2.71, 2.82, 2.89.

Additional v19.0 hardening definitions: Definitions 2.4, 2.5, 2.6, 2.30, 2.31, 2.43, 2.44, 2.45, 2.60, 2.61, 2.62, 2.72, 2.73, 2.93, 2.94, 2.95, 2.105, 2.106, 2.107.

Additional v19.0 hardening propositions/theorems: Propositions 2.7, 2.8, 2.32, 2.33, 2.46, 2.47, 2.63, 2.64, 2.74, 2.75, 2.96, 2.97, 2.108, 2.109, Theorem 2.112.

Additional v19.0 hardening remarks: Remarks 2.9, 2.52, 2.34, 2.48, 2.65, 2.76, 2.98, 2.110.

Explicit exclusions: Section V.14.

3 Appendix W: Bridge Programs

W.1. Scope and program status

This appendix defines Bridge Programs in the AK framework. It belongs to the Search / Platform Appendices. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

A Bridge Program is not a bridge theorem.

A Bridge Program organizes the tasks, hypotheses, artifacts, verification targets, failure modes, proof obligations, and promotion conditions required to turn a proposed cross-layer implication into a theorem. It does not itself prove that implication.

In particular, no Bridge Program, bridge draft, bridge candidate, bridge roadmap, dependency graph, agent recommendation, or proof-first planning artifact may replace:

$$B\text{-Gate}^{+}, \qquad \mathrm{Eval}_{1,W,\tau}(F) = 0, \qquad C_{\tau,W}F, \qquad \phi_{i,W,\tau},$$
$$(\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}) = (0,0), \qquad \mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}).$$

Remark 3.1 (Search / Platform status)**.** Bridge Programs live in the Search / Platform layer. They may coordinate work across Core, Technical Extension, Problem Interface, and proof-first layers, but they do not change the claim status of any theorem by themselves.

Remark 3.2 (Relation to Appendices U and V)**.** Appendix U defines agent semantics. Appendix V defines Hunter / Mapper / Lifter protocols. The present appendix defines how candidate bridges discovered, mapped, or lifted by those protocols are organized into auditable proof programs.

Remark 3.3 (Relation to Core re-entry)**.** A Bridge Program may prepare material for Core re-entry or for a problem-interface soundness bridge. However, re-entry requires verifier-side data, including repaired evaluation records, admissible representatives when invoked, tower artifacts when invoked, diagnostics, gate verdicts, ledgers, and artifact references.

W.1bis. Bridge Program sovereignty and verifier-side re-entry

Definition 3.4 (Bridge Program sovereignty)**.** *Bridge Program sovereignty* is the rule that a Bridge Program may organize proposed cross-layer implications but may not itself establish any Core verdict, interface theorem, external-domain theorem, or bridge theorem. Only the declared verifier-side objects, proof objects, discharged hypotheses, and registered claim-use permissions determine theorem-level consumption.

Definition 3.5 (Bridge verifier-side conversion record)**.** A *bridge verifier-side conversion record* is the record by which a Bridge Program output is converted into verifier-side or theorem-level evidence. It must specify:

(i) the source Bridge Program artifact;

(ii) the target verifier-side record type or bridge theorem statement;

(iii) the source and target semantics;

(iv) the conversion theorem, proof object, verified artifact, or valid import supporting the conversion;

(v) all hypotheses and their discharge status;

(vi) the comparison mode, if any;

(vii) metric defects and non-scalar obligations;

(viii) result status, one of *pass*, *reject*, or *undefined*;

(ix) typed failure route;

(x) immutable artifact references and claim-use preflight data.

This record is a bridge-specialized instance of the conversion discipline of Appendices U and V.

Proposition 3.6 (Bridge Program sovereignty cannot be overridden)**.** *No Bridge Program label, roadmap, dependency graph, proof-first alignment, human approval, AI assessment, search score, or platform status overrides the verifier-side requirements for a Core certificate or bridge theorem.*

Proof. Bridge Programs are proof-management records. The Core and bridge-theorem layers consume declared proof data, verifier-side artifacts, discharged hypotheses, ledgers, comparison records, and claim-governance permissions. A program label or roadmap is not one of those consumed records. Therefore it cannot override the required verifier-side conditions. □

Remark 3.7 (Program meaning is proposer-side until converted)**.** A Bridge Program may be mathematically insightful and strategically central. Nevertheless, until its outputs are converted through a bridge verifier-side conversion record, they remain proposer-side or program-side data.

W.2. Bridge statements

Definition 3.8 (Bridge statement)**.** A *bridge statement* is a statement connecting two semantic layers of the AK framework. Typical forms include:

$$\text{external domain property} \Longrightarrow \text{Core-visible behavior},$$

$$\text{Core certificate behavior} \Longrightarrow \text{external domain conclusion},$$

$$\text{technical extension condition} \Longrightarrow \text{Core-readable certificate condition},$$

or

$$\text{formal statement} \Longrightarrow \text{intended informal mathematical claim}.$$

Definition 3.9 (Core-facing bridge statement)**.** A *Core-facing bridge statement* is a bridge statement whose source or target involves verifier-side Core data, such as:

$$\mathrm{Eval}_{i,W,\tau}(F), \qquad C_{\tau,W}F, \qquad \phi_{i,W,\tau},$$
$$(\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}), \qquad \mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}).$$

Definition 3.10 (Problem-facing bridge statement)**.** A *problem-facing bridge statement* is a bridge statement whose source or target is a problem-specific conclusion, such as arithmetic finiteness, Iwasawa control, PDE regularity, categorical equivalence, Fukaya-side unobstructedness, or cryptographic security.

Definition 3.11 (Bridge theorem)**.** A *bridge theorem* is a bridge statement equipped with a complete proof under declared hypotheses, status labels, artifact references, and verifier-side reconstruction data.

Definition 3.12 (Bridge candidate)**.** A *bridge candidate* is a bridge statement proposed for possible proof but not yet established as a bridge theorem.

Definition 3.13 (Bridge draft)**.** A *bridge draft* is an informal, partial, agent-generated, exploratory, or schematic version of a bridge candidate.

Definition 3.14 (Bridge Program)**.** A *Bridge Program* is a structured project record whose purpose is to turn one or more bridge candidates into bridge theorems or to classify why they fail.

Proposition 3.15 (Bridge candidate is not bridge theorem)**.** *A bridge candidate is not a bridge theorem.*

Proof. A bridge candidate is a proposed implication. A bridge theorem requires a proof under declared hypotheses. Proposal and proof are distinct statuses by Appendix U. □

Proposition 3.16 (Bridge draft is not bridge candidate unless specified)**.** *A bridge draft is not a bridge candidate unless its statement, source semantics, target semantics, and intended status are explicitly specified.*

Proof. A draft may be informal, incomplete, ambiguous, or schematic. A bridge candidate requires a definite statement to be evaluated, mapped, decomposed, or proved. Thus a draft must be specified before it becomes a candidate. □

Remark 3.17 (Bridge terminology caution)**.** The word "bridge" should not be read as proof. Only a proved bridge theorem can support theorem-level transport across layers.

W.2bis. Bridge statement specificity and directionality

Definition 3.18 (Bridge statement specificity record)**.** A *bridge statement specificity record* records the exact source semantics, target semantics, direction of implication, hypotheses, allowed use, forbidden stronger reading, and failure behavior of a bridge statement. A bridge statement without such a record is treated as a draft or [Spec] bridge, not as theorem evidence.

Definition 3.19 (Bridge directionality record)**.** A *bridge directionality record* declares whether the intended implication is source-to-Core, Core-to-source, technical-to-Core, formal-to-informal, or bidirectional. A bidirectional bridge requires two separately proved implications unless a single theorem explicitly proves the equivalence at the declared strength.

Proposition 3.20 (Bridge direction is not reversible by name)**.** *A proved bridge in one direction does not establish the reverse bridge unless the reverse implication is separately proved or validly imported with its own hypotheses and artifacts.*

Proof. Bridge statements connect different semantic layers. The hypotheses and constructions required for one direction need not imply those required for the reverse direction. Thus direction reversal is an additional theorem obligation, not a naming convention. □

Proposition 3.21 (External theorem is not Core theorem by import name)**.** *An external theorem, even if mathematically correct in its source domain, is not a Core theorem or Core certificate unless a bridge theorem supplies the declared realization, interpretation, artifact, ledger, and claim-governance data required for Core consumption.*

Proof. External theorems and Core certificates have different source semantics and verification schemas. Core consumption requires the Core-readable records and claim-use preflight data. A theorem name in the external domain supplies neither by default. □

Remark 3.22 (No bridge theorem from slogan form)**.** A slogan such as "Core collapse implies the problem theorem" is not a bridge statement at theorem level. It becomes a bridge candidate only after the source semantics, target semantics, direction, hypotheses, proof obligations, and failure routes are fixed.

W.3. Bridge Program types

Definition 3.23 (Core-to-interface Bridge Program)**.** A *Core-to-interface Bridge Program* attempts to prove a statement of the form:

$$\text{Core certificate behavior} \Longrightarrow \text{problem-specific conclusion.}$$

Definition 3.24 (Interface-to-Core Bridge Program)**.** An *Interface-to-Core Bridge Program* attempts to prove a statement of the form:

$$\text{problem-specific or external structure} \Longrightarrow \text{Core-readable behavior.}$$

Definition 3.25 (Technical-to-Core Bridge Program)**.** A *Technical-to-Core Bridge Program* attempts to prove that technical-extension data, such as discretization, measurement, controlled commutation, Mirror/Transfer comparison, derived realization, or higher-categorical structure, safely re-enter the Core verifier.

Definition 3.26 (Search-to-proof Bridge Program)**.** A *Search-to-proof Bridge Program* attempts to convert search artifacts, candidate maps, lifted objects, bridge drafts, or advisory signals into proof objects, verified formal fragments, bridge theorems, or Core certificate candidates.

Definition 3.27 (Formal-to-informal Bridge Program)**.** A *Formal-to-informal Bridge Program* attempts to prove that a verified formal fragment or proof assistant theorem corresponds to the intended informal mathematical statement.

Definition 3.28 (Problem-to-proof-first Bridge Program)**.** A *Problem-to-proof-first Bridge Program* attempts to decompose a problem-specific theorem target into proof-first records, required bridge lemmas, Core certificate targets, analytic or arithmetic hypotheses, and proof object requirements.

Remark 3.29 (Bridge Program classes may overlap)**.** A single Bridge Program may combine several types. For example, a Navier–Stokes regularity bridge may require interface-to-Core realization, technical-to-Core discretization soundness, Core-to-interface regularity transport, and formal-to-informal interpretation.

W.4. Bridge Program record

Definition 3.30 (Bridge Program record)**.** A *Bridge Program record* contains:

(i) Bridge Program identifier;

(ii) bridge statement;

(iii) Bridge Program type;

(iv) source layer;

(v) target layer;

(vi) intended claim status;

(vii) source semantics;

(viii) target semantics;

(ix) hypotheses;

(x) required bridge lemmas;

(xi) required artifacts;

(xii) verification targets;

(xiii) required Core-facing objects, if any;

(xiv) required problem-facing objects, if any;

(xv) defect and ledger treatment;

(xvi) failure modes;

(xvii) current status;

(xviii) responsible agents or agent roles;

(xix) dependency links to other programs;

(xx) audit notes.

Definition 3.31 (Bridge Program status)**.** A Bridge Program has one of the following statuses:

(i) *proposed*;

(ii) *specified*;

(iii) *mapped*;

(iv) *decomposed*;

(v) *partially proved*;

(vi) *blocked*;

(vii) *failed*;

(viii) *verified as bridge theorem*;

(ix) *imported theorem*;

(x) *rejected*;

(xi) *superseded*;

(xii) [Spec] *only*.

Definition 3.32 (Bridge Program claim status)**.** The *claim status* of a Bridge Program is the strongest claim status justified by its current proof, artifact, and verification data. It may be lower than its intended status.

Proposition 3.33 (Program status is not theorem status)**.** *A Bridge Program status does not imply theorem status unless the status is explicitly verified as bridge theorem or imported theorem and supported by proof data.*

Proof. Most Bridge Program statuses describe project progress, not mathematical proof. Only a verified theorem status or valid imported theorem status, backed by proof data and audit references, establishes theorem-level status. □

Remark 3.34 (Status conservatism)**.** Bridge Program status should be conservative. A partially proved, specified, mapped, or decomposed program must not be described as a theorem.

W.4bis. Bridge Program status axes and promotion governance

Definition 3.35 (Bridge workflow status field)**.** The Bridge Program statuses of Definition 3.31 are workflow fields. They do not replace the agent-output status labels of Appendix U, the claim taxonomy of Chapter 1, or the evidence and invocation statuses of Appendix CR-v19. When a Bridge Program is consumed by the AK workflow, the record must carry both its workflow status and its claim-governance status.

Definition 3.36 (Bridge promotion certificate)**.** A *bridge promotion certificate* is the audit-readable record authorizing a promotion from bridge draft, bridge candidate, or Bridge Program status to bridge theorem or validly imported theorem status. It contains the precise statement, proof or valid import, discharged hypotheses, dependency closure, comparison-mode policy, defect treatment, non-claim compatibility, CR-v19 claim-use preflight result, artifact identities, and repair ancestry.

Proposition 3.37 (Workflow progress does not imply claim promotion)**.** *A Bridge Program becoming specified, mapped, decomposed, or partially proved does not promote its bridge statement to theorem evidence.*

Proof. Those statuses record project progress. Theorem evidence requires a bridge promotion certificate or a registered theorem-level source with all hypotheses discharged. Project progress alone supplies neither. □

Proposition 3.38 (Promotion certificate requires claim-use preflight)**.** *A bridge promotion certificate is invalid unless the promoted item passes the applicable CR-v19 claim-use preflight.*

Proof. The promoted bridge is a claim-bearing item. CR-v19 governs identity, source binding, evidence status, invocation role, permitted use, forbidden stronger reading, failure routing, and repair ancestry. If these fields do not pass preflight, the bridge cannot be consumed as theorem evidence. □

Remark 3.39 (Bridge status vocabulary is two-axis)**.** Terms such as *specified*, *mapped*, and *decomposed* are workflow-status tokens. Terms such as *proved*, *conditional*, *operational*, *spec*, and *rejected* are claim-governance tokens. The two axes are recorded together but are not interchangeable.

W.5. Bridge hypotheses

Definition 3.40 (Bridge hypothesis)**.** A *bridge hypothesis* is an assumption required for a bridge statement to be meaningful or provable. Examples include:

(i) realization faithfulness;

(ii) constructibility;

(iii) repaired evaluation compatibility;

(iv) admissible representative existence;

(v) eligible amplitude;

(vi) edge realization;

(vii) declared tower comparison artifact existence;

(viii) diagnostic convergence;

(ix) grid-to-continuum convergence;

(x) measurement soundness;

(xi) normalization transport;

(xii) categorical comparison compatibility;

(xiii) formal interpretation compatibility;

(xiv) analytic regularity criteria;

(xv) arithmetic control conditions.

Definition 3.41 (Bridge hypothesis status)**.** Each bridge hypothesis must be labeled as:

(i) proved;

(ii) imported theorem;

(iii) verified artifact;

(iv) operational policy;

(v) [Spec];

(vi) unresolved;

(vii) rejected;

(viii) failed.

Definition 3.42 (Hypothesis discharge)**.** A *hypothesis discharge* is a proof, valid import, verified artifact, or accepted operational policy that justifies a bridge hypothesis under the declared status.

Proposition 3.43 ([Spec] hypotheses prevent theorem-level bridge)**.** *A bridge statement depending on an unresolved* [Spec] *hypothesis is not a bridge theorem.*

Proof. A theorem requires all necessary hypotheses and implications to be proved or validly imported. An unresolved [Spec] hypothesis is not proved. Therefore the bridge statement cannot be theorem-level. □

Remark 3.44 (Spec is allowed but must be labeled)**.** A Bridge Program may contain [Spec] hypotheses. This is useful for research planning. However, they block theorem-level status until discharged.

W.6. Bridge lemmas

Definition 3.45 (Bridge lemma)**.** A *bridge lemma* is a smaller claim required to prove a bridge theorem.

Definition 3.46 (Core-facing bridge lemma)**.** A *Core-facing bridge lemma* is a bridge lemma involving Core-readable data, such as repaired evaluation records, admissible representatives, eligible realizations, tower comparison artifacts, diagnostics, gates, or ledgers.

Definition 3.47 (Problem-facing bridge lemma)**.** A *problem-facing bridge lemma* is a bridge lemma involving external semantic content, such as arithmetic, PDE, categorical, geometric, cryptographic, or formal-interpretation claims.

Definition 3.48 (Bridge lemma graph)**.** A *bridge lemma graph* is a directed graph whose nodes are bridge lemmas and whose edges represent dependency relations among them.

Definition 3.49 (Discharge condition)**.** A *discharge condition* is a criterion under which a bridge lemma is considered proved, imported, verified, rejected, or blocked.

Proposition 3.50 (All necessary bridge lemmas must be discharged)**.** *A Bridge Program cannot yield a bridge theorem unless all necessary bridge lemmas are discharged.*

Proof. A bridge theorem depends on its required lemmas. If any necessary lemma is unresolved, the proof is incomplete. Therefore all necessary bridge lemmas must be discharged. □

Remark 3.51 (Bridge lemma graph links to certificate DAG)**.** The bridge lemma graph becomes part of the global certificate DAG described in the later validity-map and certificate-DAG appendix.

W.7. Core-facing bridge obligations

Definition 3.52 (Core-facing bridge obligation)**.** A *Core-facing bridge obligation* is a required proof obligation connecting bridge data to verifier-side Core data. It may require:

(i) construction of a Core-readable object;

(ii) construction of repaired evaluation records

$$\mathrm{Eval}_{i,W,\tau}(F);$$

(iii) construction of admissible representative records

$$C_{\tau,W}F;$$

(iv) proof of eligibility when the Ext^1-clause is invoked;

(v) construction of tower comparison artifacts

$$\phi_{i,W,\tau};$$

(vi) computation or proof of monitored diagnostics;

(vii) gate verdict support;

(viii) ledger and budget support.

Definition 3.53 (Core bridge payload)**.** A *Core bridge payload* is the collection of verifier-side Core data produced or required by a Bridge Program.

Proposition 3.54 (Core-facing bridge obligations do not verify themselves)**.** *A Bridge Program listing Core-facing obligations does not verify those obligations.*

Proof. Listing an obligation records what must be proved or checked. Verification requires actual verifier-side data and the declared checking process. Therefore the obligation list is not self-verifying. □

Remark 3.55 (Core bridge payload remains candidate until checked)**.** A Core bridge payload produced by a Bridge Program remains candidate data until checked under the Core protocol.

W.8. Bridge defects and ledger integration

Definition 3.56 (Bridge defect)**.** A *bridge defect* is a declared discrepancy, approximation, comparison cost, uncertainty, semantic mismatch, or transport loss introduced by a bridge construction.

Definition 3.57 (Bridge ledger component)**.** A *bridge ledger component* is a ledger entry

$$\delta^{\mathrm{bridge}}$$

recording the aggregate defect of a Bridge Program when its output is used in a certificate. When primitive bridge discrepancies are declared in Definition 3.63, δ^{bridge} denotes the declared aggregate, alias, or refinement of exactly those invoked primitive components, subject to the alias/refinement/aggregate/disjointness declarations required by Appendix M.

Definition 3.58 (Bridge-adjusted metric-ledger value)**.** If a certificate relies on a bridge with nonzero metric bridge defect, its bridge-adjusted metric-ledger value is a declared local alias

$$\mathfrak{d}_{\mathrm{bridge}} := \mathfrak{d}_{\mathrm{base}} \oplus \delta^{\mathrm{bridge}},$$

where aggregation is interpreted in the declared metric-ledger value system. A certified metric use additionally requires a declared bottleneck scalarization

$$\mathrm{val}_B$$

and, for each consumed comparison pair of actual windowed pre-threshold profiles $B^{(r)}_{\mathrm{source}}, B^{(r)}_{\mathrm{target}}$ in the protected family $\mathfrak{B}_{\mathrm{bridge}}$, evidence that

$$d_B\big(B^{(r)}_{\mathrm{source}}, B^{(r)}_{\mathrm{target}}\big) \le \mathrm{val}_B(\mathfrak{d}_{\mathrm{bridge}}).$$

Equivalently, if a family-level shorthand is used, it denotes the supremum over these declared componentwise bottleneck bounds; no bottleneck distance is applied directly to the family symbol $\mathfrak{B}$ without such a declaration. The symbol $\mathfrak{d}_{\mathrm{bridge}}$ is not a global Core invariant unless Appendix M registers the corresponding alias/refinement/disjointness relation to the gate-scoped $\mathfrak{d}_{\mathrm{met}}$. Such registration is a governance condition, not a consequence of notation.

Definition 3.59 (Bridge defect status)**.** A bridge defect has one of the following statuses:

(i) zero;

(ii) bounded;

(iii) ledgered;

(iv) unbounded;

(v) unresolved;

(vi) failed.

Proposition 3.60 (Bridge defects affect Core only through ledger)**.** *Bridge defects affect Core audit only if they are included in the declared δ-ledger.*

Proof. Bridge defects are not gate verdicts or monitored diagnostics. They affect Core audit only by contributing to the total budget. □

Proposition 3.61 (Bridge-adjusted budget must pass)**.** *A certificate relying on a nonzero metric bridge defect may pass Audit Mode only if*

$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{bridge}}) < \mathrm{gap}_\tau(\mathfrak{B}_{\mathrm{bridge}}),$$

and the scalarized value is supported as an upper bound on the actual bottleneck discrepancy of the declared protected pre-threshold profile family $\mathfrak{B}_{\mathrm{bridge}}$.

Proof. By the v19 typed ledger and audit contract of Chapter 7 and the strict Core budget discipline of Appendix M, every metric discrepancy consumed by the certificate must be included in the declared ledger value system, scalarized by the declared bottleneck scalarization, shown to upper-bound the actual windowed pre-threshold bottleneck discrepancy, and kept strictly below the declared threshold clearance. The metric-repair theorem of Chapter 1 and Appendix A is then available only after that threshold-gap record has the required componentwise support. Non-scalar bridge obligations remain separate and cannot be repaired by this numerical inequality. □

Remark 3.62 (Semantic mismatch is a defect)**.** If a bridge changes semantics, weakens assumptions, strengthens conclusions, replaces a continuum object by a discrete object, or interprets a formal statement as an informal claim, the mismatch must be eliminated by theorem or ledgered as a defect.

W.8bis. Bridge defect namespace, non-compensation, and Type IV artifact discipline

Definition 3.63 (Primitive bridge discrepancy)**.** A *primitive bridge discrepancy* is a declared bridge-side metric discrepancy before aggregation. Typical bridge-program namespaces include

$$\delta^{\text{bridge-real}},\quad \delta^{\text{bridge-interp}},\quad \delta^{\text{bridge-formal}},\quad \delta^{\text{bridge-disc}},\quad \delta^{\text{bridge-transport}},\quad \Delta_{\phi,\text{bridge}}.$$

Each occurrence must declare whether it is an alias, refinement, aggregate component, or disjoint component relative to the global ledger catalog of Appendix M.

Definition 3.64 (Bridge non-scalar obligation)**.** A *bridge non-scalar obligation* is a bridge requirement that is not a metric discrepancy and therefore cannot be discharged by numerical slack. Examples include source theorem validity, interpretation theorem existence, realization faithfulness, representative existence, eligibility, actual tower artifact identity, terminal tameness, hypothesis discharge, and claim-use permission.

Definition 3.65 (Bridge-produced Type IV candidate)**.** A *bridge-produced Type IV candidate* is a Bridge Program output purporting to supply, compare, or interpret a Type IV artifact. When Type IV is invoked, the required artifact has the declared form

$$\phi^{\mathrm{bridge}}_{i,W,\tau} : \mathrm{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \mathrm{ApexProfile}_{i,W,\tau}(F_\infty),$$

with source, target, kernel, and cokernel terminal-tame on the declared monitored scope before any numerical diagnostic is reported.

Proposition 3.66 (Bridge numerical slack does not repair non-scalar obligations)**.** *The inequality*

$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{bridge}}) < \mathrm{gap}_\tau(\mathfrak{B}_{\mathrm{bridge}})$$

does not discharge missing bridge non-scalar obligations.

Proof. The inequality certifies only a scalarized metric discrepancy bound for the declared pre-threshold profiles. Non-scalar obligations concern existence, identity, theorem status, interpretation, eligibility, artifact construction, terminal tameness, and governance permission. These are independent proof obligations and are outside the metric-ledger value. □

Proposition 3.67 (Bridge Type IV candidate is not Type IV zero)**.** *A Bridge Program output that proposes a terminal comparison, stable tower, or zero diagnostic does not establish* $(\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}) = (0, 0)$ *unless the actual artifact, terminal-tameness evidence, and window-relative terminal-generic kernel and cokernel computations are supplied and verified.*

Proof. The Type IV diagnostic is defined from the actual comparison artifact and the terminal-generic dimensions of its kernel and cokernel. A program proposal or tower intuition is not such an artifact or computation. If the artifact or tameness data are missing, the status is undefined or not invoked, not certified zero. □

Remark 3.68 (Bridge ledger catalog registration)**.** The symbols introduced in Definition 3.63 are local Bridge Program namespaces until the global ledger catalog registers them. A certificate that consumes them must declare their relation to the gate-scoped $\mathfrak{d}_{\mathrm{met}}$, avoid double counting with interface/search defects such as $\delta^{\mathrm{reentry}}$ or $\Delta_{\phi,\mathrm{search}}$, and state whether a component is an alias, refinement, aggregate, or disjoint contribution.

W.9. Bridge Program and Core sovereignty

Theorem 3.69 (Bridge Programs do not update Core verdicts)**.** *A Bridge Program does not update Core pass/reject unless it produces verifier-side data that are checked by the Core verifier.*

Proof. A Bridge Program is a search and proof-management object. By Core sovereignty, Core verdicts are determined only by verifier-side gates, diagnostics, repaired evaluation records, representative and eligibility records when invoked, tower artifacts when invoked, ledgers, and manifest data. Thus the program itself does not update Core verdicts. □

Remark 3.70 (Bridge Program as coordination layer)**.** A Bridge Program coordinates proof work. It does not bypass proof work.

W.10. Bridge Program failure modes

Definition 3.71 (Bridge Program failure)**.** A *Bridge Program failure* occurs when a Bridge Program cannot establish its intended bridge theorem under the declared hypotheses.

Definition 3.72 (Hypothesis failure)**.** *Hypothesis failure* occurs when a required bridge hypothesis is false, unsupported, unavailable, or marked as failed.

Definition 3.73 (Lemma failure)**.** *Lemma failure* occurs when a required bridge lemma cannot be proved, imported, verified, or discharged.

Definition 3.74 (Realization failure)**.** *Realization failure* occurs when the source object cannot be realized into the target layer in the required form.

Definition 3.75 (Core payload failure)**.** *Core payload failure* occurs when the Bridge Program cannot produce required verifier-side Core data, such as:

$$\mathrm{Eval}_{i,W,\tau}(F), \qquad C_{\tau,W}F, \qquad \phi_{i,W,\tau},$$

or the required diagnostic, gate, or ledger records.

Definition 3.76 (Transport failure)**.** *Transport failure* occurs when the intended implication between source and target semantics cannot be established.

Definition 3.77 (Ledger failure)**.** *Ledger failure* occurs when bridge defects cannot be bounded or when the bridge-adjusted budget fails:

$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{bridge}}) \geq \mathrm{gap}_\tau(\mathfrak{B}_{\mathrm{bridge}}).$$

Definition 3.78 (Interpretation failure)**.** *Interpretation failure* occurs when a formally verified or Core verified statement cannot be validly interpreted as the intended external or problem-specific statement.

Definition 3.79 (Promotion failure)**.** *Promotion failure* occurs when a Bridge Program cannot satisfy the evidence required to promote a bridge draft, bridge candidate, or [Spec] bridge to theorem-level status.

Proposition 3.80 (Bridge failure blocks theorem-level transport)**.** *If a necessary Bridge Program fails, then the intended theorem-level transport is blocked.*

Proof. The intended transport depends on the bridge theorem. If the Bridge Program cannot establish that theorem, the transport implication is unavailable. □

Remark 3.81 (Failure is informative)**.** Bridge failure should be recorded precisely. It identifies whether the obstruction is in hypotheses, lemmas, realization, Core payload, transport, ledger, interpretation, or promotion.

W.11. Bridge Program lifecycle

Definition 3.82 (Bridge Program lifecycle)**.** The *Bridge Program lifecycle* is:

$$\begin{aligned}\text{propose} \longrightarrow \text{specify} \longrightarrow \text{decompose} \longrightarrow \text{prove or import lemmas}\\ \longrightarrow \text{verify} \longrightarrow \text{audit} \longrightarrow \text{promote or reject.}\end{aligned}$$

Definition 3.83 (Promotion)**.** *Promotion* is the act of changing a Bridge Program or bridge candidate to a higher status, such as from [Spec] to interface theorem or from interface theorem to imported Core theorem.

Definition 3.84 (Promotion condition)**.** A *promotion condition* is the set of checks required before promotion. At minimum, it requires:

(i) precise statement;

(ii) complete hypotheses;

(iii) proof or valid import;

(iv) discharged necessary bridge lemmas;

(v) artifact references;

(vi) consistency with non-claims;

(vii) ledger treatment of any defects;

(viii) audit-readable record.

Definition 3.85 (Promotion record)**.** A *promotion record* records the prior status, new status, evidence for promotion, approving verifier or auditor, artifact references, and any remaining limitations.

Proposition 3.86 (No silent promotion)**.** *A bridge candidate or Bridge Program may not be silently promoted to theorem status.*

Proof. Promotion changes claim status. Claim status changes must be explicit, justified, and audit-readable. Silent promotion would violate the claim taxonomy and non-claim discipline. □

Remark 3.87 (Promotion is rare)**.** Most Bridge Programs remain research programs or [Spec] interfaces for long periods. This is acceptable and should be recorded honestly.

W.11bis. Imported theorem and formal-interpretation discipline

Definition 3.88 (Valid imported bridge theorem)**.** A *valid imported bridge theorem* is an external or formal theorem imported into a Bridge Program with source identity, exact statement, hypotheses, proof location or formal environment, interpretation theorem when required, permitted use, forbidden stronger reading, and CR-v19-compatible claim-use metadata.

Definition 3.89 (Formal interpretation bridge obligation)**.** A *formal interpretation bridge obligation* is the obligation to prove that a verified formal fragment states and proves the intended informal bridge statement under the declared semantics and hypotheses.

Proposition 3.90 (Imported theorem is scoped to its imported statement)**.** *A validly imported theorem may be used only at the exact strength, scope, hypotheses, and semantics recorded in its import record.*

Proof. Import does not rewrite a theorem into a stronger or differently typed statement. The admissible use is bounded by the source statement, proof environment, hypotheses, interpretation record, and permitted-use metadata. □

Proposition 3.91 (Formal proof does not eliminate interpretation burden)**.** *A verified formal fragment does not prove the intended informal bridge unless a formal interpretation bridge obligation is discharged.*

Proof. The proof assistant checks the formal statement in its formal environment. The intended informal bridge may have different semantics, hypotheses, or scope. An interpretation theorem is required to connect them. □

Remark 3.92 (Import is not inflation)**.** A valid import is a controlled theorem-use mechanism, not a way to inflate an external theorem, formal theorem, or toy theorem into an unrestricted bridge. The imported item remains scoped to its recorded statement and dependencies.

W.12. Bridge Programs and proof-first records

Definition 3.93 (Proof-first alignment)**.** A Bridge Program is *proof-first aligned* if its bridge statement, hypotheses, lemmas, failure modes, Core payload, and promotion conditions are derived from a declared proof-first record.

Definition 3.94 (Proof-first dependency)**.** A *proof-first dependency* is a dependency between a Bridge Program and a proof-first record, target theorem template, proof object schema, or theorem-level route.

Proposition 3.95 (Proof-first alignment prevents search inflation)**.** *Proof-first alignment reduces the risk that search artifacts are mistaken for theorem-level bridges.*

Proof. A proof-first record specifies the target theorem and required bridges before search outputs are evaluated. Thus search artifacts are measured against predefined proof obligations rather than being inflated into claims after discovery. □

Remark 3.96 (Relation to Appendix NS-C)**.** Appendix NS-C is a major example of proof-first alignment. Its Navier–Stokes proof-first program decomposes the PDE theorem into bridge obligations.

W.13. Bridge Program manifest entries

Definition 3.97 (Bridge Program manifest entry)**.** A *Bridge Program manifest entry* records:

(i) Bridge Program identifier;

(ii) bridge statement;

(iii) Bridge Program type;

(iv) source layer;

(v) target layer;

(vi) current status;

(vii) intended claim status;

(viii) hypotheses and their statuses;

(ix) required lemmas and their statuses;

(x) Core-facing obligations, if any;

(xi) Core bridge payload, if any;

(xii) problem-facing obligations, if any;

(xiii) defect and ledger treatment;

(xiv) dependency graph reference;

(xv) proof-first dependency, if any;

(xvi) responsible agents or roles;

(xvii) artifact references;

(xviii) failure modes, if any;

(xix) promotion history.

Proposition 3.98 (Bridge manifest supports audit)**.** *A Bridge Program manifest entry supports audit by recording the bridge statement, dependencies, hypotheses, proof status, Core-facing obligations, defects, and artifacts.*

Proof. Auditing requires reconstruction of what was claimed, what was assumed, what was proved, what remains blocked, and what artifacts support the record. The manifest entry supplies these data. □

Remark 3.99 (Manifest does not prove bridge)**.** A Bridge Program manifest records the state of the program. It does not prove the bridge unless it references a complete proof or verified proof object.

W.14. Bridge Program invariants

Definition 3.100 (Bridge Program invariant)**.** A *Bridge Program invariant* is a rule that must remain true throughout the Bridge Program lifecycle.

Definition 3.101 (Non-theorem invariant)**.** The *non-theorem invariant* states that a Bridge Program is not theorem-level until a bridge theorem is proved or validly imported.

Definition 3.102 (Hypothesis-label invariant)**.** The *hypothesis-label invariant* states that every required bridge hypothesis must have an explicit status label.

Definition 3.103 (Lemma-discharge invariant)**.** The *lemma-discharge invariant* states that theorem-level bridge status requires discharge of all necessary bridge lemmas.

Definition 3.104 (Core-payload invariant)**.** The *Core-payload invariant* states that any Core-facing bridge payload remains candidate data until checked by the declared Core verifier.

Definition 3.105 (No-silent-promotion invariant)**.** The *no-silent-promotion invariant* states that every promotion in bridge status must be explicit, justified, and audit-readable.

Proposition 3.106 (Bridge Program invariants prevent bridge inflation)**.** *The Bridge Program invariants prevent bridge drafts, bridge candidates, and Bridge Programs from being silently inflated into theorem-level bridge claims.*

Proof. The non-theorem invariant blocks program-to-theorem conflation. The hypothesis-label invariant exposes unproved assumptions. The lemma-discharge invariant blocks incomplete proof graphs. The Core-payload invariant blocks unchecked Core-facing data. The no-silent-promotion invariant blocks unsupported claim upgrades. Together, they prevent bridge inflation. □

W.15. Summary of Bridge Programs

Theorem 3.107 (Appendix W Bridge Program package)**.** *Within the Search / Platform layer, the following package is available.*

(i) *Bridge statements connect semantic layers, but are not theorems by default.*

(ii) *Bridge drafts are not bridge candidates unless specified.*

(iii) *Bridge candidates are not bridge theorems.*

(iv) *A Bridge Program organizes proof obligations for bridge candidates.*

(v) *Bridge Program status is not theorem status unless explicitly verified.*

(vi) [Spec] *hypotheses prevent theorem-level bridge status until discharged.*

(vii) *All necessary bridge lemmas must be discharged before bridge theorem status.*

(viii) *Core-facing bridge obligations may involve*

$$\mathrm{Eval}_{i,W,\tau}, \qquad C_{\tau,W}F, \qquad \phi_{i,W,\tau},$$

but listing such obligations does not verify them.

(ix) *Core bridge payloads remain candidate data until checked.*

(x) *Bridge defects affect Core audit only through the declared ledger.*

(xi) *Bridge-adjusted budgets must satisfy*

$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{bridge}}) < \mathrm{gap}_\tau(\mathfrak{B}_{\mathrm{bridge}}).$$

(xii) *Bridge Programs do not update Core verdicts.*

(xiii) *Bridge Program failures block theorem-level transport.*

(xiv) *Bridge candidates may not be silently promoted.*

(xv) *Proof-first alignment reduces search-driven claim inflation.*

(xvi) *Bridge Program manifest entries support audit but do not themselves prove bridges.*

(xvii) *Bridge Program invariants prevent bridge inflation.*

Proof. Items (i)–(iv) follow from the bridge statement, bridge draft, bridge candidate, bridge theorem, and Bridge Program definitions, together with Propositions 3.15 and 3.16. Item (v) is Proposition 3.33. Item (vi) is Proposition 3.43. Item (vii) is Proposition 3.50. Items $(viii)$–(ix) follow from the Core-facing bridge obligation definitions and Proposition 3.54. Items (x)–(xi) are Propositions 3.60 and 3.61. Item (xii) is Theorem 3.69. Item $(xiii)$ is Proposition 3.80. Item (xiv) is Proposition 3.86. Item (xv) is Proposition 3.95. Item (xvi) follows from Proposition 3.98 and Remark 3.99. Item $(xvii)$ is Proposition 3.106. □

Theorem 3.108 (Appendix W v19 bridge-hardening package)**.** *Within the Search / Platform layer, the following additional v19.0.0 hardening package is available.*

(i) *Bridge Program sovereignty prevents program records from becoming verdicts or bridge theorems by status alone.*

(ii) *Bridge Program outputs require bridge verifier-side conversion records before theorem-level use.*

(iii) *Bridge statement specificity and directionality must be declared.*

(iv) *A bridge direction is not reversible by name.*

(v) *External theorems do not become Core theorems by import name.*

(vi) *Bridge workflow status does not replace agent-output labels or claim-governance status.*

(vii) *Bridge promotion requires a promotion certificate and CR-v19 claim-use preflight.*

(viii) *Bridge primitive discrepancies must be namespaced and related to the global ledger catalog.*

(ix) *Bridge non-scalar obligations cannot be repaired by scalarized metric slack.*

(x) *Bridge-produced Type IV candidates do not establish certified zero diagnostics without actual artifacts and terminal-generic computations.*

(xi) *Imported bridge theorems are scoped to their imported statements.*

(xii) *Formal proofs require interpretation bridges before supporting informal bridge claims.*

(xiii) *Bridge defect symbols remain local until Appendix M registers their alias/refinement/disjointness relations.*

(xiv) *Bridge adjusted budgets are read through a declared metric-ledger value system and bottleneck scalarization.*

(xv) *Search-side bridge planning remains compatible with proof-first alignment without becoming proof-first evidence by itself.*

Proof. Items (i)–(ii) are Definition 3.4, Definition 3.5, Proposition 3.6, and Remark 3.7. Items (iii)–(v) are Definitions 3.18 and 3.19, Propositions 3.20 and 3.21, and Remark 3.22. Items (vi)–(vii) are Definitions 3.35 and 3.36, Propositions 3.37 and 3.38, and Remark 3.39. Items $(viii)$–(x) are Definitions 3.63, 3.64, and 3.65, Propositions 3.66 and 3.67, and Remark 3.68. Items (xi)–(xii) are Definitions 3.88 and 3.89, Propositions 3.90 and 3.91, and Remark 3.92. Items $(xiii)$–(xiv) follow from Definition 3.58, Definition 3.63, Proposition 3.61, and Remark 3.68. Item (xv) follows from Definition 3.93, Proposition 3.95, and the Bridge Program sovereignty rule. □

W.16. Explicit exclusions

The following are not asserted in Appendix W.

- No Bridge Program is treated as a bridge theorem.
- No Bridge Program sovereignty label is treated as verifier-side evidence.
- No bridge verifier-side conversion record is accepted without result status, failure route, and immutable artifact references.
- No bridge direction is reversed without a separately proved reverse bridge.

- No external theorem is treated as a Core theorem by name alone.
- No workflow status is treated as claim-governance status.
- No bridge promotion is accepted without a promotion certificate and claim-use preflight.
- No scalarized bridge budget repairs missing source theorem validity, interpretation, eligibility, tower artifact identity, terminal tameness, or claim-use permission.
- No Bridge Program Type IV candidate is treated as certified zero without an actual tower comparison artifact and terminal-generic kernel/cokernel computation.
- No formal proof assistant result is interpreted as the intended informal bridge without an interpretation theorem.
- No bridge draft is treated as a specified bridge candidate.
- No bridge candidate is treated as proved.
- No [Spec] bridge is treated as theorem-level.
- No unresolved bridge hypothesis is treated as discharged.
- No partial bridge lemma graph is treated as complete proof.
- No Bridge Program status is treated as theorem status without proof.
- No Core-facing obligation is treated as verified by being listed.
- No Core bridge payload is treated as Core verified without Core verification.
- No bridge defect is ignored when used in certification.
- No bridge-adjusted budget is bypassed.
- No Bridge Program updates Core pass/reject by itself.
- No failed Bridge Program supports theorem-level transport.
- No search-generated bridge is promoted silently.
- No proof-first record is replaced by a Bridge Program.
- No Bridge Program replaces B-Gate^{+}.
- No Bridge Program replaces the repaired topological condition
$$\mathrm{Eval}_{1,W,\tau}(F) = 0.$$
- No Bridge Program replaces admissible representative records
$$C_{\tau,W}F.$$
- No Bridge Program replaces monitored Type IV diagnostics.
- No Bridge Program replaces tower comparison artifacts
$$\phi_{i,W,\tau}.$$
- No Bridge Program replaces the budget condition
$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}).$$

W.17. Provenance and status

The material in this appendix depends on:

(i) the claim taxonomy and Minimal Manifest discipline of Chapter 1;

(ii) the Core sovereignty and typed audit discipline of Chapter 7;

(iii) the non-claim and interface-boundary discipline of Chapter 8;

(iv) the claim-governance and claim-use preflight discipline of Appendix CR-v19;

(v) the Agent Semantics of Appendix U;

(vi) the Hunter / Mapper / Lifter protocols of Appendix V;

(vii) the repaired evaluation discipline of Appendix A;

(viii) the admissible representative discipline of Appendix B;

(ix) the eligible realization regime of Appendix C;

(x) the monitored Type IV diagnostic and tower artifact discipline of Appendix D;

(xi) the manifest schema of Appendix G;

(xii) the ledger catalog of Appendix M;

(xiii) the Problem Interface Appendices of Part IV;

(xiv) the proof-first discipline of Appendix NS-C.

Its role is to define how proposed cross-layer implications are managed as auditable proof programs without being confused with proved theorems.

Status labels for Appendix W. *Search / Platform definitions:* Definitions 3.8, 3.9, 3.10, 3.11, 3.12, 3.13, 3.14, 3.23, 3.24, 3.25, 3.26, 3.27, 3.28, 3.30, 3.31, 3.32, 3.40, 3.41, 3.42, 3.45, 3.46, 3.47, 3.48, 3.49, 3.52, 3.53, 3.56, 3.57, 3.58, 3.59, 3.71, 3.72, 3.73, 3.74, 3.75, 3.76, 3.77, 3.78, 3.79, 3.82, 3.83, 3.84, 3.85, 3.93, 3.94, 3.97, 3.100, 3.101, 3.102, 3.103, 3.104, 3.105.

Search / Platform propositions/theorems: Propositions 3.15, 3.16, 3.33, 3.43, 3.50, 3.54, 3.60, 3.61, 3.80, 3.86, 3.95, 3.98, 3.106, Theorems 3.69, 3.107.

Operational cautions: Remarks 3.1, 3.2, 3.3, 3.17, 3.29, 3.34, 3.44, 3.51, 3.55, 3.62, 3.70, 3.81, 3.87, 3.96, 3.99.

Additional v19.0 hardening definitions: Definitions 3.4, 3.5, 3.18, 3.19, 3.35, 3.36, 3.63, 3.64, 3.65, 3.88, 3.89.

Additional v19.0 hardening propositions/theorems: Theorem 3.108, Propositions 3.6, 3.20, 3.21, 3.37, 3.38, 3.66, 3.67, 3.90, 3.91.

Additional v19.0 hardening remarks: Remarks 3.7, 3.22, 3.39, 3.68, 3.92.

Explicit exclusions: Section W.16.

4 Appendix X: Validity Map and Global Certificate DAG

X.1. Scope and map status

This appendix defines the Validity Map and the Global Certificate DAG used in the AK framework. It belongs to the Search / Platform Appendices. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

A Validity Map is navigation, not a theorem.

Likewise,

A Certificate DAG organizes proof dependencies; it does not prove them by existing.

In particular, no node, edge, path, map region, graph layout, dependency chain, DAG visualization, search-map structure, or certificate skeleton may replace:

$$B\text{-Gate}^{+}, \qquad \mathrm{Eval}_{1,W,\tau}(F) = 0, \qquad C_{\tau,W}F, \qquad \phi_{i,W,\tau}, \qquad (\mu_{\mathrm{Collapse}},\, u_{\mathrm{Collapse}}) = (0,0),$$
$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_{\tau}(\mathfrak{B}).$$

Remark 4.1 (Search / Platform status)**.** Validity Maps and Certificate DAGs are platform-level organization devices. They support navigation, audit, dependency tracking, proof planning, status control, and reproducibility. They are not theorem-generating mechanisms.

Remark 4.2 (Relation to Appendices U, V, and W)**.** Appendix U defines agent semantics. Appendix V defines Hunter / Mapper / Lifter search protocols. Appendix W defines Bridge Programs and bridge lemma graphs. The present appendix specifies how those objects may be assembled into global validity maps and target-specific certificate dependency structures.

Remark 4.3 (Map and DAG are governance structures)**.** A Validity Map governs status and navigation. A Certificate DAG governs dependency and discharge. Neither governs truth unless the referenced proof, verification, artifact, and audit data are complete.

X.1bis. Map and DAG sovereignty

Definition 4.4 (Map-DAG sovereignty)**.** *Map-DAG sovereignty* is the rule that Validity Maps, Certificate DAGs, graph layouts, path searches, dependency visualizations, and certificate skeletons remain governance and navigation objects until their referenced nodes and edges are independently discharged by the appropriate verifier-side protocol. They may organize proof obligations, but they do not change the mathematical status of any node, edge, bridge, certificate, or external-domain claim by placement alone.

Definition 4.5 (Verifier-side projection of a DAG)**.** For a target Certificate DAG D, the *verifier-side projection* $\pi_{\mathrm{ver}}(D)$ is the subrecord consisting only of discharged verifier-side nodes, discharged required edges, verified bridge or interpretation nodes, ledger records with certified scalarization, diagnostic records, gate verdicts, artifact identities, and manifest data consumed by the target certificate. Map layout, advisory regions, search scores, aesthetic graph positions, and proposer-side comments are omitted.

Proposition 4.6 (Map placement does not establish discharge)**.** *Placing a node or edge in a Validity Map or Certificate DAG does not discharge that node or edge.*

Proof. Placement is a platform action. Discharge requires the evidence specified by the node or edge type: proof, verification, bridge theorem, interpretation theorem, ledger record, diagnostic computation, artifact identity check, or manifest audit. A platform action supplies none of these by default. □

Proposition 4.7 (Verifier-side projection controls target use)**.** *A target certificate may consume a Certificate DAG only through its verifier-side projection. Changing map layout, display order, path highlighting, or advisory metadata without changing* $\pi_{\text{ver}}(D)$ *does not change the target verdict.*

Proof. The target verdict is determined by discharged verifier-side obligations and their audit-readable records. Layout and advisory metadata are outside those obligations. Therefore only the verifier-side projection can affect target use. □

Remark 4.8 (Graph structure is not semantic strength)**.** A short path, central node, dense cluster, or visually prominent subgraph may be useful for navigation. It is not stronger theorem evidence than a peripheral node unless the corresponding discharge records are stronger.

X.2. Validity Map

Definition 4.9 (Validity Map)**.** A *Validity Map* is a structured map of claims, candidates, bridges, certificate fragments, proof objects, artifacts, failure modes, and proof statuses in an AK workflow. It records where claims stand relative to verification, audit, bridge discharge, and theorem-level readiness.

Definition 4.10 (Validity Map node)**.** A *Validity Map node* is an entry representing one of:

(i) a Core claim;

(ii) a Core-facing object;

(iii) a repaired evaluation record;

(iv) an admissible representative record;

(v) an eligible realization record;

(vi) a tower comparison artifact;

(vii) a diagnostic result;

(viii) a ledger component;

(ix) an interface claim;

(x) a bridge draft;

(xi) a bridge candidate;

(xii) a Bridge Program;

(xiii) a bridge theorem;

(xiv) a search artifact;

(xv) a formal fragment;

(xvi) a verified formal fragment;

(xvii) a certificate candidate;

(xviii) a verified certificate;

(xix) a proof-first record;

(xx) a proof object;

(xxi) a failure mode;

(xxii) a non-claim;

(xxiii) a [Spec] statement.

Definition 4.11 (Core-facing map node)**.** A *Core-facing map node* is a Validity Map node involving verifier-side Core data, including:

$$\mathrm{Eval}_{i,W,\tau}(F), \qquad C_{\tau,W}F, \qquad \phi_{i,W,\tau}, \qquad (\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}) = (0,0),$$
$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}),$$

or the gate verdicts and manifest entries needed to support them.

Definition 4.12 (Validity Map edge)**.** A *Validity Map edge* is a typed relation between two Validity Map nodes. Permitted edge types include:

(i) depends-on;

(ii) supports;

(iii) refines;

(iv) blocks;

(v) contradicts;

(vi) specializes;

(vii) generalizes;

(viii) imports;

(ix) discharges;

(x) suggests;

(xi) cites-artifact;

(xii) verifies;

(xiii) audits;

(xiv) realizes;

(xv) lifts-to;

(xvi) normalizes-to;

(xvii) interprets-as;

(xviii) ledger-contributes-to;

(xix) bridge-transports-to;

(xx) formalizes;

(xxi) deprecates;

(xxii) supersedes.

Proposition 4.13 (Validity Map edges must be typed)**.** *Every edge in a Validity Map must have a declared edge type.*

Proof. Untyped edges are ambiguous. They may be misread as implication, dependency, evidence, analogy, artifact reference, theorem, or verified transport. Typed edges prevent claim inflation and support audit. □

Remark 4.14 (Suggests is not implies)**.** An edge labeled *suggests* is advisory. It must not be read as logical implication.

X.2bis. Edge-mode discipline and status-axis separation

Definition 4.15 (Edge discharge mode)**.** Every consumed edge in a target Certificate DAG has exactly one *edge discharge mode*:

$$\begin{gathered}\text{proved}, \quad \text{verified\_artifact}, \quad \text{imported\_theorem}, \quad \text{operational\_rule},\\ \text{specification\_target}, \quad \text{advisory\_only}, \quad \text{failed}, \quad \text{undefined}.\end{gathered}$$

Only the first four modes can discharge a necessary edge, and only at their declared scope. The mode operational_rule discharges only operational or governance-typed edges such as labeling, serialization, preflight, routing, or manifest-conformance obligations; it does not discharge theorem, bridge-theorem, diagnostic, metric, interpretation, or formal-proof edges.

Definition 4.16 (Validity status axis)**.** The validity statuses of Section X.3 are map-local status labels. They do not replace:

(i) the agent-output status labels of Appendix U;

(ii) the workflow status fields of Appendix V;

(iii) the Bridge Program status fields of Appendix W;

(iv) the Chapter 1 document-role taxonomy;

(v) the Appendix CR-v19 evidence status and invocation role.

When a node is consumed by a certificate, the stronger governance axes must also be present.

Definition 4.17 (Node consumption profile)**.** A *node consumption profile* records, for each consumed node, its map-local validity status, Appendix U agent-output label when applicable, Chapter 1 document role, CR-v19 evidence status, CR-v19 invocation role, conformance status, permitted use, forbidden stronger reading, and artifact identity.

Proposition 4.18 (Validity status does not replace claim governance)**.** *A Validity Map status label cannot by itself authorize theorem-level use of a node.*

Proof. The map-local status records navigation and workflow state. Theorem-level use is governed by the Chapter 1 taxonomy, Appendix CR-v19, source proof strength, discharged hypotheses, permitted use, and artifact identity. Those data are independent of the map-local label. □

Remark 4.19 (Typical status projections)**.** Typical projections are: *search artifact* remains proposer-side; *bridge candidate* remains non-theorem until a bridge theorem record exists; *Core certificate candidate* remains non-verdict until the Core verifier accepts it; *Core verified* must be backed by the corresponding verifier-side record and CR-v19 permitted-use field. These projections are not bidirectional definitions.

X.3. Validity statuses

Definition 4.20 (Validity status)**.** A *validity status* is a label attached to a node describing its current verification state. Permitted statuses include:

(i) search artifact;

(ii) candidate;

(iii) draft;

(iv) advisory;

(v) [Spec];

(vi) operational policy;

(vii) interface specification;

(viii) bridge draft;

(ix) bridge candidate;

(x) Bridge Program;

(xi) partially proved;

(xii) blocked;

(xiii) failed;

(xiv) rejected;

(xv) formal fragment;

(xvi) verified formal fragment;

(xvii) Core certificate candidate;

(xviii) Core-facing candidate;

(xix) Core verified;

(xx) interface theorem;

(xxi) bridge theorem;

(xxii) imported theorem;

(xxiii) proof-first record;

(xxiv) proof object candidate;

(xxv) theorem-level proof object;

(xxvi) non-claim;

(xxvii) deprecated;

(xxviii) superseded.

Definition 4.21 (Status transition)**.** A *status transition* is a recorded change from one validity status to another.

Definition 4.22 (Status transition rule)**.** A *status transition rule* specifies what evidence, verification, audit, proof, import, discharge, or demotion condition is required before a status transition is allowed.

Definition 4.23 (Promotion transition)**.** A *promotion transition* is a status transition to a stronger claim status. Examples include:

$$\text{bridge candidate} \longrightarrow \text{bridge theorem},$$

$$\text{Core certificate candidate} \longrightarrow \text{Core verified},$$

$$\text{proof object candidate} \longrightarrow \text{theorem-level proof object}.$$

Definition 4.24 (Demotion transition)**.** A *demotion transition* is a status transition to a weaker or safer claim status due to failure, artifact drift, missing dependencies, invalid proof, or supersession.

Proposition 4.25 (Status transitions require evidence)**.** *A node may not be promoted to a stronger validity status unless the required transition evidence is recorded.*

Proof. Validity status affects how a node may be used in proof planning, certification, bridge transport, and theorem-level claims. Promotion without evidence would allow unsupported claim inflation. Therefore transition evidence is required. □

Remark 4.26 (Demotion is allowed)**.** Nodes may be demoted when failures, gaps, artifact drift, interpretation errors, or dependency invalidation are discovered. Demotion protects the integrity of the map.

X.3bis. Promotion, demotion, and transition certificates

Definition 4.27 (Map-DAG promotion certificate)**.** A *Map-DAG promotion certificate* is an audit-readable record authorizing a stronger status transition for a node or edge. It contains the prior status, proposed new status, exact source claim or object, evidence supplied, discharged hypotheses, comparison-mode policy when relevant, CR-v19 claim-use preflight result, required artifacts, result status

$$\mathrm{pass}, \qquad \mathrm{reject}, \qquad \mathrm{undefined},$$

and a typed failure route.

Definition 4.28 (Transition result status)**.** A status transition has exactly one result status: pass, reject, or $\mathrm{undefined}$. Missing evidence, stale artifacts, unresolved dependencies, or incompatible permitted use produce $\mathrm{undefined}$, not pass and not a numerical obstruction.

Proposition 4.29 (Promotion requires preflight)**.** *A node or edge cannot be promoted to theorem-level, bridge-theorem-level, Core verified, or target-discharge status unless its Map-DAG promotion certificate passes claim-use preflight and artifact identity checks.*

Proof. The promoted status changes how the node or edge may be consumed. Appendix CR-v19 requires claim identity, source binding, evidence status, invocation role, hypotheses, permitted use, failure routes, and repair ancestry to be checked before consumption. Without that preflight and artifact identity, the stronger status is unsupported. □

Proposition 4.30 (Demotion is non-retroactive)**.** *Demotion of a node, edge, or subgraph creates a new recorded state and does not mutate the historical artifact or erase the prior provenance.*

Proof. The v19 repair and provenance discipline is immutable. A demotion changes the current admissible use of an item, but the previous record remains part of repair ancestry and audit history. □

Remark 4.31 (Undefined is not a failed theorem)**.** A missing discharge certificate yields $\mathrm{undefined}$ for the transition. It does not prove the negation of the target theorem; it only blocks the attempted consumption route.

X.4. Global Certificate DAG

Definition 4.32 (Certificate DAG)**.** A *Certificate DAG* is a directed acyclic graph whose nodes are certificate-relevant objects and whose edges record typed dependency relations among them.

Definition 4.33 (Global Certificate DAG)**.** The *Global Certificate DAG* is the certificate-level DAG that organizes all Core, interface, bridge, technical-extension, proof-first, formalization, ledger, artifact, and platform dependencies relevant to a target proof object or target certificate.

Definition 4.34 (DAG node)**.** A *DAG node* may represent:

(i) a definition;

(ii) a lemma;

(iii) a theorem;

(iv) a bridge statement;

(v) a Bridge Program;

(vi) a bridge theorem;

(vii) a repaired evaluation record;

(viii) an admissible representative record;

(ix) an eligibility record;

(x) a tower comparison artifact;

(xi) a diagnostic result;

(xii) a certificate fragment;

(xiii) a ledger component;

(xiv) a gate verdict;

(xv) a formal fragment;

(xvi) a proof object;

(xvii) an artifact reference;

(xviii) a failure condition;

(xix) an imported theorem;

(xx) an interpretation theorem.

Definition 4.35 (Core DAG node)**.** A *Core DAG node* is a DAG node representing verifier-side Core content, including:

$$\begin{aligned} &\mathrm{Eval}_{i,W,\tau}(F), \qquad C_{\tau,W}F, \qquad \phi_{i,W,\tau}, \qquad B\text{-Gate}^{+}, \\ &(\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}) = (0,0), \qquad \mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}). \end{aligned}$$

Definition 4.36 (DAG edge)**.** A *DAG edge* is a typed dependency from one node to another. It must declare whether it represents logical dependency, artifact dependency, verification dependency, ledger dependency, diagnostic dependency, bridge dependency, interpretation dependency, formalization dependency, advisory dependency, or audit dependency. The DAG-edge namespace is distinct from the broader Validity Map edge namespace of Definition 4.12; a map edge may record navigation or classification, while a consumed DAG edge must satisfy the target-scope discharge discipline.

Proposition 4.37 (DAG existence is not proof)**.** *The existence of a Certificate DAG does not prove the target claim.*

Proof. A DAG records dependencies and statuses. The target claim is proved only if the required nodes are proved or verified and all necessary dependency edges are discharged according to their semantics. The graph structure alone does not establish those conditions. □

Remark 4.38 (DAG is proof skeleton)**.** A Certificate DAG is best understood as a proof skeleton or audit skeleton. It becomes proof-supporting only when its necessary nodes and edges are sufficiently discharged.

X.4bis. Target scope and necessary-edge closure

Definition 4.39 (Target certificate scope)**.** A *target certificate scope* declares the target node, required sub-DAG, necessary nodes, optional nodes, necessary edges, optional edges, accepted edge discharge modes, ledger aggregation policy, diagnostic scope, bridge and interpretation dependencies, and artifact identity policy for one certificate or theorem-level proof object.

Definition 4.40 (Necessary-edge closure)**.** A target sub-DAG has *necessary-edge closure* if every necessary edge has a declared type, source, target, discharge mode, discharge certificate, artifact references when required, and result status pass.

Proposition 4.41 (Optional edges cannot discharge necessary edges)**.** *An optional advisory, explanatory, layout, or citation edge cannot discharge a necessary logical, bridge, diagnostic, ledger, interpretation, or formalization edge.*

Proof. Optional edges are outside the declared necessary dependency set for the target certificate. A necessary edge imposes a specific proof or verification obligation. Changing or adding optional structure does not satisfy that obligation. □

Remark 4.42 (Target-local closure)**.** Closure is target-local. A global map may contain many unresolved or failed nodes while one target sub-DAG is fully discharged, provided the unresolved nodes are not necessary for that target.

X.5. Acyclicity

Definition 4.43 (DAG acyclicity condition)**.** A Certificate DAG satisfies the *acyclicity condition* if it has no directed cycles among ordinary dependency edges.

Definition 4.44 (Dependency cycle)**.** A *dependency cycle* is a directed cycle in which a node depends, directly or indirectly, on itself.

Definition 4.45 (Justified cyclic proof principle)**.** A *justified cyclic proof principle* is a separately stated fixed-point, mutual-induction, coinduction, guarded recursion, or circular-proof principle that permits a controlled cyclic dependency.

Proposition 4.46 (Dependency cycles block DAG certification)**.** *A dependency cycle blocks ordinary DAG certification unless the cycle is replaced by a separately justified fixed-point, mutual-induction, coinductive, guarded, or otherwise accepted proof principle.*

Proof. Ordinary DAG-based certification requires acyclic dependency order. A cycle prevents topological ordering of proof obligations. It can be allowed only if replaced by a justified proof principle that explicitly handles cyclic dependence. □

Remark 4.47 (No hidden circular proof)**.** The DAG discipline is designed to expose circular reasoning. If a circular structure is mathematically valid, its justification must be explicit.

X.6. Edge semantics

Definition 4.48 (Logical edge)**.** A *logical edge*

$$A \to B$$

means that B logically depends on A.

Definition 4.49 (Artifact edge)**.** An *artifact edge*

$$A \to B$$

means that B references or requires artifact A.

Definition 4.50 (Ledger edge)**.** A *ledger edge*

$$A \to B$$

means that ledger component A contributes to the budget or audit condition of B.

Definition 4.51 (Diagnostic edge)**.** A *diagnostic edge*

$$A \to B$$

means that diagnostic result A is required for certificate node B.

Definition 4.52 (Bridge edge)**.** A *bridge edge*

$$A \to B$$

means that bridge theorem or bridge candidate A is used to connect the semantics of B to another layer. Its status must specify whether A is theorem-level, [Spec], candidate, failed, or imported.

Definition 4.53 (Interpretation edge)**.** An *interpretation edge*

$$A \to B$$

means that formal, Core, or platform-side statement A is interpreted as supporting informal or problem-specific claim B. Such an edge requires an interpretation theorem or explicit [Spec] status.

Definition 4.54 (Formalization edge)**.** A *formalization edge*

$$A \to B$$

means that informal statement A is represented by formal fragment or formal theorem B.

Definition 4.55 (Verification edge)**.** A *verification edge*

$$A \to B$$

means that verifier-side check or verified record A is required to establish the declared verification status of node B. It must record the verification authority, authority scope, environment when applicable, result status, and typed failure route. A verification edge is not interchangeable with an artifact, audit, advisory, or layout edge.

Definition 4.56 (Audit edge)**.** An *audit edge*

$$A \to B$$

means that audit record A supports the reconstructibility, provenance, manifest completeness, artifact identity, or replay-readiness of node B. An audit edge supports certification governance only at its declared audit scope; it does not by itself prove the mathematical theorem represented by B.

Definition 4.57 (Advisory edge)**.** An *advisory edge*

$$A \to B$$

means that A suggests, motivates, or prioritizes B, but does not logically support B.

Proposition 4.58 (Advisory edges are non-proof edges)**.** *Advisory edges do not discharge proof obligations.*

Proof. An advisory edge records motivation or navigation. It is not a logical, verification, audit, diagnostic, ledger, verified bridge, interpretation, or formal proof dependency. Therefore it cannot discharge a proof obligation. □

Remark 4.59 (Edge typing prevents misuse)**.** A single visual arrow can be misleading. The typed edge system prevents advisory, artifact, formalization, interpretation, bridge, and logical relations from being conflated.

X.6bis. Edge compatibility and misrouting control

Definition 4.60 (Edge compatibility record)**.** An *edge compatibility record* states that the source node type, target node type, edge type, discharge mode, status labels, and permitted use are mutually compatible for the intended consumption. For example, an advisory edge is compatible with prioritization, but not with theorem discharge.

Definition 4.61 (Edge misrouting)**.** *Edge misrouting* occurs when an edge is consumed as if it had a stronger or different semantic type than its declared type, for example using *suggests* as *proves*, artifact citation as theorem evidence, formalization as interpretation, or Bridge Program placement as bridge theorem discharge.

Proposition 4.62 (Misrouted edges block target discharge)**.** *If a necessary edge is misrouted, the target node is not discharged until the edge is corrected or replaced by an edge with the required type and discharge certificate.*

Proof. The target consumes the edge at a specific semantic strength. A misrouted edge lacks that strength. Therefore the required dependency remains undischarged. □

Remark 4.63 (Visual arrows are never enough)**.** The same drawn arrow can mean citation, motivation, proof, obstruction, dependency, interpretation, or transport. Only the declared edge type and discharge mode determine admissible use.

X.7. Node discharge

Definition 4.64 (Discharged node)**.** A node is *discharged* if its required verification, proof, artifact, diagnostic, ledger, bridge, interpretation, formalization, or audit condition has been satisfied.

Definition 4.65 (Undischarged node)**.** A node is *undischarged* if at least one necessary condition remains unresolved.

Definition 4.66 (Discharge certificate)**.** A *discharge certificate* is a record explaining why a node is considered discharged. It must include:

(i) node identifier;

(ii) node type;

(iii) required conditions;

(iv) evidence supplied;

(v) verifier or verifying agent, if any;

(vi) verification environment, if any;

(vii) artifact references;

(viii) status label after discharge;

(ix) limitations or residual [Spec] assumptions, if any.

Definition 4.67 (Core node discharge)**.** A *Core node discharge* is the discharge of a Core DAG node through the relevant verifier-side data, such as repaired evaluation records, admissible representatives, eligibility records, tower comparison artifacts, monitored diagnostics, gate verdicts, and ledger checks.

Proposition 4.68 (Target node requires all necessary dependencies discharged)**.** *A target node in the Global Certificate DAG may be considered discharged only if all necessary dependency nodes and dependency edges are discharged.*

Proof. The target depends on its necessary dependency structure. If any necessary dependency remains unresolved, the target proof is incomplete. Therefore all necessary dependencies must be discharged. □

Proposition 4.69 (Core node discharge requires Core verifier data)**.** *A Core DAG node cannot be discharged merely by map status, agent assertion, or DAG placement. It requires the relevant Core verifier data.*

Proof. Core DAG nodes represent Core-facing objects or verdicts. By Core sovereignty and Appendix U, such nodes require verifier-side support. Map placement, agent assertion, or dependency layout does not supply that support. □

Remark 4.70 (Optional support is different)**.** Optional advisory or explanatory dependencies need not be discharged for theorem-level status unless explicitly marked as necessary.

X.8. Global certificate assembly

Definition 4.71 (Global certificate assembly)**.** *Global certificate assembly* is the process of collecting discharged nodes, verified edges, ledger records, diagnostics, bridge theorems, interpretation theorems, and artifacts into a target certificate record or theorem-level proof object.

Definition 4.72 (Assembly policy)**.** An *assembly policy* declares:

(i) target node;

(ii) required dependency subgraph;

(iii) necessary and optional dependencies;

(iv) discharge conditions;

(v) ledger aggregation policy;

(vi) diagnostic inclusion policy;

(vii) bridge inclusion policy;

(viii) interpretation inclusion policy;

(ix) artifact completeness policy;

(x) audit requirements;

(xi) failure and demotion rules.

Definition 4.73 (Certificate assembly payload)**.** A *certificate assembly payload* is the collection of records assembled for the target certificate, including repaired evaluation records, representative records, eligibility records, tower artifacts, diagnostics, ledgers, bridge theorem references, artifact references, and manifest entries.

Proposition 4.74 (Global assembly requires manifest completeness)**.** *Global certificate assembly is admissible only if the resulting certificate record satisfies the Minimal Manifest Axiom.*

Proof. A global certificate is still a certificate record. By Appendix G, any certificate record must include the required declarations, repaired evaluation records, threshold and collapse policies, ledger policy, diagnostics, gate verdicts, and artifact references. Therefore manifest completeness is required. □

Remark 4.75 (Assembly is not invention)**.** Assembly collects and organizes already verified, proved, declared, or explicitly status-labeled components. It does not invent missing proof obligations.

X.8bis. Assembly budget and non-compensation

Definition 4.76 (Assembly ledger record)**.** An *assembly ledger record* declares the gate-scoped metric-ledger value system used by a target assembly, the local alias $\mathfrak{d}_{\mathrm{asm}}$ if any, every included metric component, the scalarization val_B, evidence that the scalarized value upper-bounds the actual bottleneck discrepancies consumed by the target, and the strict comparison

$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{asm}}) < \mathrm{gap}_\tau(\mathfrak{B}_{\mathrm{asm}}).$$

The alias $\mathfrak{d}_{\mathrm{asm}}$ is not global unless Appendix M registers its relation to the gate-scoped $\mathfrak{d}_{\mathrm{met}}$.

Definition 4.77 (Map-DAG assembly defect)**.** A *Map-DAG assembly defect* is a declared discrepancy introduced by assembling nodes, edges, artifacts, bridge records, interpretation records, or ledger components into a target certificate. Typical local symbols include

$$\delta^{\text{X-edge}}, \quad \delta^{\text{X-asm}}, \quad \delta^{\text{X-art}}, \quad \delta^{\text{X-interp}}, \quad \Delta_{\phi,\mathrm{DAG}}.$$

They are local Search/Platform-layer names until the global ledger catalog assigns alias, refinement, aggregate, or disjointness relations.

Proposition 4.78 (Assembly budget cannot repair non-scalar obligations)**.** *No scalarized assembly budget can repair a missing proof, missing interpretation theorem, missing bridge theorem, missing Type IV artifact, missing terminal-tameness evidence, failed claim-use preflight, or artifact identity failure.*

Proof. These obligations are non-scalar. They concern proof, semantics, artifact identity, status, or typed morphism existence rather than bottleneck discrepancy. Ledger slack can bound metric discrepancies only when the required discrepancy theorem and scalarization record exist. □

Remark 4.79 (Map-DAG ledger catalog registration)**.** The symbols $\delta^{\text{X-edge}}$, $\delta^{\text{X-asm}}$, $\delta^{\text{X-art}}$, $\delta^{\text{X-interp}}$, and $\Delta_{\phi,\mathrm{DAG}}$ must be compared with the Search-layer and Bridge-layer symbols of Appendices V and W before global aggregation. Undeclared overlap is inadmissible because it can double-count or hide a primitive discrepancy.

X.9. Validity Map versus Certificate DAG

Definition 4.80 (Map-DAG distinction)**.** The *Validity Map* is a broad navigation and status structure. The *Global Certificate DAG* is a proof-dependency structure for a specific target certificate or proof object.

Proposition 4.81 (Validity Map is broader than Certificate DAG)**.** *A Validity Map may contain nodes and edges that are not part of the Global Certificate DAG for a given target.*

Proof. A Validity Map includes candidates, rejected routes, advisory relations, search artifacts, failed bridge programs, deprecated nodes, and [Spec] statements. A Certificate DAG includes the dependencies relevant to a specific certificate target. Thus the map is broader. □

Definition 4.82 (Target sub-DAG)**.** A *target sub-DAG* is the dependency subgraph of the Global Certificate DAG required to support a specific target node.

Remark 4.83 (Do not over-certify the map)**.** Only the target certificate sub-DAG requires discharge for a given certificate. The entire Validity Map need not be fully discharged.

X.10. Failure and demotion

Definition 4.84 (Map failure)**.** A *Map failure* occurs when a Validity Map node, edge, status label, artifact reference, or transition record is incorrect, ambiguous, stale, unsupported, or inconsistent.

Definition 4.85 (DAG failure)**.** A *DAG failure* occurs when a Certificate DAG contains an unresolved necessary dependency, invalid edge, undischarged required node, forbidden cycle, inconsistent status, missing artifact, failed bridge, invalid interpretation edge, or unsupported Core node.

Definition 4.86 (Demotion event)**.** A *demotion event* is a recorded downgrade of a node, edge, subgraph, map region, or certificate status due to discovered failure, artifact drift, unresolved dependency, invalid interpretation, or bridge failure.

Definition 4.87 (Subgraph quarantine)**.** *Subgraph quarantine* is the temporary isolation of a map or DAG subgraph whose status is uncertain, inconsistent, stale, or under review.

Proposition 4.88 (DAG failure blocks target certification)**.** *A DAG failure in a necessary dependency subgraph blocks target certification until repaired.*

Proof. A necessary dependency subgraph is required for the target certificate. If it contains failure, the target certificate is unsupported. Repair, replacement, demotion, or removal of the failed dependency is required. □

Remark 4.89 (Demotion protects integrity)**.** Demotion is not a failure of the platform. It is a safety mechanism that prevents stale, invalid, or over-promoted claims from remaining active at too strong a status.

X.11. Artifact consistency

Definition 4.90 (Artifact consistency)**.** A node or edge has *artifact consistency* if every referenced artifact exists, has the declared identity, has the declared version, and supports the claim or relation for which it is cited.

Definition 4.91 (Artifact drift)**.** *Artifact drift* occurs when a referenced artifact changes, disappears, is superseded, becomes inconsistent, or no longer supports the node or edge that cites it.

Definition 4.92 (Artifact identity record)**.** An *artifact identity record* is a record of an artifact's identifier, version, hash or equivalent integrity marker, source, status, and supported nodes or edges.

Definition 4.93 (Artifact support relation)**.** An *artifact support relation* declares which claim, node, edge, diagnostic, ledger, proof, or certificate component an artifact supports.

Proposition 4.94 (Artifact drift requires review)**.** *If artifact drift is detected in a necessary dependency, the affected node or edge must be reviewed and may need demotion.*

Proof. Artifact drift may invalidate the evidence supporting a node or edge. Until reviewed, the dependency is not audit-stable. Thus review and possible demotion are required. □

Remark 4.95 (Hashing and versioning)**.** Hashing, versioning, immutable identifiers, and proof-store references are platform mechanisms for controlling artifact drift. They are developed further in the execution and reproducibility appendices.

X.11bis. Artifact identity and replay limits

Definition 4.96 (Artifact identity preflight)**.** An *artifact identity preflight* checks that every necessary artifact has the declared identifier, version, hash or integrity marker, source, status, authority scope, and support relation before a node or edge is consumed.

Definition 4.97 (Replay-success node)**.** A *replay-success node* records that a declared workflow replayed under a declared environment. It establishes reproducibility of the replayed workflow at that scope, not mathematical truth beyond the replayed criteria.

Proposition 4.98 (Replay success is not proof by itself)**.** *A replay-success node does not prove a theorem unless the replayed workflow includes the required proof or verifier-side checks for that theorem.*

Proof. Replay verifies reproducibility of an execution or reconstruction. Mathematical proof requires the relevant theorem evidence, bridge evidence, diagnostic evidence, ledger comparison, and artifact checks. A replay may include such checks, but replay status alone is not identical to theorem proof. □

Proposition 4.99 (Artifact identity failure blocks necessary dependencies)**.** *If a necessary node or edge fails artifact identity preflight, the target dependency remains undischarged.*

Proof. The dependency is supported by a particular artifact identity. If that identity is missing, stale, mismatched, or unsupported, the claimed support cannot be reconstructed. Thus the dependency cannot be consumed. □

Remark 4.100 (Storage is not support)**.** A platform may store an artifact without the artifact supporting the claim for which it is cited. The support relation must be declared and checked separately.

X.12. Bridge Programs inside the DAG

Definition 4.101 (Bridge Program DAG node)**.** A *Bridge Program DAG node* is a DAG node representing a Bridge Program, bridge candidate, bridge theorem, bridge lemma graph, or bridge discharge record.

Definition 4.102 (Bridge theorem discharge edge)**.** A *bridge theorem discharge edge* is an edge showing that a bridge theorem discharges a bridge dependency required by a target node.

Definition 4.103 (Bridge Program blocked edge)**.** A *Bridge Program blocked edge* is an edge showing that an unresolved, failed, or [Spec] Bridge Program blocks theorem-level transport.

Proposition 4.104 (Bridge Program node does not discharge bridge dependency by existence)**.** *A Bridge Program DAG node does not discharge a bridge dependency unless it is supported by a bridge theorem, valid import, or verified discharge record.*

Proof. By Appendix W, a Bridge Program is not a bridge theorem. Therefore its presence in the DAG records a proof program or dependency, but does not discharge the dependency by itself. □

Remark 4.105 (Bridge Program placement)**.** Bridge Programs are useful DAG nodes because they expose what must be proved. Their status must remain distinct from bridge theorem nodes.

X.12bis. Bridge and Type IV nodes inside the DAG

Definition 4.106 (DAG Type IV artifact node)**.** A *DAG Type IV artifact node* records an actual comparison artifact

$$\phi_{i,W,\tau} : \mathrm{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \mathrm{ApexProfile}_{i,W,\tau}(F_\infty)$$

together with source and target identities, monitored scope, terminal-tameness evidence, and diagnostic computation.

Definition 4.107 (DAG Type IV zero node)**.** A *DAG Type IV zero node* is a discharged diagnostic node asserting certified zero Type IV status for the declared monitored scope. It may be used only when the actual artifact node exists, terminal-tameness is verified, and

$$\mu_{\mathrm{Collapse}} = \sum_{i \in I_{\mathrm{mon}}} \mathrm{tgdim}_W \ker(\phi_{i,W,\tau}) = 0, \qquad u_{\mathrm{Collapse}} = \sum_{i \in I_{\mathrm{mon}}} \mathrm{tgdim}_W \operatorname{coker}(\phi_{i,W,\tau}) = 0.$$

The node must point to the full typed Type IV diagnostic record of Chapter 5, including the interior-defect field and its declared status, such as $\mathrm{not\_checked}$, $\mathrm{certified\_absent}$, or $\mathrm{certified\_present}$, whenever that field is part of the invoked diagnostic schema. A zero node is therefore a pointer to a complete diagnostic record, not a reduced replacement schema.

Proposition 4.108 (Bridge Program path is not bridge theorem path)**.** *A path through Bridge Program nodes does not discharge a bridge dependency unless the path contains a discharged bridge theorem, valid import, or verified bridge-discharge record at the required semantic direction.*

Proof. Appendix W separates Bridge Programs from bridge theorems. A path through programs records planning and dependencies. It does not establish the theorem-level implication without a discharged bridge theorem or equivalent verified record. □

Proposition 4.109 (Type IV placement does not imply zero)**.** *Placing a Type IV artifact candidate, proxy, or diagnostic placeholder in a map or DAG does not imply certified zero Type IV status.*

Proof. Certified zero Type IV status requires an actual comparison artifact, terminal-tameness, monitored scope, and terminal-generic kernel and cokernel computations. A placeholder or candidate node lacks those data by default. □

Remark 4.110 (Undefined Type IV is not zero in the DAG)**.** A missing or ill-typed Type IV artifact node has status $\mathrm{undefined}$ or $\mathrm{not\_invoked}$; the DAG must not encode it as a zero diagnostic node.

X.13. Formalization and interpretation inside the DAG

Definition 4.111 (Formalization DAG node)**.** A *formalization DAG node* is a node representing a formal statement, formal fragment, proof assistant file, verified formal theorem, or formal environment.

Definition 4.112 (Interpretation theorem node)**.** An *interpretation theorem node* is a node representing a theorem that connects a verified formal statement to the intended informal or problem-specific claim.

Proposition 4.113 (Verified formal node does not imply intended claim without interpretation)**.** *A verified formal node does not discharge an intended informal or problem-specific claim unless the required interpretation edge is discharged.*

Proof. A proof assistant verifies the formal statement in its formal environment. The intended informal claim may have different semantics, hypotheses, or scope. An interpretation theorem is required to connect them. □

Remark 4.114 (Formalization mismatch tracking)**.** Formalization mismatch should appear in the Validity Map and Certificate DAG as either an undischarged interpretation edge, a blocked bridge, or a demotion event.

X.14. Validity Map manifest entries

Definition 4.115 (Validity Map manifest entry)**.** A *Validity Map manifest entry* records:

(i) map identifier;

(ii) node set;

(iii) edge set;

(iv) node status labels;

(v) edge types;

(vi) Core-facing nodes;

(vii) Bridge Program nodes;

(viii) formalization and interpretation nodes;

(ix) artifact references;

(x) status transition history;

(xi) demotion events;

(xii) quarantined subgraphs, if any;

(xiii) audit notes.

Definition 4.116 (Certificate DAG manifest entry)**.** A *Certificate DAG manifest entry* records:

(i) DAG identifier;

(ii) target node;

(iii) required dependency subgraph;

(iv) node discharge statuses;

(v) edge discharge statuses;

(vi) Core node discharge records;

(vii) ledger dependencies;

(viii) diagnostic dependencies;

(ix) bridge dependencies;

(x) interpretation dependencies;

(xi) artifact references;

(xii) assembly policy;

(xiii) audit result.

Proposition 4.117 (Map and DAG manifests support audit)**.** *Validity Map and Certificate DAG manifest entries support audit by recording structure, status, dependencies, artifacts, discharge conditions, and demotion history.*

Proof. Audit requires reconstructing what was used, how it was connected, which dependencies were necessary, and whether those dependencies were discharged. The map and DAG manifests record these data. □

Remark 4.118 (Manifest is not proof)**.** A map or DAG manifest records structure and status. It does not prove the target unless the referenced proof, verification, interpretation, and artifact data are complete.

X.15. Map and DAG invariants

Definition 4.119 (Map-DAG invariant)**.** A *Map-DAG invariant* is a rule that must remain true across Validity Map and Certificate DAG updates.

Definition 4.120 (Typed-edge invariant)**.** The *typed-edge invariant* states that every map or DAG edge must have declared semantics.

Definition 4.121 (Evidence-promotion invariant)**.** The *evidence-promotion invariant* states that no node may be promoted to a stronger status without recorded evidence.

Definition 4.122 (Core-sovereignty invariant)**.** The *Core-sovereignty invariant* states that no map or DAG structure may replace Core verifier conditions.

Definition 4.123 (Bridge-status invariant)**.** The *bridge-status invariant* states that bridge drafts, bridge candidates, Bridge Programs, and bridge theorem nodes must remain status-distinct.

Definition 4.124 (Artifact-consistency invariant)**.** The *artifact-consistency invariant* states that necessary nodes and edges must maintain artifact consistency or be reviewed and possibly demoted.

Proposition 4.125 (Map-DAG invariants prevent proof inflation)**.** *The typed-edge, evidence-promotion, Core-sovereignty, bridge-status, and artifact-consistency invariants prevent navigation structures from being inflated into proof.*

Proof. Typed edges prevent ambiguous arrows. Evidence-promotion blocks unsupported status upgrades. Core sovereignty blocks map/DAG replacement of verifier-side conditions. Bridge-status separation blocks Bridge Program inflation. Artifact consistency blocks stale dependency support. Together, these invariants preserve proof integrity. □

X.15bis. v19.0 Map-DAG hardening package

Theorem 4.126 (Appendix X v19 map-DAG hardening package)**.** *Within the Search / Platform layer, the following additional v19.0 hardening package is available.*

(i) Map-DAG sovereignty prevents map placement or graph layout from determining proof status.

(ii) Target use is controlled by the verifier-side DAG projection.

(iii) Edge discharge modes separate proved, verified, imported, operational, specification, advisory, failed, and undefined uses.

(iv) *Validity statuses are map-local and do not replace Appendix U labels, Chapter 1 claim roles, or Appendix CR-v19 statuses.*

(v) *Node consumption profiles record the governance axes needed for theorem-level use.*

(vi) *Promotion requires a Map-DAG promotion certificate and claim-use preflight.*

(vii) *Demotion is immutable and non-retroactive.*

(viii) *Target certificate scope and necessary-edge closure are target-local.*

(ix) *Optional or advisory edges cannot discharge necessary proof edges.*

(x) *Edge misrouting blocks target discharge.*

(xi) *Assembly ledger records require certified scalarization and strict threshold-gap comparison.*

(xii) *Assembly budgets cannot repair non-scalar obligations.*

(xiii) *Artifact identity preflight and replay limits prevent storage or replay from being mistaken for proof.*

(xiv) *Bridge Program paths and Type IV placeholders do not discharge bridge or Type IV zero obligations.*

(xv) *Undefined or not-invoked diagnostics are never encoded as certified zero nodes.*

Proof. Items (i)–(ii) follow from Definitions 4.4, 4.5, and Propositions 4.6, 4.7. Items (iii)–(v) follow from Definitions 4.15, 4.16, 4.17, and Proposition 4.18. Items (vi)–(vii) are Definitions 4.27, 4.28 and Propositions 4.29, 4.30. Items $(viii)$–(ix) follow from Definitions 4.39, 4.40, Proposition 4.41, and Remark 4.42. Item (x) is Proposition 4.62. Items (xi)–(xii) follow from Definition 4.76 and Proposition 4.78. Item $(xiii)$ follows from Definitions 4.96, 4.97, Propositions 4.98, 4.99, and Remark 4.100. Items (xiv)–(xv) follow from Definitions 4.106, 4.107, Propositions 4.108, 4.109, and Remark 4.110. □

X.16. Summary of Validity Map and Global Certificate DAG

Theorem 4.127 (Appendix X map-DAG package)**.** *Within the Search / Platform layer, the following package is available.*

(i) *A Validity Map is navigation, not theorem.*

(ii) *A Certificate DAG organizes proof dependencies, but does not prove them merely by existing.*

(iii) *Validity Map nodes and edges must be typed and status-labeled.*

(iv) *Core-facing nodes may represent*

$$\mathrm{Eval}_{i,W,\tau}, \qquad C_{\tau,W}F, \qquad \phi_{i,W,\tau}, \qquad (\mu_{\mathrm{Collapse}},\, u_{\mathrm{Collapse}}), \qquad \mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}),$$

but map placement does not verify them.

(v) *Status promotion requires recorded evidence.*

(vi) *Dependency cycles block ordinary DAG certification unless explicitly justified.*

(vii) *Advisory edges do not discharge proof obligations.*

(viii) *Core node discharge requires Core verifier data.*

(ix) *A target node requires all necessary dependency nodes and edges to be discharged.*

(x) *Global certificate assembly requires Minimal Manifest completeness.*

(xi) *The Validity Map is broader than a target Certificate DAG.*

(xii) *DAG failure in a necessary subgraph blocks target certification.*

(xiii) *Artifact drift requires review and possible demotion.*

(xiv) *Bridge Program nodes do not discharge bridge dependencies by existence.*

(xv) *Verified formal nodes do not imply intended informal claims without discharged interpretation edges.*

(xvi) *Map and DAG manifests support audit but do not themselves prove the target.*

(xvii) *Map-DAG invariants prevent proof inflation.*

Proof. Items (i)–(ii) are Proposition 4.37 and the definition of the Validity Map. Item (iii) follows from the node, edge, and status definitions and Proposition 4.13. Item (iv) follows from Definitions 4.11 and 4.35, and Proposition 4.69. Item (v) is Proposition 4.25. Item (vi) is Proposition 4.46. Item (vii) is Proposition 4.58. Item $(viii)$ is Proposition 4.69. Item (ix) is Proposition 4.68. Item (x) is Proposition 4.74. Item (xi) is Proposition 4.81. Item (xii) is Proposition 4.88. Item $(xiii)$ is Proposition 4.94. Item (xiv) is Proposition 4.104. Item (xv) is Proposition 4.113. Item (xvi) follows from Proposition 4.117 and Remark 4.118. Item $(xvii)$ is Proposition 4.125. □

X.17. Explicit exclusions

The following are not asserted in Appendix X.

- No Validity Map is treated as a theorem.
- No Certificate DAG is treated as proof merely by existing.
- No untyped edge is treated as logical implication.
- No advisory edge discharges a proof obligation.
- No node is promoted without recorded evidence.
- No dependency cycle is accepted without explicit justification.
- No undischarged target dependency is ignored.
- No artifact reference is assumed stable without version or consistency control.
- No map visualization is treated as proof.
- No DAG layout is treated as verification.
- No certificate assembly bypasses the Minimal Manifest Axiom.
- No failed necessary subgraph supports target certification.

- No Bridge Program node is treated as a bridge theorem.
- No bridge candidate node is treated as a discharged bridge dependency.
- No verified formal node is interpreted as an informal theorem without an interpretation theorem.
- No Core-facing node is treated as Core verified merely by appearing in the map or DAG.
- No map or DAG node replaces B-Gate^{+}.
- No map or DAG node replaces the repaired topological condition

$$\mathrm{Eval}_{1,W,\tau}(F) = 0.$$

- No map or DAG node replaces admissible representative records

$$C_{\tau,W}F.$$

- No map or DAG node replaces monitored Type IV diagnostics.
- No map or DAG node replaces tower comparison artifacts

$$\phi_{i,W,\tau}.$$

- No Validity Map or Certificate DAG replaces the budget condition

$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}).$$

- No map layout, path length, or graph centrality is treated as theorem evidence.
- No map-local validity status replaces Appendix CR-v19 claim-use preflight.
- No optional or advisory edge discharges a necessary proof, bridge, diagnostic, ledger, or interpretation edge.
- No replay-success node is treated as proof unless the replayed workflow includes the required verifier-side proof checks.
- No artifact storage event is treated as artifact support.
- No assembly budget repairs missing proof, missing interpretation, missing bridge theorem, missing Type IV artifact, or failed artifact identity.
- No Bridge Program path is treated as a bridge theorem path without a discharged bridge theorem or valid import.
- No Type IV placeholder, proxy, or candidate node is treated as a certified-zero Type IV diagnostic.

X.18. Provenance and status

The material in this appendix depends on:

(i) the claim taxonomy and manifest discipline of Chapter 1;

(ii) the claim-use governance of Appendix CR-v19;

(iii) the Agent Semantics of Appendix U;

(iv) the Hunter / Mapper / Lifter protocols of Appendix V;

(v) the Bridge Programs of Appendix W;

(vi) the repaired evaluation discipline of Appendix A;

(vii) the admissible representative discipline of Appendix B;

(viii) the eligible realization regime of Appendix C;

(ix) the monitored Type IV diagnostic and tower artifact discipline of Appendix D;

(x) the Core sovereignty doctrine of Chapter 7;

(xi) the non-claim and interface-boundary discipline of Chapter 8;

(xii) the manifest schema of Appendix G;

(xiii) the ledger catalog of Appendix M;

(xiv) the proof-first discipline of Appendix NS-C.

Its role is to provide the global map and DAG structure required for proof planning, dependency tracking, bridge governance, certificate assembly, and audit without confusing navigation with proof.

Status labels for Appendix X. *Search / Platform definitions:* Definitions 4.9, 4.10, 4.11, 4.12, 4.20, 4.21, 4.22, 4.23, 4.24, 4.32, 4.33, 4.34, 4.35, 4.36, 4.43, 4.44, 4.45, 4.48, 4.49, 4.50, 4.51, 4.52, 4.53, 4.54, 4.55, 4.56, 4.57, 4.64, 4.65, 4.66, 4.67, 4.71, 4.72, 4.73, 4.80, 4.82, 4.84, 4.85, 4.86, 4.87, 4.90, 4.91, 4.92, 4.93, 4.101, 4.102, 4.103, 4.111, 4.112, 4.115, 4.116, 4.119, 4.120, 4.121, 4.122, 4.123, 4.124.

Search / Platform propositions/theorems: Propositions 4.13, 4.25, 4.37, 4.46, 4.58, 4.68, 4.69, 4.74, 4.81, 4.88, 4.94, 4.104, 4.113, 4.117, 4.125, Theorem 4.127.

Operational cautions: Remarks 4.1, 4.2, 4.3, 4.14, 4.26, 4.38, 4.47, 4.59, 4.70, 4.75, 4.83, 4.89, 4.95, 4.105, 4.114, 4.118.

Additional v19.0 hardening definitions: Definitions 4.4, 4.5, 4.15, 4.16, 4.17, 4.27, 4.28, 4.39, 4.40, 4.60, 4.61, 4.76, 4.77, 4.96, 4.97, 4.106, 4.107.

Additional v19.0 hardening propositions/theorems: Propositions 4.6, 4.7, 4.18, 4.29, 4.30, 4.41, 4.62, 4.78, 4.98, 4.99, 4.108, 4.109, Theorem 4.126.

Additional v19.0 hardening remarks: Remarks 4.8, 4.19, 4.31, 4.42, 4.63, 4.79, 4.100, 4.110.

Explicit exclusions: Section X.17.

5 Appendix Y: AI Platform and Proof Store

Y.1. Scope and platform status

This appendix defines the AI Platform and Proof Store layer of the AK framework. It belongs to the Search / Platform Appendices. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

Storage is not certification.

A Proof Store may store, index, version, retrieve, compare, and audit proof-related objects. However, the fact that an object is stored does not imply that it is true, verified, Core-admissible, or theorem-level.

In particular, no platform entry, stored proof object, AI-generated artifact, claim registry item, version tag, proof-store identifier, hash, query result, or storage event may replace:

$$B\text{-Gate}^{+}, \qquad \mathrm{Eval}_{1,W,\tau}(F) = 0, \qquad C_{\tau,W}F, \qquad \phi_{i,W,\tau}, \qquad (\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}) = (0,0), \qquad \mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}).$$

Remark 5.1 (Search / Platform status)**.** This appendix concerns infrastructure for managing proof objects, artifacts, claims, agent outputs, verification records, audit records, and replay references. It does not create mathematical validity.

Remark 5.2 (Relation to Appendices U–X)**.** Appendix U defines agent semantics. Appendix V defines search protocols. Appendix W defines Bridge Programs. Appendix X defines Validity Maps and Certificate DAGs. The present appendix defines the storage, registry, lifecycle, and platform layer supporting those structures.

Remark 5.3 (Storage supports governance)**.** The platform layer is not a proof engine by default. It preserves identities, histories, statuses, and dependencies so that proof engines, auditors, and human reviewers can reconstruct what has been claimed and checked.

Y.1bis. Platform sovereignty and verifier-side conversion

Definition 5.4 (Platform sovereignty)**.** *Platform sovereignty* is the rule that platform storage, indexing, retrieval, versioning, hashing, synchronization, dashboard display, access control, and lifecycle management do not alter the mathematical status of an object. A platform record affects theorem use only through a declared verifier-side record, governance record, or audit record at its stated scope.

Definition 5.5 (Platform-to-verifier conversion record)**.** A *platform-to-verifier conversion record* is a record that attempts to use a platform object as verifier-side evidence. It must contain:

(i) the source platform entry identifier and immutable content identity;

(ii) the target verifier-side record type;

(iii) the scope, window, threshold, monitored degree, and comparison mode, when relevant;

(iv) the conversion theorem, verified computation, or proof assistant artifact supporting the conversion;

(v) the authority scope and environment of the conversion check;

(vi) all non-scalar obligations consumed by the target record;

(vii) the resulting status, exactly one of *pass*, *reject*, or *undefined*;

(viii) the typed failure route when the result is reject or undefined;

(ix) artifact references and repair ancestry.

It is the platform-specialized instance of the verifier-side conversion discipline of Appendix U.

Proposition 5.6 (Platform record does not override verifier-side status)**.** *No platform record, by being stored, indexed, displayed, versioned, hashed, queried, synchronized, or access-approved, overrides a verifier-side status.*

Proof. The listed operations are platform actions. They affect availability, identity, visibility, routing, or governance metadata. They do not by themselves establish repaired evaluation, representative compatibility, eligibility, tower artifact validity, diagnostic vanishing, metric budget admissibility, or bridge soundness. Therefore they cannot override verifier-side status. □

Proposition 5.7 (Verifier-side use requires conversion)**.** *A platform object may be consumed as verifier-side evidence only through a platform-to-verifier conversion record with result pass.*

Proof. A platform object is initially storage-side or workflow-side data. Verifier-side consumption requires typed reconstruction of the target record and its obligations. Definition 5.5 supplies exactly that boundary. Without it, the object remains non-verdict platform data. □

Remark 5.8 (Platform noninterference)**.** Two certificate records with the same verifier-side projection have the same Core status even if their storage locations, query rankings, dashboard views, access histories, or Proof Store identifiers differ.

Y.2. AI Platform

Definition 5.9 (AI Platform)**.** An *AI Platform* is a software, data, and workflow environment supporting agent-assisted search, proof management, artifact storage, verification, audit, replay, and reproducibility for the AK framework.

Definition 5.10 (Platform component)**.** A *platform component* is a subsystem of the AI Platform. Examples include:

(i) agent interface;

(ii) search engine;

(iii) proof assistant interface;

(iv) persistence-computation engine;

(v) artifact store;

(vi) proof store;

(vii) claim registry;

(viii) validity map service;

(ix) certificate DAG service;

(x) ledger engine;

(xi) diagnostic engine;

(xii) replay engine;

(xiii) audit dashboard;

(xiv) access-control service.

Definition 5.11 (Platform action)**.** A *platform action* is an operation performed by the AI Platform or one of its components. Examples include:

create, store, retrieve, version, hash, verify, audit, replay, promote, demote, reject, supersede.

Definition 5.12 (Platform authority scope)**.** A *platform authority scope* is the declared scope within which a platform component or action is authorized to operate.

Proposition 5.13 (Platform action is scoped)**.** *A platform action establishes only the effect declared by its action type and authority scope.*

Proof. Different platform actions have different meanings. Storing an object records it. Hashing identifies content. Verification checks declared criteria. Audit checks reconstructibility. Promotion changes status only under an authority rule. Therefore each action is scoped to its declared semantics. □

Remark 5.14 (Platform semantics must be explicit)**.** The same interface button or workflow step may hide multiple platform actions. For auditability, the semantic action must be recorded explicitly.

Y.3. Proof Store

Definition 5.15 (Proof Store)**.** A *Proof Store* is a repository for proof objects, certificate records, formal fragments, Bridge Programs, Validity Map entries, Certificate DAG entries, artifact references, ledger records, diagnostic records, and associated metadata.

Definition 5.16 (Proof Store entry)**.** A *Proof Store entry* is a stored item in the Proof Store. It must include:

(i) entry identifier;

(ii) object type;

(iii) content or content reference;

(iv) status label;

(v) version identifier;

(vi) provenance metadata;

(vii) dependency references, if any;

(viii) artifact references;

(ix) audit metadata;

(x) authority and access metadata.

Definition 5.17 (Stored object)**.** A *stored object* is any object contained in or referenced by the Proof Store.

Definition 5.18 (Stored Core-facing object)**.** A *stored Core-facing object* is a stored object representing or referencing Core-relevant verifier-side material, such as:

$$\mathrm{Eval}_{i,W,\tau}(F), \qquad C_{\tau,W}F, \qquad \phi_{i,W,\tau}, \qquad B\text{-Gate}^{+}, \qquad (\mu_{\mathrm{Collapse}},\, u_{\mathrm{Collapse}}), \qquad \mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}).$$

Definition 5.19 (Verified Core-facing object)**.** A *verified Core-facing object* is a stored Core-facing object whose relevant Core verifier-side conditions have been checked and whose verification record is stored and audit-readable.

Proposition 5.20 (Stored object is not certified by storage)**.** *A stored object is not certified merely by being stored in the Proof Store.*

Proof. Storage records availability and identity. Certification requires verifier-side conditions, proof data, diagnostics, ledgers, gate verdicts, and audit-readable manifests. Therefore storage alone does not certify an object. □

Proposition 5.21 (Stored Core-facing object is not verified by storage)**.** *A stored Core-facing object is not Core verified merely by being stored.*

Proof. A stored Core-facing object may represent a repaired evaluation record, representative record, tower artifact, diagnostic, or ledger condition. Core verification requires checking the corresponding verifier-side criteria. Storage records the object but does not perform or imply that check. □

Remark 5.22 (Storage supports, but does not replace, verification)**.** The Proof Store is essential for reproducibility and audit. Its function is to preserve and organize evidence, not to create validity automatically.

Y.4. Object classes

Definition 5.23 (Proof Store object class)**.** A *Proof Store object class* is a declared type of object stored in the Proof Store. Standard classes include:

(i) search artifact;

(ii) candidate;

(iii) bridge draft;

(iv) bridge candidate;

(v) Bridge Program;

(vi) bridge theorem;

(vii) formal fragment;

(viii) verified formal fragment;

(ix) certificate candidate;

(x) Core certificate;

(xi) Core-facing object;

(xii) verified Core-facing object;

(xiii) interface theorem;

(xiv) proof-first record;

(xv) proof object;

(xvi) theorem-level proof object;

(xvii) ledger record;

(xviii) diagnostic record;

(xix) artifact reference;

(xx) interpretation theorem;

(xxi) non-claim;

(xxii) rejected object;

(xxiii) deprecated object;

(xxiv) superseded object.

Definition 5.24 (Object status)**.** An *object status* is a validity or lifecycle label attached to a Proof Store object. It must be compatible with the status taxonomy of Appendices U, V, W, and X.

Definition 5.25 (Status authority)**.** A *status authority* is the declared agent, tool, procedure, or governance rule authorized to assign or update a particular object status.

Proposition 5.26 (Object class does not imply validity)**.** *The object class of a stored item does not by itself imply validity.*

Proof. An object class describes what kind of object is stored. Validity depends on proof, verification, status, discharge, and audit records. Therefore class and validity are distinct. □

Remark 5.27 (Class/status separation)**.** A formal fragment may be unverified or verified. A Core-facing object may be stored but unchecked. A certificate candidate may pass or fail. A bridge theorem may later be superseded. Thus object class and object status must be tracked separately.

Y.4bis. Status axes and token compatibility

Definition 5.28 (Proof Store status axis)**.** The Proof Store status axis is a platform-local lifecycle and validity field. It does not replace:

(i) the agent-output status labels of Appendix U;

(ii) the search workflow statuses of Appendix V;

(iii) the Bridge Program statuses of Appendix W;

(iv) the map and DAG statuses of Appendix X;

(v) the Chapter 1 claim taxonomy;

(vi) the CR-v19 evidence status, invocation role, conformance status, or claim-use preflight result.

Definition 5.29 (Status compatibility record)**.** A *status compatibility record* declares how a Proof Store object class and platform status relate to the relevant Appendix U label, Chapter 1 document role, and CR-v19 governance status. It must state whether the platform status is lifecycle-only, theorem-facing, audit-facing, or non-verdict.

Proposition 5.30 (Platform status is not claim-governance status)**.** *A platform status label does not by itself determine a claim's theorem evidence status, invocation role, or permitted use.*

Proof. Platform status records workflow and storage state. Claim-governance status is controlled by Chapter 1 and CR-v19 and depends on proof evidence, hypotheses, dependencies, permitted use, and failure routing. The axes are therefore distinct unless a status compatibility record and claim-use preflight explicitly connect them. □

Remark 5.31 (Machine-readable token discipline)**.** Human-readable labels such as "Core verified" or "audit-ready" require machine-readable canonical tokens in the registry. Display punctuation, spacing, or capitalization is not a status proof.

Y.5. Claim Registry

Definition 5.32 (Claim Registry)**.** A *Claim Registry* is a platform component that stores mathematical, operational, interface, bridge, platform, and non-claim statements together with status labels, dependencies, proof records, and artifact references.

Definition 5.33 (Registered claim)**.** A *registered claim* is a claim recorded in the Claim Registry. It must include:

(i) claim identifier;

(ii) claim text;

(iii) claim type;

(iv) status label;

(v) dependencies;

(vi) supporting artifacts;

(vii) responsible agents or roles;

(viii) verification or rejection record, if any;

(ix) demotion or supersession record, if any.

Definition 5.34 (Claim type)**.** A *claim type* may be:

(i) Core theorem;

(ii) Core lemma;

(iii) Core certificate claim;

(iv) operational policy;

(v) interface theorem;

(vi) bridge draft;

(vii) bridge candidate;

(viii) bridge theorem;

(ix) [Spec];

(x) search artifact;

(xi) platform claim;

(xii) non-claim;

(xiii) rejected claim.

Proposition 5.35 (Registration is not validation)**.** *A registered claim is not validated merely by being registered.*

Proof. Registration records the claim and its metadata. Validation requires proof or verification under the declared status criteria. Therefore registration is not validation. □

Remark 5.36 (Registry prevents orphan claims)**.** The Claim Registry helps prevent unsupported or forgotten claims by requiring status labels, dependencies, and artifact references.

Y.5bis. Claim-use preflight and registration sovereignty

Definition 5.37 (Platform claim-use preflight)**.** A *platform claim-use preflight* is the platform-side execution of the CR-v19 claim-use checks before a registered claim is consumed. It checks at least identity, source binding, document role, evidence status, invocation role, conformance status, hypotheses, dependencies, comparison policy, artifact requirements, permitted use, forbidden stronger reading, repair ancestry, and typed failure route.

Proposition 5.38 (Claim Registry entry is not permitted use)**.** *The mere presence of a claim in the Claim Registry does not authorize its use as theorem evidence.*

Proof. A registry entry records metadata and governance fields. Theorem-evidence use additionally requires that the entry's role, status, hypotheses, dependencies, source binding, comparison policy, and permitted-use field pass preflight. Therefore registration alone is insufficient. □

Remark 5.39 (Rejected and non-claim entries remain useful)**.** The platform may store rejected claims, deprecated claims, explicit non-claims, and failed bridge attempts. Their stored presence supports audit and avoids repetition; it does not promote them.

Y.6. Artifact Store

Definition 5.40 (Artifact Store)**.** An *Artifact Store* is a repository for files, datasets, code, logs, formal proof scripts, numerical outputs, barcodes, diagrams, manifests, ledger files, diagnostic files, replay records, and other artifacts referenced by proof objects or certificates.

Definition 5.41 (Artifact entry)**.** An *artifact entry* contains:

(i) artifact identifier;

(ii) artifact type;

(iii) content address or location;

(iv) hash or checksum, if used;

(v) version identifier;

(vi) creator or source;

(vii) associated proof-store entries;

(viii) supported claim, node, or edge references;

(ix) license or access constraints, if relevant;

(x) audit notes.

Definition 5.42 (Artifact binding)**.** An *artifact binding* is a declared relation between a proof-store object and an artifact entry.

Definition 5.43 (Artifact adequacy)**.** *Artifact adequacy* is the property that an artifact supports the specific claim, computation, proof, diagnostic, ledger, or certificate condition for which it is cited.

Proposition 5.44 (Artifact availability is not artifact adequacy)**.** *The availability of an artifact does not imply that it adequately supports the claim that cites it.*

Proof. Availability means the artifact can be retrieved. Adequacy means it supports the relevant claim, proof, computation, diagnostic, ledger, or certificate condition. These are distinct properties. □

Remark 5.45 (Artifact adequacy is audit matter)**.** Whether an artifact adequately supports a claim must be checked during verification or audit.

Y.6bis. Artifact identity, adequacy, and drift preflight

Definition 5.46 (Artifact preflight)**.** An *artifact preflight* checks that an artifact is present, identity-matched, version-correct, readable in the declared environment, adequate for the cited claim, and not superseded for the intended use.

Definition 5.47 (Artifact adequacy failure)**.** An *artifact adequacy failure* occurs when an available artifact does not support the specific proof, computation, diagnostic, ledger, bridge, interpretation, or audit claim for which it is cited.

Proposition 5.48 (Available artifact with failed adequacy is non-evidence)**.** *An artifact that is available but fails adequacy preflight cannot be consumed as supporting evidence for the cited claim.*

Proof. Evidence use requires not only retrieval but also a support relation between artifact and claim. Adequacy failure destroys that support relation. Thus the artifact remains available data but not evidence for that use. □

Remark 5.49 (Artifact drift is semantic drift when adequacy changes)**.** A byte-identical artifact can become inadequate if the cited claim, scope, theorem target, or verifier criterion changes. Conversely, a content change creates a new artifact identity even when the informal description is similar.

Y.7. Versioning and immutability

Definition 5.50 (Version identifier)**.** A *version identifier* is a label or content-derived identifier distinguishing one version of an object or artifact from another.

Definition 5.51 (Immutable object)**.** An *immutable object* is an object whose proof-relevant content is not changed after storage. Any modification creates a new version.

Definition 5.52 (Mutable record)**.** A *mutable record* is a platform record whose metadata, status, or pointers may change over time while preserving history.

Definition 5.53 (Version lineage)**.** A *version lineage* is the recorded history of object versions and their relationships.

Definition 5.54 (Status inheritance)**.** *Status inheritance* is the attempted transfer of a status label from one version of an object to another.

Proposition 5.55 (Version change may require re-verification)**.** *If a stored object's proof-relevant content changes, any verification or audit status depending on the old content does not automatically transfer to the new version.*

Proof. Verification applies to the object content that was checked. A changed version may have different definitions, proofs, dependencies, or artifacts. Therefore status transfer requires justification or re-verification. □

Proposition 5.56 (Status inheritance requires justification)**.** *Status inheritance from one version to another is admissible only if a preservation argument, equivalence check, or re-verification record justifies it.*

Proof. A status label is attached to a verified or reviewed object under declared conditions. If the object changes, those conditions may no longer hold. Thus inheritance requires explicit justification. □

Remark 5.57 (Immutable content, mutable metadata)**.** A robust Proof Store should treat proof-relevant content as immutable while allowing metadata and status records to evolve with full history.

Y.8. Hashing and content identity

Definition 5.58 (Content hash)**.** A *content hash* is a deterministic identifier derived from the content of an object or artifact.

Definition 5.59 (Hash binding)**.** A *hash binding* is a relation between an object identifier and a content hash.

Definition 5.60 (Hash mismatch)**.** A *hash mismatch* occurs when retrieved content does not match the declared hash.

Definition 5.61 (Content identity)**.** *Content identity* means that the retrieved content agrees with the declared stored content according to the platform identity policy.

Proposition 5.62 (Hash match proves identity, not validity)**.** *A content hash match supports content identity, but it does not prove mathematical validity.*

Proof. A hash can show that the retrieved content is the same as the stored or referenced content. It does not establish that the content is true, verified, sufficient, or theorem-level. □

Proposition 5.63 (Hash mismatch invalidates artifact binding until resolved)**.** *A hash mismatch invalidates the affected artifact binding until the mismatch is resolved.*

Proof. If the content no longer matches the declared identity, the proof-store object may be referencing the wrong artifact. The binding is therefore not audit-stable until resolved. □

Remark 5.64 (Hashing controls drift)**.** Hashing is a practical mechanism for controlling artifact drift, but it must be paired with adequacy checks.

Y.8bis. Identity witnesses and semantic preservation

Definition 5.65 (Identity witness)**.** An *identity witness* is a hash match, signature verification, content-addressed reference, immutable storage pointer, or equivalent mechanism establishing that retrieved content is the declared content.

Definition 5.66 (Semantic preservation record)**.** A *semantic preservation record* justifies that a transformation, migration, format conversion, normalization, compression, export, import, or rendering preserves the proof-relevant content needed by a claim.

Proposition 5.67 (Identity witness is not semantic preservation)**.** *An identity witness for one artifact does not prove semantic preservation across a transformed or migrated artifact.*

Proof. Identity confirms sameness of one declared content object. Semantic preservation compares two objects across a transformation. The first supplies no theorem that the transformation preserved the proof-relevant statement. □

Remark 5.68 (PDF, source, and machine-readable sidecars)**.** A PDF, source file, JSON sidecar, CSV register, or rendered image may be stored as an artifact. Equivalence among them is not automatic; it requires a semantic preservation or register-equivalence record.

Y.9. Proof object lifecycle

Definition 5.69 (Proof object lifecycle)**.** The *proof object lifecycle* is the sequence of states through which a proof-related object may pass:

$$\text{created} \longrightarrow \text{stored} \longrightarrow \text{labeled} \longrightarrow \text{linked} \longrightarrow \text{verified} \longrightarrow \text{audited} \longrightarrow \text{promoted or rejected.}$$

Definition 5.70 (Lifecycle event)**.** A *lifecycle event* is a recorded state change or action in the proof object lifecycle.

Definition 5.71 (Lifecycle log)**.** A *lifecycle log* is an ordered record of lifecycle events for a stored object.

Definition 5.72 (Lifecycle authority)**.** A *lifecycle authority* is the agent, tool, or governance rule authorized to perform a lifecycle event.

Proposition 5.73 (Lifecycle progress is not validity monotonicity)**.** *Progress through the proof object lifecycle does not monotonically imply increasing mathematical validity.*

Proof. An object may be stored, linked, and audited but later rejected due to a discovered gap. Lifecycle progress records workflow state, not mathematical truth. □

Remark 5.74 (Lifecycle logs support accountability)**.** Lifecycle logs support accountability by showing when and why status changes occurred.

Y.10. Promotion, demotion, and rejection

Definition 5.75 (Promotion event)**.** A *promotion event* is a recorded status change to a stronger status, such as from candidate to verified fragment, from bridge candidate to bridge theorem, or from Core certificate candidate to Core verified.

Definition 5.76 (Demotion event)**.** A *demotion event* is a recorded status change to a weaker status, such as from verified to blocked, superseded, rejected, deprecated, or incomplete.

Definition 5.77 (Rejection event)**.** A *rejection event* is a recorded status change indicating that an object or claim has failed the relevant criteria.

Definition 5.78 (Promotion evidence)**.** *Promotion evidence* is the proof, verification record, import record, audit record, discharge certificate, or authority record required to justify promotion.

Proposition 5.79 (Promotion requires recorded evidence)**.** *A promotion event is admissible only if the required evidence for the stronger status is recorded.*

Proof. Promotion changes how an object may be used in proof planning, certification, and theorem-level claims. Without evidence, promotion would enable claim inflation. Therefore evidence is required. □

Proposition 5.80 (Demotion preserves audit integrity)**.** *Demotion preserves audit integrity by preventing known-defective or unsupported objects from retaining stronger statuses.*

Proof. If a defect is discovered, the previous status may no longer be justified. Demotion records this change and prevents continued misuse. □

Remark 5.81 (Rejection is useful information)**.** Rejected objects should usually remain stored with their rejection reason. They prevent repeated work and help map failure regions.

Y.10bis. Promotion certificates and non-retroactive repair

Definition 5.82 (Platform promotion certificate)**.** A *platform promotion certificate* is a record authorizing a stronger platform or claim-facing status. It must contain the prior status, target status, evidence, authority scope, CR-v19 preflight result when a claim is consumed, artifact identities, result status pass/reject/undefined, and failure route when applicable.

Definition 5.83 (Non-retroactive platform repair)**.** *Non-retroactive platform repair* is the rule that repairing a stored object, artifact binding, status label, or registry entry creates a new revision or repair record rather than mutating the historical verdict of the prior record.

Proposition 5.84 (Promotion certificate is required for theorem-facing promotion)**.** *A stored item may not be promoted to theorem-facing, Core verified, bridge theorem, interface theorem, or theorem-level proof-object status without a platform promotion certificate.*

Proof. Theorem-facing promotion changes permitted use. It therefore requires evidence, authority, preflight, artifact identity, and failure routing. These are exactly the fields of a platform promotion certificate. □

Proposition 5.85 (Repair does not rewrite prior verdicts)**.** *A platform repair does not retroactively convert a prior failed, undefined, stale, or superseded record into a passing one.*

Proof. Historical audit requires that prior records remain reconstructible at their original content and status. A repair supplies a new record or revision with its own evidence. Thus the previous verdict is not mutated. □

Y.11. AI output management

Definition 5.86 (AI output record)**.** An *AI output record* is a Proof Store or Artifact Store entry recording an AI-generated output and associated metadata.

Definition 5.87 (AI output metadata)**.** AI output metadata may include:

(i) model or agent identifier;

(ii) prompt or task description;

(iii) output content;

(iv) confidence or ranking, if supplied;

(v) output status label;

(vi) human review status;

(vii) verification status;

(viii) artifact references;

(ix) supersession or rejection history, if any.

Definition 5.88 (AI-generated Core-facing record)**.** An *AI-generated Core-facing record* is an AI output claiming to produce or describe Core-facing material, such as:

$$\mathrm{Eval}_{i,W,\tau}(F), \qquad C_{\tau,W}F, \qquad \phi_{i,W,\tau}, \qquad (\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}), \qquad \mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}).$$

Proposition 5.89 (AI output must be status-labeled)**.** *AI output used in the AK workflow must be assigned an explicit status label.*

Proof. By Appendix U, unlabeled agent output is non-verdict by default. To use AI output responsibly in workflow management, its status must be declared. □

Proposition 5.90 (AI storage does not validate AI output)**.** *Storing AI output in the Proof Store or Artifact Store does not validate it.*

Proof. Storage preserves the output and metadata. Validation requires proof, verification, or audit appropriate to the output's intended status. □

Proposition 5.91 (AI-generated Core-facing records require Core verification)**.** *AI-generated Core-facing records cannot support Core certification unless checked under the declared Core verifier-side protocol.*

Proof. AI generation is proposer-side production. Core certification requires verifier-side checks. Therefore AI-generated Core-facing records remain candidates until verified. □

Remark 5.92 (AI outputs are traceable candidates)**.** AI outputs should be treated as traceable candidates, drafts, summaries, bridge sketches, or search artifacts unless and until verified.

Y.11bis. AI retrieval, consensus, and context records

Definition 5.93 (AI context record)**.** An *AI context record* stores the retrieval context, prompt context, model context, selected artifacts, summaries, memory items, or platform snippets used to generate an AI output.

Definition 5.94 (AI consensus record)**.** An *AI consensus record* records agreement among multiple AI agents, model runs, prompts, reviewers, or ranking procedures.

Proposition 5.95 (Retrieved context is not evidence by inclusion)**.** *Including a source, snippet, context item, or memory record in an AI context record does not make it theorem evidence.*

Proof. Context inclusion records what the model could use or did use. Theorem evidence requires verified source binding, admissible claim use, and the appropriate verifier-side or bridge record. Thus inclusion alone is non-verdict. □

Proposition 5.96 (AI consensus is not proof)**.** *AI consensus, ensemble agreement, or repeated generation of the same claim is not proof.*

Proof. Consensus is a proposer-side or review-side signal. It does not supply the mathematical proof, formal verification, Core record, bridge theorem, or artifact adequacy needed for theorem status. □

Remark 5.97 (Context retention supports audit)**.** Retaining AI context is useful because it lets auditors reconstruct how an output was generated. It does not validate the output.

Y.12. Verification records

Definition 5.98 (Verification record)**.** A *verification record* is a Proof Store entry recording the result of a verification action. It must include:

(i) verified object identifier;

(ii) verification criteria;

(iii) verification authority;

(iv) verification result;

(v) environment or configuration;

(vi) artifact references;

(vii) timestamp or version reference;

(viii) failure log, if verification failed;

(ix) scope limitations, if any.

Definition 5.99 (Verification result)**.** A *verification result* is one of:

(i) pass;

(ii) fail;

(iii) inconclusive;

(iv) not applicable;

(v) blocked;

(vi) superseded.

Definition 5.100 (Core verification record)**.** A *Core verification record* is a verification record for a Core-facing object or Core certificate condition. It must specify which Core condition was checked, such as repaired evaluation, admissible representative, eligibility, tower artifact, diagnostic, gate verdict, or ledger budget.

Proposition 5.101 (Verification result is scoped to criteria)**.** *A verification result establishes only what was checked under the declared criteria.*

Proof. Verification criteria define the property being checked. A pass under one criterion does not imply pass under another. Thus the result is scoped. □

Remark 5.102 (Verification record must include environment)**.** For computational or formal verification, the environment and configuration may affect reproducibility. They should be recorded.

Y.12bis. Verification-result mapping and scope control

Definition 5.103 (Verification-to-atomic-status mapping)**.** A verification record maps to the canonical atomic status calculus as follows:

(i) pass maps to pass within the declared criteria;

(ii) fail maps to reject within the declared criteria;

(iii) inconclusive and blocked map to undefined;

(iv) not applicable maps to not_invoked only when a declared scope-exclusion policy positively justifies that the criterion is not invoked; absent such a declaration, it maps to undefined for theorem-facing use;

(v) superseded is unavailable for current theorem evidence unless revalidated.

Definition 5.104 (Verification scope overrun)**.** A *verification scope overrun* occurs when a verification result is used for a statement, theorem target, environment, version, artifact, comparison mode, or authority scope beyond what was checked.

Proposition 5.105 (Verification scope overrun invalidates the use)**.** *A verification result used outside its declared scope is not valid evidence for that use.*

Proof. Verification establishes only the checked criterion in the declared environment and scope. Using it elsewhere changes the claim without supplying a new check. The use is therefore unsupported. □

Remark 5.106 (Formal and computational pass are scoped)**.** A proof-assistant pass, parser pass, build pass, replay pass, or computation pass establishes only the property that the corresponding authority is declared to check.

Y.13. Audit records

Definition 5.107 (Audit record)**.** An *audit record* is a Proof Store entry recording whether a proof object, certificate, artifact set, platform record, map, DAG, or lifecycle history is independently reconstructible.

Definition 5.108 (Audit result)**.** An *audit result* is one of:

(i) audit-pass;

(ii) audit-fail;

(iii) audit-incomplete;

(iv) audit-blocked;

(v) audit-superseded.

Definition 5.109 (Audit scope)**.** The *audit scope* specifies which objects, artifacts, dependencies, ledgers, diagnostics, maps, DAGs, execution records, and proof-store entries were checked.

Proposition 5.110 (Audit pass is not mathematical proof by itself)**.** *An audit-pass result shows reconstructibility within the declared audit scope, but does not by itself prove the underlying mathematical claim.*

Proof. Audit checks reconstructibility and record completeness. Mathematical proof requires the relevant logical or verifier-side conditions to be satisfied. Thus audit pass and proof are distinct. □

Remark 5.111 (Audit is necessary but not sufficient)**.** For high-integrity proof infrastructure, auditability is necessary. It is not sufficient unless the audited proof content is also valid.

Y.13bis. Audit completeness and theorem incompleteness

Definition 5.112 (Audit completeness record)**.** An *audit completeness record* states which artifacts, dependencies, ledger entries, diagnostic records, verification records, environment records, and status transitions were inspected.

Proposition 5.113 (Audit completeness is not theorem completeness)**.** *An audit-complete platform record does not imply that the underlying theorem has been proved.*

Proof. Audit completeness concerns reconstructibility and record coverage. Theorem completeness concerns mathematical discharge of the target obligations. The former can hold while a required theorem, bridge, interpretation, or Core condition remains unproved. □

Remark 5.114 (Audit can certify incompleteness)**.** A successful audit may correctly certify that a record is incomplete, blocked, or undefined. This is a valid audit outcome, not a proof failure of the audit system.

Y.14. Proof Store queries

Definition 5.115 (Proof Store query)**.** A *Proof Store query* is a request to retrieve, filter, compare, rank, or inspect stored objects.

Definition 5.116 (Query result)**.** A *query result* is the list, set, graph, table, or summary returned by a Proof Store query.

Definition 5.117 (Status-filtered query)**.** A *status-filtered query* retrieves objects satisfying declared status constraints.

Definition 5.118 (Dependency query)**.** A *dependency query* retrieves dependency relations, map edges, DAG edges, proof-store links, or artifact bindings relevant to one or more objects.

Proposition 5.119 (Query result is retrieval, not verification)**.** *A Proof Store query result does not verify the returned objects.*

Proof. A query retrieves or organizes existing records. Verification requires checking declared criteria. Retrieval is not checking. □

Remark 5.120 (Search bias)**.** Query ranking may bias what users inspect. It should not bias what is accepted as valid.

Y.15. Platform security and access

Definition 5.121 (Access policy)**.** An *access policy* specifies who or what may read, write, modify, verify, audit, promote, demote, reject, supersede, or delete platform objects.

Definition 5.122 (Write authority)**.** A *write authority* is permission to create or modify Proof Store or Artifact Store entries.

Definition 5.123 (Promotion authority)**.** A *promotion authority* is permission to change an object's status to a stronger status.

Definition 5.124 (Demotion authority)**.** A *demotion authority* is permission to change an object's status to a weaker or safer status.

Definition 5.125 (Verification authority record)**.** A *verification authority record* identifies the agent, tool, or procedure authorized to perform a verification action and its authority scope.

Proposition 5.126 (Write authority is not promotion authority)**.** *Write authority does not imply promotion authority.*

Proof. Writing or modifying records is operational access. Promotion changes validity status. These permissions have different risk profiles and must be separately controlled. □

Remark 5.127 (Access control protects claim status)**.** Access control is not merely cybersecurity. It protects the integrity of mathematical status labels and proof records.

Y.16. Platform manifest entries

Definition 5.128 (Platform manifest entry)**.** A *platform manifest entry* records:

(i) platform component;

(ii) action performed;

(iii) agent or process identifier;

(iv) input objects;

(v) output objects;

(vi) object classes;

(vii) object statuses;

(viii) version identifiers;

(ix) hashes or content identities, if used;

(x) verification or audit records, if any;

(xi) access and authority scope;

(xii) artifact references;

(xiii) lifecycle event identifier.

Definition 5.129 (Proof Store manifest)**.** A *Proof Store manifest* is a collection of platform manifest entries sufficient to reconstruct the lifecycle of a proof object, certificate record, map node, DAG node, bridge program, or stored artifact.

Proposition 5.130 (Platform manifests support reproducibility)**.** *Platform manifests support reproducibility by recording actions, versions, identities, statuses, authorities, and artifact references.*

Proof. Reproducibility requires reconstructing not only the content but also the workflow and status history. Platform manifests record these elements. □

Remark 5.131 (Manifest still needs execution schema)**.** The execution-level schema for replay, environment capture, run configuration, and computational reproducibility is specified in Appendix Z.

Y.17. Proof Store and DAG synchronization

Definition 5.132 (Map-store synchronization)**.** *Map-store synchronization* is the process of keeping Validity Map nodes and Proof Store entries consistent in identifiers, statuses, artifacts, and lifecycle history.

Definition 5.133 (DAG-store synchronization)**.** *DAG-store synchronization* is the process of keeping Certificate DAG nodes, edges, discharge records, and Proof Store entries mutually consistent.

Definition 5.134 (Synchronization failure)**.** A *synchronization failure* occurs when a Proof Store entry, Validity Map node, Certificate DAG node, artifact binding, or status record becomes inconsistent with its counterpart.

Proposition 5.135 (Synchronization failure requires review)**.** *A synchronization failure affecting a necessary proof dependency requires review and may require demotion or quarantine.*

Proof. If the Proof Store and map/DAG records disagree, an auditor cannot reliably reconstruct dependency status. A necessary dependency with inconsistent records is not audit-stable until reviewed. □

Remark 5.136 (Synchronization is governance, not proof)**.** Synchronization keeps records coherent. It does not by itself verify the mathematical content of those records.

Y.17bis. Store synchronization, mirrors, and ledger defects

Definition 5.137 (Mirror store)**.** A *mirror store* is a replicated or synchronized copy of Proof Store or Artifact Store content.

Definition 5.138 (Platform synchronization defect)**.** A *platform synchronization defect* is a discrepancy between store, map, DAG, registry, artifact, or mirror records that affects identity, status, dependency, or permitted use. It is denoted

$$\delta^{Y\text{-}sync}.$$

Definition 5.139 (Platform identity defect)**.** A *platform identity defect* is an unresolved inconsistency in hash binding, version binding, artifact identity, or object identity. It is denoted

$$\delta^{Y\text{-}id}.$$

Definition 5.140 (Platform adequacy defect)**.** A *platform adequacy defect* is an artifact-adequacy, source-binding, or support-relation discrepancy that is consumed by a proof or audit route. It is denoted

$$\delta^{Y\text{-}adequacy}.$$

Proposition 5.141 (Mirroring does not multiply evidence)**.** *Replicating an artifact or proof-store entry across mirror stores does not create independent mathematical evidence.*

Proof. Mirroring copies or synchronizes content. It improves availability and redundancy, but it does not create a new proof, verifier-side record, bridge theorem, or independent artifact adequacy argument. □

Remark 5.142 (Platform defect catalog registration)**.** The symbols $\delta^{Y\text{-}sync}$, $\delta^{Y\text{-}id}$, and $\delta^{Y\text{-}adequacy}$ are local platform-layer defect namespaces until Appendix M registers their relation to any gate-scoped $\mathfrak{d}_{\mathrm{met}}$, local alias, refinement, aggregate, or disjointness convention. The identity and adequacy components are typically status-ledger or qualitative-governance components, not metric-admissible components; identity or adequacy failure routes to undefined or reject and is not repaired by scalar metric slack unless Appendix M and the local certificate record explicitly supply a metric-admissible conversion theorem. They must not be double-counted with artifact, DAG, search, or bridge defects from Appendices V–X.

Y.18. Summary of AI Platform and Proof Store

Theorem 5.143 (Appendix Y platform-store package)**.** *Within the Search / Platform layer, the following package is available.*

(i) The AI Platform supports search, storage, verification, audit, replay, and reproducibility workflows.

(ii) Platform actions are scoped.

(iii) A Proof Store stores proof-related objects but does not certify them by storage.

(iv) Stored Core-facing objects are not Core verified merely by storage.

(v) Object class and object status must be separated.

(vi) Registration is not validation.

(vii) Artifact availability is not artifact adequacy.

(viii) Version changes may require re-verification.

(ix) *Status inheritance requires justification.*

(x) *Hash matches support identity, not validity.*

(xi) *Hash mismatches invalidate affected artifact bindings until resolved.*

(xii) *Lifecycle progress is not validity monotonicity.*

(xiii) *Promotion requires recorded evidence.*

(xiv) *AI output must be status-labeled, and AI storage does not validate it.*

(xv) *AI-generated Core-facing records require Core verification.*

(xvi) *Verification results are scoped to declared criteria.*

(xvii) *Audit pass is not mathematical proof by itself.*

(xviii) *Query results are retrieval, not verification.*

(xix) *Write authority is not promotion authority.*

(xx) *Platform manifests support reproducibility but do not themselves prove mathematical claims.*

(xxi) *Map-store and DAG-store synchronization failures require review when they affect necessary dependencies.*

Proof. Items (i)–(ii) follow from the AI Platform and platform action definitions and Proposition 5.13. Item (iii) is Proposition 5.20. Item (iv) is Proposition 5.21. Item (v) follows from object class and status definitions and Proposition 5.26. Item (vi) is Proposition 5.35. Item (vii) is Proposition 5.44. Item $(viii)$ is Proposition 5.55. Item (ix) is Proposition 5.56. Items (x)–(xi) are Propositions 5.62 and 5.63. Item (xii) is Proposition 5.73. Item $(xiii)$ is Proposition 5.79. Item (xiv) follows from Propositions 5.89 and 5.90. Item (xv) is Proposition 5.91. Item (xvi) is Proposition 5.101. Item $(xvii)$ is Proposition 5.110. Item $(xviii)$ is Proposition 5.119. Item (xix) is Proposition 5.126. Item (xx) follows from Proposition 5.130. Item (xxi) is Proposition 5.135.

□

Y.18bis. v19 platform hardening package

Theorem 5.144 (Appendix Y v19 platform-hardening package)**.** *Within the Search / Platform layer, the following additional v19 hardening package is available.*

(i) *Platform sovereignty prevents storage, indexing, retrieval, hashing, synchronization, display, or access approval from changing verifier-side status.*

(ii) *Platform-to-verifier use requires a conversion record with result pass.*

(iii) *Proof Store status is distinct from Appendix U labels, Chapter 1 roles, and CR-v19 claim-governance status.*

(iv) *Platform claim use requires CR-v19 preflight.*

(v) *Available artifacts must still pass adequacy preflight.*

(vi) *Identity witnesses do not prove semantic preservation across transformed artifacts.*

(vii) *Theorem-facing promotion requires a platform promotion certificate.*

(viii) *Repairs are non-retroactive and create new records or revisions.*

(ix) *AI context inclusion is not theorem evidence.*

(x) *AI consensus is not proof.*

(xi) *Verification results map to pass, reject, undefined, or non-verdict only within their declared criteria.*

(xii) *Verification scope overrun invalidates the attempted use.*

(xiii) *Audit completeness is not theorem completeness.*

(xiv) *Mirror stores do not multiply evidence.*

(xv) *Platform synchronization, identity, and adequacy defects require Appendix M catalog registration before global budget consumption.*

Proof. Items (i)–(ii) are Definition 5.4 and Propositions 5.6 and 5.7. Item (iii) is Definition 5.28 and Proposition 5.30. Item (iv) is Definition 5.37 and Proposition 5.38. Item (v) is Definition 5.46 and Proposition 5.48. Item (vi) is Proposition 5.67. Items (vii)–$(viii)$ are Propositions 5.84 and 5.85. Items (ix)–(x) are Propositions 5.95 and 5.96. Items (xi)–(xii) are Definition 5.103 and Proposition 5.105. Item $(xiii)$ is Proposition 5.113. Item (xiv) is Proposition 5.141. Item (xv) is Remark 5.142. □

Y.19. Explicit exclusions

The following are not asserted in Appendix Y.

- No stored object is certified by storage alone.
- No stored Core-facing object is Core verified by storage alone.
- No registered claim is validated by registration alone.
- No artifact is adequate merely because it is available.
- No hash match proves mathematical validity.
- No version tag proves theorem status.
- No lifecycle progress proves increasing validity.
- No AI output is validated by being stored.
- No AI confidence value is treated as proof.
- No AI-generated Core-facing record is accepted without Core verification.
- No query result is treated as verification.
- No audit pass is treated as mathematical proof by itself.
- No write authority is treated as promotion authority.
- No promotion is allowed without recorded evidence.

- No status inheritance is allowed without justification.
- No Proof Store entry replaces B-Gate^{+}.
- No Proof Store entry replaces the repaired topological condition

$$\mathrm{Eval}_{1,W,\tau}(F) = 0.$$

- No Proof Store entry replaces admissible representative records

$$C_{\tau,W}F.$$

- No Proof Store entry replaces monitored Type IV diagnostics.
- No Proof Store entry replaces tower comparison artifacts

$$\phi_{i,W,\tau}.$$

- No Proof Store entry replaces the budget condition

$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}).$$

- No platform storage event turns a Bridge Program into a bridge theorem.
- No platform record turns a Validity Map into a theorem.
- No map-store or DAG-store synchronization event proves the synchronized mathematical content.
- No platform query, retrieval context, dashboard view, or AI context record is theorem evidence by inclusion alone.
- No AI consensus record is treated as proof.
- No content hash, signature, or immutable pointer proves semantic preservation across transformed artifacts.
- No mirror store or replicated artifact creates independent mathematical evidence.
- No platform repair retroactively rewrites an old verdict.
- No operational access permission discharges theorem-facing obligations.

Y.20. Provenance and status

The material in this appendix depends on:

(i) the Agent Semantics of Appendix U;

(ii) the Hunter / Mapper / Lifter protocols of Appendix V;

(iii) the Bridge Programs of Appendix W;

(iv) the Validity Map and Global Certificate DAG of Appendix X;

(v) the claim taxonomy and manifest discipline of Chapter 1;

(vi) the CR-v19 claim-use and repair-governance register;

(vii) the repaired evaluation discipline of Appendix A;

(viii) the admissible representative discipline of Appendix B;

(ix) the eligible realization regime of Appendix C;

(x) the monitored Type IV diagnostic and tower artifact discipline of Appendix D;

(xi) the Core sovereignty doctrine of Chapter 7;

(xii) the non-claim and interface-boundary discipline of Chapter 8;

(xiii) the manifest schema of Appendix G;

(xiv) the ledger catalog of Appendix M.

Its role is to provide the storage, registry, lifecycle, synchronization, and platform infrastructure required for auditable proof management without confusing storage with certification.

Status labels for Appendix Y. *Search / Platform definitions:* Definitions 5.9, 5.10, 5.11, 5.12, 5.15, 5.16, 5.17, 5.18, 5.19, 5.23, 5.24, 5.25, 5.32, 5.33, 5.34, 5.40, 5.41, 5.42, 5.43, 5.50, 5.51, 5.52, 5.53, 5.54, 5.58, 5.59, 5.60, 5.61, 5.69, 5.70, 5.71, 5.72, 5.75, 5.76, 5.77, 5.78, 5.86, 5.87, 5.88, 5.98, 5.99, 5.100, 5.107, 5.108, 5.109, 5.115, 5.116, 5.117, 5.118, 5.121, 5.122, 5.123, 5.124, 5.125, 5.128, 5.129, 5.132, 5.133, 5.134.

Additional v19.0 hardening definitions: Definitions 5.4, 5.5, 5.28, 5.29, 5.37, 5.46, 5.47, 5.65, 5.66, 5.82, 5.83, 5.93, 5.94, 5.103, 5.104, 5.112, 5.137, 5.138, 5.139, 5.140.

Search / Platform propositions/theorems: Propositions 5.13, 5.20, 5.21, 5.26, 5.35, 5.44, 5.55, 5.56, 5.62, 5.63, 5.73, 5.79, 5.80, 5.89, 5.90, 5.91, 5.101, 5.110, 5.119, 5.126, 5.130, 5.135, Theorem 5.143.

Additional v19.0 hardening propositions/theorems: Propositions 5.6, 5.7, 5.30, 5.38, 5.48, 5.67, 5.84, 5.85, 5.95, 5.96, 5.105, 5.113, 5.141, Theorem 5.144.

Operational cautions: Remarks 5.1, 5.2, 5.3, 5.14, 5.22, 5.27, 5.36, 5.45, 5.57, 5.64, 5.74, 5.81, 5.92, 5.102, 5.111, 5.120, 5.127, 5.131, 5.136.

Additional v19.0 hardening operational cautions: Remarks 5.8, 5.31, 5.39, 5.49, 5.68, 5.97, 5.106, 5.114, 5.142.

Explicit exclusions: Section Y.19.

6 Appendix Z: Execution and Reproducibility Schema

Z.1. Scope and reproducibility status

This appendix defines the execution and reproducibility schema for the AK framework. It belongs to the Search / Platform Appendices. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

Reproducibility supports audit; it is not proof by itself.

A reproducible execution record may allow another auditor to reconstruct a run, recompute artifacts, verify hashes, inspect diagnostics, replay ledger aggregation, replay gate checks, and inspect platform actions. However, reproducibility alone does not establish mathematical validity.

In particular, no execution record, replay log, run configuration, environment hash, container image, output file, reproducibility report, or replay-pass result may replace:

$$B\text{-Gate}^{+}, \quad \mathrm{Eval}_{1,W,\tau}(F) = 0, \quad C_{\tau,W}F, \quad \phi_{i,W,\tau},$$
$$(\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}) = (0,0), \quad \mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_{\tau}(\mathfrak{B}).$$

Remark 6.1 (Search / Platform status)**.** This appendix specifies how runs, artifacts, ledgers, manifests, diagnostics, gate records, and environments should be recorded so that results are reconstructible. It does not create new theorems.

Remark 6.2 (Relation to Appendix Y)**.** Appendix Y defines the AI Platform and Proof Store. The present appendix defines the execution-level schema by which platform entries become replayable and audit-readable.

Remark 6.3 (Final platform appendix)**.** This is the final Search / Platform Appendix. Its role is to close the operational infrastructure around the AK Core while preserving the distinction between execution, reproducibility, auditability, Core certification, and theorem-level proof.

Z.2. Execution run

Definition 6.4 (Execution run)**.** An *execution run* is a recorded instance of an AK workflow, computation, verification, audit, replay, search, lifting, diagnostic calculation, ledger calculation, gate check, or certificate assembly.

Definition 6.5 (Run identifier)**.** A *run identifier* is a unique label assigned to an execution run.

Definition 6.6 (Run scope)**.** The *run scope* specifies what the execution run is intended to do. Examples include:

(i) compute persistence artifacts;

(ii) compute repaired evaluations
$$\mathrm{Eval}_{i,W,\tau}(F);$$

(iii) construct or check admissible representative records
$$C_{\tau,W}F;$$

(iv) compute or check tower comparison artifacts
$$\phi_{i,W,\tau};$$

(v) compute monitored Type IV diagnostics;

(vi) evaluate ledger budgets;

(vii) check B-Gate^{+};

(viii) replay a certificate;

(ix) verify a formal fragment;

(x) assemble a Certificate DAG;

(xi) reproduce a search result;

(xii) audit artifact completeness.

Definition 6.7 (Run authority)**.** A *run authority* is the agent, tool, platform component, or verification procedure authorized to execute or validate a run within its declared scope.

Proposition 6.8 (Run scope limits interpretation)**.** *An execution run establishes only what lies within its declared run scope.*

Proof. A run is configured to perform a declared task. Outputs outside that task are not checked by the run unless explicitly included. Therefore interpretation is limited by the run scope. □

Remark 6.9 (Run success is scoped)**.** A successful run may mean that a script executed, a file was generated, a diagnostic was computed, a gate clause was checked, or a proof assistant accepted a formal fragment. These meanings are distinct and must be recorded separately.

Z.3. Execution schema

Definition 6.10 (Execution schema)**.** An *execution schema* is a structured specification of the inputs, parameters, environment, tools, artifacts, commands, outputs, logs, and verification criteria of an execution run.

Definition 6.11 (run.yaml)**.** A run.yaml file is a human-readable execution schema file describing an AK execution run. It should contain:

(i) run identifier;

(ii) run scope;

(iii) run authority;

(iv) input artifacts;

(v) output artifacts;

(vi) commands or workflow steps;

(vii) environment references;

(viii) parameter values;

(ix) random seeds, if applicable;

(x) repaired evaluation configuration, if applicable;

(xi) representative configuration, if applicable;

(xii) tower artifact configuration, if applicable;

(xiii) ledger configuration;

(xiv) diagnostic configuration;

(xv) gate configuration;

(xvi) verification criteria;

(xvii) expected outputs;

(xviii) artifact hashes or hash policy;

(xix) replay instructions.

Definition 6.12 (Execution parameter)**.** An *execution parameter* is a value controlling an execution run. Examples include:

$$\tau, \qquad W, \qquad i, \qquad \beta, \qquad M(\tau), \qquad s, \qquad \text{grid size}, \qquad \text{random seed}.$$

Definition 6.13 (Parameter binding)**.** A *parameter binding* is a recorded assignment of a value to an execution parameter within a run.

Proposition 6.14 (Execution schema is required for replayability)**.** *A run is replayable only if its execution schema records sufficient information to reconstruct the run.*

Proof. Replay requires reconstructing inputs, parameters, commands, environment, and expected outputs. If these data are missing, the run cannot be reproduced. Therefore a sufficient execution schema is required. □

Remark 6.15 (Schema completeness is task-relative)**.** A schema sufficient for recomputing barcodes may not be sufficient for replaying a proof assistant verification or a Certificate DAG assembly. Schema completeness depends on the run scope.

Z.4. Artifact manifest

Definition 6.16 (Artifact manifest)**.** An *artifact manifest* is a structured list of all artifacts used or produced by an execution run.

Definition 6.17 (Input artifact)**.** An *input artifact* is an artifact required by an execution run.

Definition 6.18 (Output artifact)**.** An *output artifact* is an artifact produced by an execution run.

Definition 6.19 (Intermediate artifact)**.** An *intermediate artifact* is an artifact produced during a run and consumed by later steps of the same or dependent runs.

Definition 6.20 (Artifact manifest entry)**.** An *artifact manifest entry* records:

(i) artifact identifier;

(ii) artifact role: input, output, intermediate, log, reference, or replay artifact;

(iii) artifact type;

(iv) storage location or content address;

(v) version identifier;

(vi) hash or checksum, if used;

(vii) producing run, if applicable;

(viii) consuming runs, if known;

(ix) adequacy notes;

(x) audit notes.

Proposition 6.21 (Artifact manifest supports audit but does not prove adequacy)**.** *An artifact manifest supports auditability, but it does not by itself prove that the artifacts are adequate for the claims that cite them.*

Proof. The manifest records artifact existence, identity, and role. Adequacy requires checking that each artifact supports the relevant claim, computation, diagnostic, ledger entry, gate clause, or certificate condition. Thus manifest presence and artifact adequacy are distinct. □

Remark 6.22 (Completeness versus adequacy)**.** A manifest may be complete but still inadequate if the listed artifacts do not support the intended claims. Both completeness and adequacy must be considered during audit.

Z.5. Environment capture

Definition 6.23 (Execution environment)**.** An *execution environment* is the collection of software, hardware, operating system, dependencies, libraries, proof assistant versions, computation engines, container images, and configuration settings used in an execution run.

Definition 6.24 (Environment capture)**.** *Environment capture* is the process of recording the execution environment sufficiently to support replay.

Definition 6.25 (Environment identifier)**.** An *environment identifier* is a label, hash, container digest, dependency lockfile, proof assistant environment record, or platform record identifying an execution environment.

Definition 6.26 (Environment compatibility)**.** *Environment compatibility* is a declared relation specifying when a different environment may be accepted as replay-equivalent to the original environment.

Proposition 6.27 (Environment capture is required for computational replay)**.** *Computational replay requires environment capture sufficient for the declared run scope.*

Proof. Different environments may produce different outputs due to dependency versions, numerical libraries, proof assistant versions, or system configuration. Replay requires controlling or recording these dependencies. Therefore environment capture is required. □

Remark 6.28 (Environment equality is stronger than needed in some cases)**.** Exact environment equality may be unnecessary for purely formal or deterministic checks if semantic compatibility is proved. However, absent such a proof, environment capture should be conservative.

Z.6. Determinism and randomness

Definition 6.29 (Deterministic run)**.** A run is *deterministic* if the same inputs, parameters, environment, and commands produce the same outputs under the declared replay equivalence.

Definition 6.30 (Randomized run)**.** A run is *randomized* if it depends on random seeds, sampling, stochastic optimization, randomized search, probabilistic algorithms, or nondeterministic execution.

Definition 6.31 (Random seed policy)**.** A *random seed policy* specifies how random seeds are chosen, recorded, replayed, and varied.

Definition 6.32 (Nondeterminism record)**.** A *nondeterminism record* records sources of nondeterminism in a run, including parallel execution, hardware nondeterminism, randomized algorithms, sampling procedures, and external service calls.

Proposition 6.33 (Randomized runs require seed or distribution records)**.** *A randomized run is reproducible only if the seed, sampling policy, nondeterminism record, or probability distribution is recorded sufficiently for the run scope.*

Proof. Without seed, distribution, or nondeterminism records, the random choices cannot be reconstructed or statistically characterized. Therefore reproducibility requires recording them. □

Remark 6.34 (Statistical reproducibility)**.** For randomized search, exact replay may be less important than statistical reproducibility. If so, the statistical criterion must be declared.

Z.7. Repaired evaluation execution files

Definition 6.35 (eval.json)**.** An eval.json file is a structured record of repaired evaluation computations:

$$\mathrm{Eval}_{i,W,\tau}(F) = T_{\tau}^{\mathrm{bar}}\big(\mathsf{W}_W \mathbf{P}_i(F)\big).$$

Definition 6.36 (Evaluation execution entry)**.** An *evaluation execution entry* records:

(i) input object identifier;

(ii) monitored degree i;

(iii) window W;

(iv) threshold τ;

(v) persistence extraction method;

(vi) barcode policy;

(vii) window restriction policy;

(viii) threshold deletion policy;

(ix) resulting repaired profile;

(x) vanishing or nonvanishing result, if checked;

(xi) artifact references.

Proposition 6.37 (Repaired evaluation replay requires evaluation records)**.** *A repaired evaluation is replayable only if the evaluation execution record contains sufficient data to reconstruct*

$$\mathrm{Eval}_{i,W,\tau}(F).$$

Proof. The repaired evaluation depends on the input object, persistence extraction, degree, window, threshold, barcode policy, and threshold deletion policy. Without these records, an auditor cannot reconstruct the evaluation. □

Remark 6.38 (Evaluation summary is not enough)**.** A statement such as

$$\mathrm{Eval}_{1,W,\tau}(F) = 0$$

is not audit-readable unless the supporting evaluation computation or proof can be reconstructed.

Z.8. Ledger execution files

Definition 6.39 (ledger.json)**.** A ledger.json file is a structured record of ledger components, aggregation semantics, gap values, and budget verdicts pass, reject, undefined, or not_invoked used in an execution run.

Definition 6.40 (Ledger execution entry)**.** A *ledger execution entry* records:

(i) ledger semantics;

(ii) component identifiers;

(iii) component values or bounds;

(iv) component sources;

(v) aggregation law;

(vi) declared metric ledger value $\mathfrak{d}_{\mathrm{met}}$;

(vii) scalarized budget $\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}})$;

(viii) protected-profile clearance value $\mathrm{gap}_\tau(\mathfrak{B})$;

(ix) strict comparison convention;

(x) budget verdict pass, reject, undefined, or not_invoked;

(xi) artifact references.

Proposition 6.41 (Ledger replay requires ledger execution records)**.** *A budget comparison is replayable only if the ledger execution record contains sufficient component and aggregation data.*

Proof. Budget replay requires recomputing or checking

$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}).$$

Without component values, aggregation semantics, and gap values, the comparison cannot be reconstructed. □

Remark 6.42 (Ledger file is not budget pass by itself)**.** The existence of ledger.json does not imply budget pass. The recorded comparison must actually satisfy the declared strict inequality.

Z.9. Diagnostic execution files

Definition 6.43 (diagnostics.json)**.** A diagnostics.json file is a structured record of diagnostic computations, including declared tower comparison artifacts, kernel and cokernel defect data, terminal-tame dimension measurements, interior-defect status fields, and Type IV verdicts.

Definition 6.44 (Diagnostic execution entry)**.** A *diagnostic execution entry* records:

(i) diagnostic type;

(ii) monitored degree;

(iii) window;

(iv) threshold;

(v) tower comparison artifact
$$\phi_{i,W,\tau};$$

(vi) source profile
$$\mathrm{ColimProfile}_{i,W,\tau}(F_\bullet);$$

(vii) target profile
$$\mathrm{ApexProfile}_{i,W,\tau}(F_\infty);$$

(viii) kernel defect object;

(ix) cokernel defect object;

(x) tgdim_W-kernel and cokernel dimensions;

(xi) values of μ_{Collapse} and u_{Collapse}, if applicable;

(xii) diagnostic verdict;

(xiii) artifact references.

Proposition 6.45 (Diagnostic replay requires diagnostic execution records)**.** *A monitored Type IV diagnostic is replayable only if the diagnostic execution record contains sufficient data to reconstruct the tower comparison artifact and defect calculations.*

Proof. Type IV diagnostics are defined by kernel and cokernel defect objects of the declared tower comparison artifact
$$\phi_{i,W,\tau}.$$
Replay requires reconstructing this artifact and its defect dimensions. Therefore the diagnostic execution record must include the required data. □

Remark 6.46 (Diagnostic summary is not enough)**.** A line saying
$$(\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}) = (0, 0)$$
is not enough for audit unless the supporting diagnostic computation can be reconstructed.

Z.10. Gate execution files

Definition 6.47 (gate.json)**.** A gate.json file is a structured record of Core gate checks, including B-Gate$^+$, admissibility conditions, representative requirements, diagnostics, ledger conditions, and gate verdicts.

Definition 6.48 (Gate execution entry)**.** A *gate execution entry* records:

(i) gate identifier;

(ii) gate definition;

(iii) input object;

(iv) window and threshold;

(v) checked clauses;

(vi) repaired evaluation clause, if used;

(vii) representative clause, if used;

(viii) eligibility clause, if used;

(ix) tower diagnostic clause, if used;

(x) ledger clause, if used;

(xi) clause-level results;

(xii) final gate verdict;

(xiii) verifying authority;

(xiv) artifact references.

Proposition 6.49 (Gate verdict requires clause-level support)**.** *A gate verdict is audit-readable only if its required clauses are recorded with sufficient support.*

Proof. A gate verdict is determined by its clauses. Without clause-level records, an auditor cannot reconstruct why the verdict was assigned. Therefore clause-level support is required. □

Remark 6.50 (Gate file is not self-certifying)**.** The existence of gate.json does not by itself prove that the gate passed. Its contents must support the verdict.

Z.11. Replay

Definition 6.51 (Replay)**.** A *replay* is an attempt to reproduce an execution run from its recorded schema, environment, inputs, parameters, commands, and artifacts.

Definition 6.52 (Replay result)**.** A *replay result* is one of:

(i) replay-pass;

(ii) replay-fail;

(iii) replay-incomplete;

(iv) replay-divergent;

(v) replay-not-applicable;

(vi) replay-blocked.

Definition 6.53 (Replay equivalence)**.** *Replay equivalence* is a declared relation specifying when replay output is considered equivalent to the original output. It may require exact equality, hash equality, numerical tolerance, proof assistant acceptance, diagnostic equality, ledger equality, gate verdict equality, or semantic equivalence.

Definition 6.54 (Replay witness)**.** A *replay witness* is an artifact, log, hash, proof assistant output, diagnostic output, ledger output, or gate output supporting a replay result.

Proposition 6.55 (Replay pass is not theorem proof by itself)**.** *A replay-pass result shows reproducibility under the declared replay criterion, but it does not by itself prove the underlying mathematical claim.*

Proof. Replay checks whether a run can be reproduced. Mathematical proof requires the relevant verifier-side or logical conditions. Reproduction and proof are distinct. □

Remark 6.56 (Replay criterion must be declared)**.** For numerical runs, exact equality may be too strict or too weak depending on the task. For formal runs, proof assistant acceptance may be appropriate. For Core diagnostics, diagnostic equality or certified semantic equivalence may be required. The replay equivalence criterion must be declared.

Z.12. Reproducibility levels

Definition 6.57 (Reproducibility level)**.** A *reproducibility level* classifies how much of an execution run can be reconstructed.

Definition 6.58 (Level R0)**.** *Level R0* means that the result is stated but no sufficient execution schema or artifacts are available.

Definition 6.59 (Level R1)**.** *Level R1* means that artifacts are referenced, but replay is not possible from the record.

Definition 6.60 (Level R2)**.** *Level R2* means that inputs, parameters, and outputs are recorded, but the environment or commands are incomplete.

Definition 6.61 (Level R3)**.** *Level R3* means that the run can be replayed in a compatible environment with declared tolerances.

Definition 6.62 (Level R4)**.** *Level R4* means that the run can be replayed exactly or content-identically using captured environment, artifacts, commands, and hashes.

Definition 6.63 (Level R5)**.** *Level R5* means that the result is reproducible and independently audited, with all required proof-store, artifact-store, evaluation, ledger, diagnostic, gate, map, DAG, and replay records linked.

Proposition 6.64 (Higher reproducibility level does not imply theorem validity)**.** *A higher reproducibility level improves auditability but does not by itself imply theorem validity.*

Proof. Reproducibility concerns reconstruction of runs and artifacts. Theorem validity concerns the correctness of mathematical claims. A reproducible invalid computation remains invalid. □

Remark 6.65 (Preferred level)**.** For high-stakes Core certificates, Level R5 is preferred. However, even R5 must be paired with valid proof content.

Z.13. Formal verification replay

Definition 6.66 (Formal verification replay)**.** A *formal verification replay* is a replay in which proof assistant code or formal fragments are checked again under a recorded proof assistant environment.

Definition 6.67 (Formal environment)**.** A *formal environment* records:

(i) proof assistant name and version;

(ii) libraries and versions;

(iii) axioms used;

(iv) definitions imported;

(v) build commands;

(vi) compilation options;

(vii) formal artifact hashes.

Definition 6.68 (Formal replay artifact)**.** A *formal replay artifact* is a proof assistant output, build log, checked theorem object, compiled file, or formal verification report produced during formal replay.

Proposition 6.69 (Formal replay is scoped to formal statements)**.** *Formal verification replay establishes only the checked formal statements in the recorded formal environment.*

Proof. A proof assistant checks formal statements relative to its environment. It does not automatically prove informal interpretations or external problem-specific claims. Those require interpretation theorems or interface bridges. □

Remark 6.70 (Formal replay and interpretation)**.** A formally replayed theorem may still need an interpretation layer to connect it to the intended AK or problem-specific claim.

Z.14. Numerical reproducibility

Definition 6.71 (Numerical reproducibility)**.** *Numerical reproducibility* is the ability to reproduce numerical outputs within declared tolerances, methods, and environments.

Definition 6.72 (Numerical tolerance)**.** A *numerical tolerance* is a declared acceptable discrepancy between original and replayed numerical outputs.

Definition 6.73 (Numerical stability record)**.** A *numerical stability record* documents sensitivity of numerical outputs to grid, time step, precision, algorithmic choices, and perturbations.

Definition 6.74 (Continuum interpretation record)**.** A *continuum interpretation record* documents the bridge, theorem, or [Spec] status by which numerical or discretized outputs are interpreted as statements about a continuum object.

Proposition 6.75 (Numerical reproducibility is not numerical validity)**.** *Numerical reproducibility does not by itself establish numerical validity or mathematical proof.*

Proof. A numerical computation may be reproducible and still approximate the wrong object, use an unstable method, or fail to justify continuum interpretation. Numerical validity requires additional error, stability, and soundness analysis. □

Remark 6.76 (Relation to Appendices I and J)**.** Appendices I and J provide discretization and measurement stability disciplines. This appendix records the execution schema needed to replay such analyses.

Z.15. Reproducibility manifest

Definition 6.77 (Reproducibility manifest)**.** A *reproducibility manifest* is a top-level record connecting run schema, artifact manifest, environment capture, repaired evaluation records, ledger records, diagnostic records, gate records, replay results, audit records, and proof-store links.

Definition 6.78 (Reproducibility manifest entry)**.** A *reproducibility manifest entry* records:

(i) run identifier;

(ii) run scope;

(iii) $\mathrm{run.yaml}$ reference;

(iv) artifact manifest reference;

(v) environment reference;

(vi) $\mathrm{eval.json}$ reference, if applicable;

(vii) $\mathrm{ledger.json}$ reference, if applicable;

(viii) $\mathrm{diagnostics.json}$ reference, if applicable;

(ix) $\mathrm{gate.json}$ reference, if applicable;

(x) replay result;

(xi) reproducibility level;

(xii) audit result;

(xiii) proof-store links;

(xiv) Validity Map or Certificate DAG links, if applicable.

Proposition 6.79 (Reproducibility manifest supports audit)**.** *A reproducibility manifest supports audit by linking the records needed to reconstruct and inspect an execution run.*

Proof. Audit requires access to the run schema, artifacts, environment, repaired evaluations, ledger, diagnostics, gates, replay results, and proof-store records. The reproducibility manifest links these records. □

Remark 6.80 (Manifest is not proof)**.** A reproducibility manifest may make a run audit-readable. It does not by itself establish that the mathematical conclusion is valid.

Z.16. Reproducibility failure modes

Definition 6.81 (Reproducibility failure)**.** A *reproducibility failure* occurs when an execution run cannot be reconstructed, replayed, or audited at the claimed reproducibility level.

Definition 6.82 (Missing artifact failure)**.** A *missing artifact failure* occurs when a required artifact is absent or inaccessible.

Definition 6.83 (Environment failure)**.** An *environment failure* occurs when the recorded environment is insufficient, unavailable, or incompatible with replay.

Definition 6.84 (Parameter failure)**.** A *parameter failure* occurs when required execution parameters are missing, ambiguous, or inconsistent.

Definition 6.85 (Replay divergence)**.** *Replay divergence* occurs when replayed outputs fail the declared replay equivalence criterion.

Definition 6.86 (Manifest inconsistency)**.** *Manifest inconsistency* occurs when run schema, artifact manifest, evaluation records, ledger records, diagnostic records, gate records, or proof-store entries disagree.

Definition 6.87 (Evaluation replay failure)**.** An *evaluation replay failure* occurs when

$$\mathrm{Eval}_{i,W,\tau}(F)$$

cannot be reconstructed or does not satisfy the declared replay equivalence.

Definition 6.88 (Diagnostic replay failure)**.** A *diagnostic replay failure* occurs when monitored diagnostic values or tower artifact computations cannot be reconstructed or do not satisfy the declared replay equivalence.

Proposition 6.89 (Reproducibility failure requires status review)**.** *If reproducibility failure affects a necessary dependency, the corresponding certificate or proof object requires status review and possible demotion.*

Proof. A necessary dependency must be audit-readable and reconstructible at the claimed level. If reproducibility fails, the support for that dependency is weakened. Therefore status review and possible demotion are required. □

Remark 6.90 (Reproducibility failure is not always mathematical failure)**.** A reproducibility failure may indicate missing infrastructure rather than false mathematics. However, until repaired, the affected proof object is not audit-stable.

Z.17. Execution-to-certificate boundary

Definition 6.91 (Execution-to-certificate boundary)**.** The *execution-to-certificate boundary* is the boundary between producing execution artifacts and establishing certificate status.

Definition 6.92 (Execution-supported certificate)**.** An *execution-supported certificate* is a certificate whose computational, diagnostic, ledger, gate, or artifact-dependent components are supported by adequate execution and reproducibility records.

Proposition 6.93 (Execution success does not imply certificate success)**.** *Successful execution of a workflow does not imply that the resulting object is a valid certificate.*

Proof. Execution success may mean that commands ran and outputs were generated. Certificate success requires that the outputs satisfy the Core gate, repaired evaluation, diagnostic, ledger, manifest, and audit conditions. These are distinct. □

Proposition 6.94 (Certificate success requires execution support when computation is used)**.** *If a certificate relies on computed artifacts, then those computations must be supported by adequate execution and reproducibility records.*

Proof. Computed artifacts must be reconstructible and auditable when used as certificate evidence. Execution and reproducibility records provide the required support. □

Remark 6.95 (Boundary discipline)**.** Execution infrastructure should make certificate checking easier. It should never be confused with certificate checking itself.

Z.17bis. Additional v19.0 execution hardening

Definition 6.96 (Execution sovereignty)**.** *Execution sovereignty* is the rule that execution records, replay logs, environment captures, and reproducibility manifests may support audit but may not override verifier-side Core conditions, bridge-theorem requirements, interpretation theorems, or claim-use permissions.

Definition 6.97 (Execution-to-verifier conversion record)**.** An *execution-to-verifier conversion record* is a record converting an execution output into verifier-side data for a declared use. It must include:

(i) source run identifier;

(ii) source execution schema and artifact manifest;

(iii) target verifier-side condition;

(iv) authority scope;

(v) conversion rule;

(vi) result status pass, reject, or undefined;

(vii) typed failure route when the result is reject or undefined;

(viii) immutable artifact references;

(ix) repair ancestry, if any;

(x) claim-use preflight record when the converted data are used in a theorem-facing or Core-facing claim.

Definition 6.98 (Execution status axis)**.** The *execution status axis* records operational outcomes such as executed, failed, replayed, divergent, blocked, or superseded. It is an additional workflow field and does not replace Appendix U agent-output status labels, Appendix Y Proof Store statuses, Chapter 1 claim taxonomy, CR-v19 claim-use status, or Core verifier verdicts.

Definition 6.99 (Execution claim-use preflight)**.** An *execution claim-use preflight* is the check that an execution output is permitted to be used for the specific target claim, target gate, target diagnostic, target ledger comparison, target bridge, or target proof object. It records the intended use, required status, available evidence, missing evidence, and result pass, reject, or undefined.

Definition 6.100 (Run identity witness)**.** A *run identity witness* is a content hash, environment digest, command transcript, signed log, or equivalent record showing that a replayed or referenced run is the intended run. It witnesses identity, not mathematical validity.

Definition 6.101 (Execution semantic preservation record)**.** An *execution semantic preservation record* states why replacing one environment, implementation, file format, numerical backend, proof-assistant version, or replay tolerance by another preserves the verifier-relevant semantics of the run.

Definition 6.102 (Replay atomic status mapping)**.** *Replay atomic status mapping* maps replay results to theorem-facing atomic statuses. A replay-pass may support reconstructibility only within its declared replay criterion. A replay-fail or replay-divergent result routes to reject or undefined for the affected execution support. A replay-incomplete or replay-blocked result maps to undefined for the affected execution support until the missing replay data or blocking condition is repaired and rechecked. A replay-not-applicable result maps to not_invoked only when a declared scope-exclusion policy shows that the replay criterion is irrelevant; without such a policy, theorem-facing use is undefined.

Definition 6.103 (Execution scope overrun)**.** *Execution scope overrun* occurs when a run result, replay result, formal replay, numerical replay, environment capture, or reproducibility manifest is used beyond its declared run scope or authority scope.

Definition 6.104 (Execution adequacy defect)**.** An *execution adequacy defect*, denoted locally by $\delta^{Z\text{-}adequacy}$, records a failure or uncertainty in whether execution records are adequate for the claim that cites them. It is typically a status-ledger or qualitative-governance component, not a metric-admissible bottleneck component.

Definition 6.105 (Environment compatibility defect)**.** An *environment compatibility defect*, denoted locally by $\delta^{Z\text{-}env}$, records a failure, uncertainty, or unproved substitution between the original environment and a replay environment. It cannot be repaired by metric budget slack unless Appendix M registers a metric-admissible interpretation with supporting bounds.

Definition 6.106 (Replay divergence defect)**.** A *replay divergence defect*, denoted locally by $\delta^{Z\text{-}replay}$, records divergence between original and replayed outputs under the declared replay equivalence. For Core-facing use, unresolved replay divergence routes to reject or undefined rather than to a silent tolerance waiver.

Definition 6.107 (Execution Type IV replay record)**.** An *execution Type IV replay record* is a replay record for Type IV diagnostics containing the declared tower comparison artifact

$$\begin{gathered}\phi_{i,W,\tau}: \\ \mathrm{ColimProfile}_{i,W,\tau}(F_\bullet) \\ \longrightarrow \\ \mathrm{ApexProfile}_{i,W,\tau}(F_\infty),\end{gathered}$$

Besides the displayed artifact, the record includes:

(i) terminal-tameness support;

(ii) kernel and cokernel defect objects;

(iii) tgdim_W computations;

(iv) the interior-defect status field;

(v) the four-state diagnostic result.

Proposition 6.108 (Execution records do not override verifier conditions)**.** *No execution record, replay log, environment capture, or reproducibility manifest overrides a missing or failed verifier-side condition.*

Proof. Execution artifacts are infrastructure records. Verifier-side conditions are the declared mathematical and Core-facing criteria for certification. By execution sovereignty, infrastructure support cannot replace the criteria it is meant to support. □

Proposition 6.109 (Verifier-side use requires conversion)**.** *An execution output may be used as verifier-side evidence only through an execution-to-verifier conversion record with an appropriate pass status for the intended use.*

Proof. The same output file may be a log, a diagnostic support artifact, a ledger component source, or merely a failed run artifact. The conversion record declares which use is intended and whether the evidence supports that use. Without it, the output remains execution-side data. □

Proposition 6.110 (Execution status is not claim status)**.** *Execution status does not determine mathematical claim status.*

Proof. Execution status records workflow outcomes such as execution, replay, divergence, or blockage. Claim status requires proof, verification, import, discharge, or explicit non-claim governance. The two axes are distinct. □

Proposition 6.111 (Run identity is not semantic preservation)**.** *A run identity witness establishes identity under the declared identity policy, but it does not establish semantic preservation across environments, formats, or interpretations.*

Proof. Identity records that the referenced content or run is the intended one. Semantic preservation is a statement about meaning under replacement or interpretation. The latter requires a separate preservation record or theorem. □

Proposition 6.112 (Replay pass remains scoped)**.** *A replay-pass result supports only the declared replay criterion and does not discharge theorem, bridge, interpretation, diagnostic, or metric obligations outside that criterion.*

Proof. Replay equivalence is declared by the run scope and replay criterion. A pass under one criterion does not prove obligations not checked by that criterion. □

Proposition 6.113 (Replay-not-applicable requires scope exclusion)**.** *A replay-not-applicable result may be treated as* not_invoked *only under a declared scope-exclusion policy; otherwise theorem-facing use is* undefined*.*

Proof. A clause is not safely omitted merely because a replay report says it was not applicable. The omission must be justified by the scope policy. Without such justification, the missing check remains undefined for theorem-facing use. □

Proposition 6.114 (Execution scope overrun invalidates use)**.** *If an execution output is used beyond its declared scope, that use is invalid until a new scope, conversion record, or verification record justifies it.*

Proof. The run authority and execution schema define what the run checked or produced. Use outside that scope lacks authorization and verifier-side interpretation. □

Proposition 6.115 (Execution defects are not automatically metric defects)**.** *The symbols* $\delta^{Z\text{-}adequacy}$*,* $\delta^{Z\text{-}env}$*, and* $\delta^{Z\text{-}replay}$ *are not automatically metric-admissible components of* $\mathfrak{d}_{\mathrm{met}}$*.*

Proof. Adequacy, environment compatibility, and replay divergence may be qualitative or governance failures. Metric budget slack can repair only declared metric-admissible deviations with supporting bounds and Appendix M registration. □

Proposition 6.116 (Type IV replay summary is insufficient)**.** *A replayed line reporting*

$$(\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}) = (0, 0)$$

is insufficient for Type IV certification without the full execution Type IV replay record.

Proof. Type IV certification depends on the declared tower comparison artifact, terminal-tameness, kernel and cokernel defect objects, dimension calculations, and interior-defect status. A numerical pair alone does not provide those data. □

Theorem 6.117 (Appendix Z v19 execution hardening package)**.** *The v19 execution hardening layer supplies the following safeguards.*

(i) Execution sovereignty separates infrastructure records from verifier conditions.

(ii) Execution-to-verifier conversion records are required for theorem-facing use.

(iii) Execution status is distinct from claim status.

(iv) Claim-use preflight is required when execution outputs support claims.

(v) Run identity witnesses do not establish semantic preservation.

(vi) Replay pass is scoped to the declared replay criterion.

(vii) Replay-not-applicable requires scope exclusion before it may be treated as not invoked.

(viii) Execution scope overrun invalidates the unsupported use.

(ix) Execution adequacy, environment compatibility, and replay divergence defects are not automatically metric defects.

(x) Type IV replay requires a full typed replay record, not just a zero pair.

Proof. Items (i)–(ii) are Propositions 6.108 and 6.109. Item (iii) is Proposition 6.110. Item (iv) follows from Definition 6.99. Item (v) is Proposition 6.111. Item (vi) is Proposition 6.112. Item (vii) is Proposition 6.113. Item $(viii)$ is Proposition 6.114. Item (ix) is Proposition 6.115. Item (x) is Proposition 6.116. □

Remark 6.118 (Execution noninterference)**.** Execution infrastructure may preserve, replay, and expose evidence. It does not add theorem-level force beyond the checked mathematical or verifier-side content.

Remark 6.119 (Status token discipline)**.** Execution tokens such as replay-pass, replay-fail, and replay-not-applicable should be translated into the governing pass/reject/undefined/not-invoked vocabulary only through an explicit mapping record.

Remark 6.120 (Execution defect catalog registration)**.** The local symbols $\delta^{\text{Z-adequacy}}$, $\delta^{\text{Z-env}}$, and $\delta^{\text{Z-replay}}$ require Appendix M registration before global use. Their relations to Part V symbols such as $\delta^{\text{Y-adequacy}}$, $\delta^{\text{Y-id}}$, $\delta^{\text{X-art}}$, and $\delta^{\text{X-asm}}$ must be declared by alias, refinement, aggregation, or disjointness; otherwise double counting or hidden mismatch is possible.

Remark 6.121 (PDF, image, and source sidecars)**.** A rendered PDF, page image, or screenshot is not automatically equivalent to the source artifact used for verification. If execution depends on source files, sidecar source identity and source-to-render preservation records should be stored.

Z.18. Closure of Part V

Theorem 6.122 (Part V closure principle)**.** *The Search / Platform Appendices U–Z provide agent, search, bridge-program, map, proof-store, and execution infrastructure around the AK Core. They do not enlarge the Core mathematical package, do not replace Core gates, do not replace repaired evaluations, do not replace admissible representatives, do not replace tower comparison artifacts, do not replace monitored Type IV diagnostics, and do not alter the strict scalarized audit budget condition.*

Proof. Appendix U separates proposer-side and verifier-side agent semantics. Appendix V makes search protocols candidate-generating rather than certifying. Appendix W makes Bridge Programs proof-management objects rather than bridge theorems. Appendix X makes Validity Maps and Certificate DAGs navigation and dependency structures rather than proof by existence. Appendix Y makes Proof Store storage non-certifying. The present appendix makes execution and reproducibility audit-supporting rather than proof-generating and adds execution-to-verifier conversion safeguards. Together, these appendices preserve Core sovereignty in the sense of Chapter 7. □

Remark 6.123 (Meaning of Part V closure)**.** Part V closes the infrastructure layer. It explains how agents, search, bridges, maps, proof stores, and execution records may support proof work without becoming proof merely by being present.

Z.19. Final non-confusion principle

Theorem 6.124 (Final non-confusion principle)**.** *Across the AK framework, the following distinctions are mandatory:*

$$\textit{search eqverification}, \quad \textit{storage eqcertification}, \quad \textit{execution eqproof},$$
$$\textit{reproducibility eqvalidity}, \quad \textit{Core pass eqexternal theorem without a bridge},$$
$$[\text{Spec}]\ \textit{eqtheorem}.$$

Proof. The first distinction follows from Appendices U and V. The second follows from Appendix Y. The third and fourth follow from the present appendix. The fifth follows from the Problem Interface discipline of Chapter 8 and the soundness-bridge requirements of the Problem Interface Appendices. The sixth follows from the non-claim and boundary discipline of Chapter 8. □

Remark 6.125 (Final role of Appendix Z)**.** Appendix Z is the final guard against infrastructure inflation. It ensures that even a fully reproducible, well-stored, agent-assisted, platform-managed, search-discovered, DAG-organized result must still pass the relevant mathematical and verifier-side conditions before it can be treated as proof.

Z.20. Summary of execution and reproducibility schema

Theorem 6.126 (Appendix Z execution-reproducibility package)**.** *Within the Search / Platform layer, the following package is available.*

(i) Execution runs must have declared scope.

(ii) Execution schemas such as run.yaml *are required for replayability.*

(iii) Artifact manifests support audit but do not prove artifact adequacy.

(iv) Environment capture is required for computational replay.

(v) Randomized runs require seed, distribution, or nondeterminism records.

(vi) Repaired evaluation replay requires evaluation execution records.

(vii) Ledger replay requires ledger execution records.

(viii) Diagnostic replay requires diagnostic execution records based on declared tower comparison artifacts.

(ix) *Gate verdicts require clause-level support.*

(x) *Replay pass is not theorem proof by itself.*

(xi) *Higher reproducibility level improves auditability but does not imply theorem validity.*

(xii) *Formal verification replay is scoped to formal statements.*

(xiii) *Numerical reproducibility is not numerical validity.*

(xiv) *Reproducibility manifests support audit but do not prove mathematical claims.*

(xv) *Reproducibility failure requires status review when necessary dependencies are affected.*

(xvi) *Execution success does not imply certificate success.*

(xvii) *Certificates relying on computed artifacts require adequate execution support.*

(xviii) *v19 execution hardening requires conversion records, claim-use preflight, scoped replay interpretation, and full Type IV replay records.*

(xix) *Part V infrastructure does not enlarge the AK Core.*

(xx) *The final non-confusion principle separates search, storage, execution, reproducibility, verification, Core pass, external theorem status, and* [Spec] *status.*

Proof. Item (i) follows from the definitions of execution run and run scope. Item (ii) is Proposition 6.14. Item (iii) is Proposition 6.21. Item (iv) is Proposition 6.27. Item (v) is Proposition 6.33. Item (vi) is Proposition 6.37. Item (vii) is Proposition 6.41. Item $(viii)$ is Proposition 6.45. Item (ix) is Proposition 6.49. Item (x) is Proposition 6.55. Item (xi) is Proposition 6.64. Item (xii) is Proposition 6.69. Item $(xiii)$ is Proposition 6.75. Item (xiv) follows from Proposition 6.79 and Remark 6.80. Item (xv) is Proposition 6.89. Item (xvi) is Proposition 6.93. Item $(xvii)$ is Proposition 6.94. Item $(xviii)$ is Theorem 6.117. Item (xix) is Theorem 6.122. Item (xx) is Theorem 6.124. □

Z.21. Explicit exclusions

The following are not asserted in Appendix Z.

- No execution run is treated as proof by itself.
- No run.yaml file is treated as certificate by itself.
- No artifact manifest proves artifact adequacy.
- No environment hash proves mathematical validity.
- No random seed proves correctness.
- No eval.json file implies repaired evaluation correctness by existence alone.
- No ledger.json file implies budget pass by existence alone.
- No diagnostics.json file implies Type IV cleanliness by existence alone.
- No gate.json file implies gate pass by existence alone.

- No replay-pass result proves a theorem by itself.
- No high reproducibility level implies theorem validity.
- No formal replay proves informal interpretation without an interpretation theorem.
- No numerical reproducibility proves continuum validity.
- No reproducibility manifest proves a mathematical claim by itself.
- No execution success implies certificate success.
- No reproducibility infrastructure replaces B-Gate^{+}.
- No reproducibility infrastructure replaces the repaired topological condition

$$\mathrm{Eval}_{1,W,\tau}(F) = 0.$$

- No reproducibility infrastructure replaces admissible representative records

$$C_{\tau,W}F.$$

- No reproducibility infrastructure replaces monitored Type IV diagnostics.
- No reproducibility infrastructure replaces tower comparison artifacts

$$\phi_{i,W,\tau}.$$

- No reproducibility infrastructure replaces the budget condition

$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}).$$

- No Search / Platform appendix upgrades a [Spec] statement into a theorem.
- No Search / Platform appendix converts Core pass into an external theorem without a proved bridge.
- No execution output is used as verifier-side evidence without an execution-to-verifier conversion record.
- No replay-not-applicable result is treated as not invoked without a declared scope-exclusion policy.
- No run identity witness proves semantic preservation.
- No Type IV replay zero pair replaces the full typed diagnostic replay record.

Z.22. Provenance and status

The material in this appendix depends on:

(i) the Agent Semantics of Appendix U;

(ii) the Hunter / Mapper / Lifter protocols of Appendix V;

(iii) the Bridge Programs of Appendix W;

(iv) the Validity Map and Global Certificate DAG of Appendix X;

(v) the AI Platform and Proof Store of Appendix Y;

(vi) the claim taxonomy and Minimal Manifest discipline of Chapter 1;

(vii) the repaired evaluation discipline of Appendix A;

(viii) the admissible representative discipline of Appendix B;

(ix) the eligible realization regime of Appendix C;

(x) the monitored Type IV diagnostic and tower artifact discipline of Appendix D;

(xi) the manifest schema of Appendix G;

(xii) the ledger catalog of Appendix M;

(xiii) the Core sovereignty and typed-audit doctrine of Chapter 7;

(xiv) the non-claim, boundary, and interface protocol of Chapter 8;

(xv) the CR-v19 claim-use and status-transition registry.

Its role is to close the Search / Platform layer by specifying how execution records, artifacts, environments, repaired evaluations, ledgers, diagnostics, gates, and replay results are made reproducible without being mistaken for proofs.

Status labels for Appendix Z. *Search / Platform definitions:* Definitions 6.4, 6.5, 6.6, 6.7, 6.10, 6.11, 6.12, 6.13, 6.16, 6.17, 6.18, 6.19, 6.20, 6.23, 6.24, 6.25, 6.26, 6.29, 6.30, 6.31, 6.32, 6.35, 6.36, 6.39, 6.40, 6.43, 6.44, 6.47, 6.48, 6.51, 6.52, 6.53, 6.54, 6.57, 6.58, 6.59, 6.60, 6.61, 6.62, 6.63, 6.66, 6.67, 6.68, 6.71, 6.72, 6.73, 6.74, 6.77, 6.78, 6.81, 6.82, 6.83, 6.84, 6.85, 6.86, 6.87, 6.88, 6.91, 6.92.

Search / Platform propositions/theorems: Propositions 6.8, 6.14, 6.21, 6.27, 6.33, 6.37, 6.41, 6.45, 6.49, 6.55, 6.64, 6.69, 6.75, 6.79, 6.89, 6.93, 6.94, Theorems 6.122, 6.124, 6.126.

Operational cautions: Remarks 6.1, 6.2, 6.3, 6.9, 6.15, 6.22, 6.28, 6.34, 6.38, 6.42, 6.46, 6.50, 6.56, 6.65, 6.70, 6.76, 6.80, 6.90, 6.95, 6.123, 6.125.

Additional v19.0 hardening definitions: Definitions 6.96, 6.97, 6.98, 6.99, 6.100, 6.101, 6.102, 6.103, 6.104, 6.105, 6.106, 6.107.

Additional v19.0 hardening propositions/theorems: Propositions 6.108, 6.109, 6.110, 6.111, 6.112, 6.113, 6.114, 6.115, 6.116, Theorem 6.117.

Additional v19.0 hardening operational cautions: Remarks 6.118, 6.119, 6.120, 6.121.

Explicit exclusions: Section Z.21.

Part closure: Theorem 6.122.

Final non-confusion principle: Theorem 6.124.